

Secure and Compliant Machine Learning Security Pipeline

An End-to-End Privacy-Aware, Fairness-Assessed, Securely Deployed Fraud Detection System

Technical Project Report

University / Institution: [Insert institution name]

Program: Machine Learning / Cybersecurity Graduation Project

Team Members: Nour Eldin Ahmed (ML Engineering & Risk Analysis, Milestone 2),
Yara [surname placeholder] (Data Security, Milestone 1), Ali Yasser (Secure Deployment,
Milestone 3),

Farida Elgharbawy (Compliance QA, Milestone 4), Youssef Tarek (MLOps, Milestone 5)

Supervisor: [Insert supervisor name]

Submission Date: [Insert submission date]

Table of Contents

Table of Contents.....	2
Declaration of Originality.....	11
Acknowledgements	11
Executive Summary.....	11
Abstract.....	12
Keywords	12
Table of Contents.....	13
List of Figures	13
List of Tables	13
List of Abbreviations	13
Chapter 1 — Introduction.....	15
1.1 Background	15
1.2 Motivation	15
1.3 Problem Statement.....	15
1.4 Project Objectives	16
1.5 Engineering Questions	16
1.6 Scope.....	16
1.7 Out of Scope	16
1.8 Contributions	17
1.9 Report Organization.....	17
Chapter 2 — Background and Technical Foundations.....	18
2.1 Fraud Detection as a Classification Problem	18
2.2 Evaluation Metrics for Imbalanced Classification	18
2.3 Data Leakage.....	18
2.4 Feature Engineering and Selection	18
2.5 Model Families Evaluated	19
2.6 Differential Privacy.....	19
2.7 Algorithmic Fairness Metrics	19
2.8 Secure API and Deployment Concepts	20
2.9 Operations Concepts	20
2.10 Regulatory Context	20

Chapter 3 — Requirements Analysis	21
3.1 Functional Requirements.....	21
3.2 Non-Functional Requirements.....	22
3.3 Requirements Traceability Matrix	23
Chapter 4 — Dataset and Data Understanding	24
4.1 Source and Composition.....	24
4.2 Verified Dimensions.....	24
4.3 Target Variable and Class Imbalance	24
4.4 Why PR-AUC Matters Especially Here	24
4.5 Data Types and Missingness	25
4.6 High Dimensionality	25
4.7 Potential Sensitive or Proxy Attributes.....	25
4.8 Dataset Limitations and Ethical Considerations	25
Chapter 5 — Overall System Architecture.....	26
5.1 Architectural Philosophy.....	26
5.2 End-to-End Pipeline	26
5.3 Component Architecture	26
5.4 Data Flow and Trust Boundaries.....	27
5.5 Machine Learning Lifecycle.....	27
5.6 Deployment Architecture	28
5.7 Shared Infrastructure.....	29
Chapter 6 — Milestone 1: Data Security	30
6.1 Data Classification.....	30
6.2 Field-Level Encryption: Fernet, Not AES-256	30
6.3 The CreditCardNumber Classification-versus-Lineage Disagreement	30
6.4 Key Handling	30
6.5 Data Integrity and Lineage	31
6.6 Protected Storage: Azure Blob Storage Target Architecture	31
6.7 Key-Management Risks and Production Recommendations.....	31
6.8 Access Control Considerations.....	31
6.9 Risks and Limitations.....	31
6.10 Implementation Status Summary	32
Chapter 7 — Milestone 2, Module 1: Data Validation.....	33

7.1 Purpose	33
7.2 The Nine Validation Checks	33
7.3 The Encryption Verification Mechanism.....	33
7.4 Failure Handling	34
7.5 The Regenerate-vs-Load-Existing Design	34
7.6 A Real Bug Found and Fixed at This Stage	34
7.7 Implementation Status	34
Chapter 8 — Milestone 2, Module 2: Feature Engineering and Selection	35
8.1 Preprocessing Architecture.....	35
8.2 Handling Numerical and Categorical Features.....	35
8.3 Missing-Value Treatment and Feature Creation.....	36
8.4 Candidate and Selected Feature Counts.....	36
8.5 The Four-Method Selection Consensus	36
8.6 Leakage Prevention and the Metadata-Column Guarantee.....	37
8.7 A Real Bug Found and Fixed at This Stage	37
8.8 Persisted Artifacts	37
8.9 Complete Feature List	37
8.10 Risks of High-Dimensional Sparse Data and Limitations of Feature Importance	37
8.11 Implementation Status	38
Chapter 9 — Milestone 2, Module 3: Model Development	39
9.1 Evaluation Design.....	39
9.2 The Five Model Families	39
9.3 Real Search Space (LightGBM, the Eventual Winner).....	39
9.4 Real Model Comparison Results	40
9.5 Model Selection Rule and Statistical Comparison	40
9.6 Why LightGBM	40
9.7 Error and Threshold-Trade-off Discussion	40
9.8 Real Evaluation Plots.....	41
9.9 Implementation Status	44
Chapter 10 — Milestone 2, Module 4: Differential Privacy	46
10.1 Why Privacy Protection Is Relevant Here	46
10.2 DP-SGD Mechanics as Implemented.....	46
10.3 Verified Identical Initial Weights.....	46

10.4 The Real Experiment	46
10.5 Why Two Epsilon Values Are Reported, and Why Only One Is Authoritative	47
10.6 The Root-Cause Investigation	47
10.7 Root-Cause Synthesis.....	48
10.8 Honest Statement of the Result	48
10.9 Recommended Future Improvements (Not Implemented).....	48
10.10 Implementation Status	50
Chapter 11 — Milestone 2, Module 5: Fairness Analysis	51
11.1 Why Fairness Matters Here, and What "Protected Group" Means in This Dataset	51
11.2 Metric Definitions as Implemented	51
11.3 Real Findings — card6 (Primary Attribute; reference group: debit).....	51
11.4 Real Findings — card4 (reference group: Visa).....	52
11.5 Real Findings — DeviceType (reference group: desktop; "Missing" reported but excluded from pairwise comparison).....	52
11.6 The 24 Bias Findings.....	52
11.7 Operational Implications and Remediation Options	53
11.8 Fairness Assessment Does Not Establish Fairness	53
11.9 Implementation Status	55
Chapter 12 — Experiment Tracking and Model Artifact Management.....	56
12.1 MLflow Integration	56
12.2 Parameters, Metrics, and Artifacts Logged.....	56
12.3 Model Persistence: joblib and Keras Artifacts	56
12.4 Versioning and Reproducibility	56
12.5 Configuration Management.....	57
12.6 Local Tracking vs. Committed Artifacts vs. Production Registry Target	57
Chapter 13 — Milestone 3: Secure Inference API	58
13.1 Application Structure	58
13.2 Authentication Flow.....	58
13.3 Credential Handling	58
13.4 The Prediction Endpoint	58
13.5 Model Loading and the Real-vs-Placeholder Distinction	59
13.6 Input Validation and Error Handling	59
13.7 Rate Limiting	59

13.8 Security Headers and Logging.....	59
13.9 Health and Documentation Endpoints	59
13.10 Secrets Abstraction and Local Fallback.....	60
13.11 API Endpoint Table.....	60
13.12 API Request Sequence	60
13.13 Implementation Status	61
Chapter 14 — Container and Deployment Security	62
14.1 Multi-Stage Docker Build	62
14.2 Non-Root Execution and Attack-Surface Reduction	62
14.3 Container Limitations.....	62
14.4 Kubernetes Manifests	62
14.5 Separating Implemented Controls from Recommendations	63
Chapter 15 — Azure Infrastructure-as-Code Architecture	63
15.1 Overview and Explicit Non-Deployment Statement.....	63
15.2 Azure Resource Inventory by Module	63
15.3 WAF Custom Rules (verbatim from `docs/WAF_Rules.md`)	65
15.4 Defense in Depth	65
15.5 Azure Deployment Diagram.....	65
15.6 What This Report Does Not Claim	67
15.7 Implementation Status	67
Chapter 16 — Threat Model and Security Analysis	68
16.1 Assets, Actors, and Trust Boundaries	68
16.2 STRIDE Threat Table.....	68
16.3 Additional ML- and Supply-Chain-Specific Threats.....	69
16.4 Summary	70
Chapter 17 — Penetration Testing and Security Validation	70
17.1 Scope and Methodology.....	70
17.2 Tests Performed.....	70
17.3 Findings	71
17.4 OWASP API Security Top 10 Coverage.....	71
17.5 Distinguishing Evidence Types	72
Chapter 18 — Milestone 4: Compliance Quality Assurance	73
18.1 Purpose and Materials Reviewed	73

18.2 GDPR-Relevant Concepts and Their Application	73
18.3 HIPAA Applicability	73
18.4 ISO/IEC 27001 Control Themes.....	73
18.5 Compliance Control Mapping and Gap Analysis.....	74
18.6 Framing These Inconsistencies	74
18.7 Milestone 4's Own Test Suite	75
18.8 Current State vs. Intended Target State	75
18.9 Remediation Priorities	75
Chapter 19 — Milestone 5: Drift Monitoring.....	76
19.1 Concept.....	76
19.2 Reference and Current Data, and the Synthetic Fallback	76
19.3 Feature Manifest Integration and a Real Bug Found	76
19.4 Drift Thresholds and Exit Behavior	76
19.5 Generated Reports.....	76
19.6 Monitoring Workflow	77
19.7 Limitation: No Live Production Traffic	77
19.8 Implementation Status	77
Chapter 20 — WORM Audit Storage	78
20.1 What WORM Means and Why It Matters.....	78
20.2 Implementation	78
20.3 File Discovery and a Real Path-Mapping Bug Found	78
20.4 A Second Real Bug: The Container-Name Default	78
20.5 Dry-Run Verification.....	78
20.6 Retention and Legal Hold (Designed, Not Exercised)	79
20.7 Implementation Status	79
Chapter 21 — CI/CD Pipeline	80
21.1 Workflow Triggers.....	80
21.2 Jobs	80
21.3 Real Bugs Found and Fixed During This Project's Integration	80
21.4 Fallback Behavior	80
21.5 Secret Requirements	81
21.6 CI/CD Workflow Diagram.....	81
21.7 Security Considerations and Recommendations Not Currently Implemented	81

21.8 Implementation Status	82
Chapter 22 — Testing and Quality Assurance	82
22.1 Test Suite Overview	82
22.2 Test Categories	82
22.3 Real Execution Evidence	82
22.4 Fixtures and Test Design Patterns.....	83
22.5 Mocking Discipline	83
22.6 Coverage Limitations	83
22.7 What Could Not Be Run	83
22.8 Implementation Status	83
Chapter 23 — Results and Evaluation.....	85
23.1 Consolidated Results.....	85
23.2 What Worked Well	86
23.3 What Did Not Work Well	86
23.4 Trade-offs.....	86
Chapter 24 — Limitations	86
Chapter 25 — Recommendations and Future Work	87
25.1 Immediate.....	87
25.2 Near Term	88
25.3 Long Term	88
Chapter 26 — Conclusion	88
References	90
Appendices	92
Appendix A — Repository Structure	92
Appendix B — Complete Feature Manifest (150 Selected Features)	92
Appendix C — Model Hyperparameters (Real, from `reports_m3/model_comparison_leaderboard.csv`)..	93
Appendix D — API Schemas and Endpoint Examples	93
Appendix E — Test Inventory (Complete, 178 Tests / 35 Files)	93
Appendix F — Requirements Traceability Matrix (Complete)	94
Appendix G — Compliance Control Matrix (Complete).....	94
Appendix H — Threat Model (Complete)	94
Appendix I — Terraform Resource Inventory (Complete)	94
Appendix J — Kubernetes Resource Inventory (Complete).....	94

Appendix K — Abbreviations94

Appendix L — Reproduction Guide.....94

Appendix M — Verification-Status Register95

Declaration of Originality

This report was produced by inspecting and directly verifying the contents of the `Secure\_Compliant\_ML\_Security` repository (https://github.com/Noo2r/Secure_Compliant_ML_Security). Every project-specific technical claim in this document — every metric, file reference, configuration value, and test result — was checked against the repository's source code, configuration files, and generated reports at the time of writing, rather than assumed or estimated. Where the repository did not provide sufficient evidence to support a claim, this is stated explicitly rather than filled in with an assumption. Personal information not present in the repository (student names beyond what is recorded in commit history and file headers, institutional affiliation, supervisor identity, submission date) is represented with placeholders for the submitting team to complete.

Acknowledgements

[Placeholder — to be completed by the submitting team: course instructors, technical supervisors, and any external contributors who supported this project.]

Executive Summary

This report documents a five-part machine learning system built to detect credit-card fraud in the IEEE-CIS Fraud Detection dataset while treating data security, individual privacy, demographic fairness, secure deployment, regulatory compliance, and production operations as first-class engineering requirements rather than afterthoughts. The system is organized as five integrated milestones: **Milestone 1** encrypts and classifies sensitive transaction fields; **Milestone 2** performs the core machine learning work — data validation, feature engineering, model selection, differential-privacy training, and fairness analysis; **Milestone 3** exposes the selected model through an authenticated, rate-limited FastAPI inference service with a Terraform-defined Azure deployment target; **Milestone 4** independently audits Milestones 1 and 3 against GDPR, HIPAA, and ISO/IEC 27001 control themes; and **Milestone 5** adds automated feature drift monitoring and immutable audit-log storage, wired together by a GitHub Actions pipeline.

The best-performing conventional model — a LightGBM gradient-boosted classifier selected from a five-model comparison — achieved a holdout Precision-Recall Area Under Curve (PR-AUC) of 0.4869 and a Receiver Operating Characteristic Area Under Curve (ROC-AUC) of 0.8807 on a chronologically held-out test set of 118,108 transactions. A parallel differentially private variant, trained with DP-SGD to a Rényi Differential Privacy-derived guarantee of $\epsilon \approx 45.573$ at $\delta = 1 \times 10^{-5}$, showed a substantial utility cost (PR-AUC fell from 0.1646 without privacy protection to 0.0285 with it, on a smaller neural-network exploration configuration); this trade-off is reported honestly rather than minimized, alongside an eight-phase root-cause investigation into why the cost was so large on this dataset. A fairness audit across three payment-behavior proxy attributes (card type, card network, and device type) — since the dataset contains no genuine demographic fields — produced 24 distinct findings, including a statistically significant four-times difference in flagging rate between credit- and debit-card transactions (Disparate Impact Ratio 3.996) and a materially higher missed-fraud rate for transactions with no recorded device information.

This report distinguishes throughout between what has been implemented and experimentally verified (the entire Milestone 2 ML pipeline, run against the real 590,540-row dataset), what has been implemented and locally tested (the Milestone 3 inference API, the Milestone 5 drift monitor and WORM-upload dry run), and what remains a designed but undeployed target architecture (the Terraform-defined Azure infrastructure —

AKS, Key Vault, Web Application Firewall — which has never been applied to a live Azure subscription). It also documents a small number of factual inconsistencies found in the Milestone 4 compliance documentation (a claim of AES-256 encryption where the actual implementation uses Fernet, which is AES-128; a claim of verification "across live AKS endpoints" where no such endpoint has been provisioned) so that the project's documentation and its verified state are not conflated.

Abstract

Machine learning systems deployed for financial fraud detection process highly sensitive personal and transactional data, make consequential decisions at scale, and are increasingly subject to regulatory scrutiny around privacy, fairness, and security. Much of the academic and industrial literature on fraud detection optimizes predictive accuracy in isolation, leaving data protection, differential privacy, algorithmic fairness, and secure operationalization as separate, later concerns. This project instead integrates all of these as co-equal requirements within a single pipeline, built and evaluated on the IEEE-CIS Fraud Detection dataset (590,540 transactions, 428 raw columns). The pipeline independently re-verifies an upstream data-encryption and classification contract before any modeling occurs; engineers and statistically selects 150 features from 466 candidates; compares five model families under time-aware cross-validation with Optuna hyperparameter search; trains and evaluates a differentially private variant of the winning architecture with formal privacy accounting; audits the deployed model for fairness across payment behavior proxy attributes with bootstrap confidence intervals and an independent Fairlearn cross-check; exposes the model through an authenticated, rate-limited REST API with a security-hardened container and a Terraform-defined cloud deployment target; and closes the loop with automated drift monitoring and immutable audit-log storage in a continuous-integration pipeline. The result is a system whose every claim — including its limitations — is traceable to committed source code, configuration, and generated evidence, and whose most significant technical finding is not a headline accuracy number but a rigorously investigated account of why differential privacy imposes a severe utility cost on this specific dataset and model architecture.

Keywords

Fraud detection; machine learning security; differential privacy; DP-SGD; algorithmic fairness; secure API design; Kubernetes security; Infrastructure as Code; GDPR; ISO/IEC 27001; MLOps; data drift monitoring; immutable audit storage; LightGBM; IEEE-CIS dataset.

Table of Contents

(Generated automatically in the DOCX/PDF build via heading styles; see the rendered document.)

List of Figures

(Populated automatically from figure captions in the DOCX/PDF build.)

List of Tables

(Populated automatically from table captions in the DOCX/PDF build.)

List of Abbreviations

Abbreviation	Meaning
AES	Advanced Encryption Standard
AKS	Azure Kubernetes Service
AOD	Average Odds Difference
API	Application Programming Interface
AUC	Area Under Curve
CBC	Cipher Block Chaining (block-cipher mode)
CI	Confidence Interval (statistics) / Continuous Integration (software) — disambiguated in context
CI/CD	Continuous Integration / Continuous Deployment
CRS	Core Rule Set (OWASP, used by WAF)
DIR	Disparate Impact Ratio
DP	Differential Privacy
DP-SGD	Differentially Private Stochastic Gradient Descent
DPIA	Data Protection Impact Assessment
EOD	Equal Opportunity Difference
GDPR	General Data Protection Regulation
HIPAA	Health Insurance Portability and Accountability Act
HMAC	Hash-Based Message Authentication Code
IaC	Infrastructure as Code
ISO/IEC 27001	International information security management standard
JWT	JSON Web Token
MLflow	Open-source machine learning experiment tracking platform
MLOps	Machine Learning Operations
NSG	Network Security Group (Azure)
OWASP	Open Worldwide Application Security Project
PII	Personally Identifiable Information
PR-AUC	Precision-Recall Area Under Curve
RBAC	Role-Based Access Control
RDP	Rényi Differential Privacy
ROC-AUC	Receiver Operating Characteristic Area Under Curve
SPD	Statistical Parity Difference
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege (threat-modeling method)
TLS	Transport Layer Security
TPR / FPR	True Positive Rate / False Positive Rate
WAF	Web Application Firewall

WORM	Write-Once-Read-Many (storage)
------	--------------------------------

Chapter 1 — Introduction

1.1 Background

Digital payment volume has grown steadily for more than a decade, and card-not-present fraud has grown alongside it: attackers no longer need to physically steal a card, only its details, and the IEEE-CIS Fraud Detection dataset used throughout this project reflects that reality — the majority of its transactions are online, and fraud is disproportionately concentrated in specific product categories, device types, and time windows rather than spread uniformly. Machine learning has become the dominant tool for detecting this fraud at scale, because rule-based systems cannot keep pace with the volume and evolving structure of transaction data. That shift, however, introduces its own risks: a model trained on transaction data is itself a potential source of information leakage about the individuals in that data; a model that performs well in aggregate may perform very differently across demographic or behavioral subgroups; and a model exposed as a live service becomes a new attack surface, distinct from — but connected to — the data pipeline that trained it.

Financial institutions operate under real regulatory obligations (data protection law, sector security standards) even when a specific dataset or prototype system is not itself subject to formal certification. A project that trains a fraud model without also considering how that model's training data is protected, how the model's predictions might disadvantage particular groups, whether the model leaks information about individual training examples, and how the serving infrastructure is secured, addresses only part of the real engineering problem.

1.2 Motivation

This project's motivation was to treat those non-functional concerns as first-class requirements of the system rather than optional extensions bolted on after a model was trained. Concretely, this meant: verifying — not merely assuming — that upstream data protection controls actually held; measuring the cost, in real terms, of adding a formal differential-privacy guarantee to the trained model rather than asserting privacy was "handled"; measuring whether the model's predictions differ systematically across payment-behavior groups, using the same statistical rigor (confidence intervals, significance testing, independent cross-validation of the fairness metrics themselves) that is applied to the model's predictive performance; and building a deployment and operations layer — authentication, rate limiting, container hardening, infrastructure-as-code, drift monitoring, immutable audit logging — with the same level of intentional design as the model itself, while being explicit about which parts of that layer have actually been run and verified versus which remain a designed, but not yet provisioned, target architecture.

1.3 Problem Statement

Build a fraud-detection system for the IEEE-CIS Fraud Detection dataset that (1) independently verifies the protection of sensitive input fields before any modeling occurs, (2) selects and tunes a predictive model using leakage-safe, chronologically valid evaluation, (3) quantifies the utility cost of adding a formal differential-privacy guarantee to that model, (4) quantifies whether the model's predictions differ across available payment-behavior proxy groups with statistical rigor, (5) exposes the model through a secure, authenticated API with a defined (if not fully provisioned) cloud deployment architecture, (6) is independently auditable

against common regulatory control themes, and (7) can detect when its own input data distribution has drifted from the data it was trained on, with an audit trail that cannot be silently altered after the fact.

1.4 Project Objectives

1. Regenerate or load a security-classified, field-level-encrypted version of the dataset and independently verify that encryption is actually in effect before any feature is derived from it.
2. Engineer and statistically select a feature set that avoids train/test leakage, justified by exploratory data analysis rather than convention.
3. Compare at least five distinct model families under a chronologically valid evaluation methodology and select a production candidate using a documented, reproducible rule.
4. Train a differentially private variant of the selected architecture, compute its formal privacy budget, and investigate — rather than merely report — any large utility gap between the private and non-private variants.
5. Audit the selected model for fairness across every protected or proxy attribute available in the dataset, using bootstrap confidence intervals and an independent second implementation (Fairlearn) to validate the project's own fairness metric code.
6. Expose the selected model through an authenticated, rate-limited inference API, and design (with Infrastructure-as-Code) a hardened cloud deployment for it.
7. Independently audit the data-security and deployment milestones against GDPR, HIPAA, and ISO/IEC 27001 control themes.
8. Monitor deployed feature distributions for drift and store audit-relevant reports in a tamper-evident manner, wired into a continuous-integration pipeline.

1.5 Engineering Questions

This project is an applied engineering system rather than a controlled scientific experiment, so its guiding questions are engineering questions: Can a fraud-classification pipeline be built such that every claim about its security and compliance posture is independently checkable against the running system, rather than asserted in documentation alone? What is the actual, measured utility cost of adding a rigorous differential-privacy guarantee to a gradient-boosted or neural fraud classifier trained on this specific, severely imbalanced dataset, and why does that cost arise? Do the payment-behavior attributes available in this dataset — in the explicit absence of any demographic field — reveal a statistically meaningful predictive disparity, and if so, is that disparity plausibly explained by genuine risk correlation, a data-quality artifact, or something else?

1.6 Scope

In scope: the complete Milestone 2 machine learning pipeline (data validation, feature engineering and selection, model comparison, differential privacy, fairness analysis) run against the real dataset; the Milestone 3 FastAPI inference service, run and tested locally against the trained model; Infrastructure-as-Code (Terraform) and Kubernetes manifests describing a target Azure deployment; a Milestone 4 independent compliance audit against GDPR/HIPAA/ISO 27001 control themes; and a Milestone 5 drift-monitoring and immutable audit-storage layer wired into a CI/CD pipeline.

1.7 Out of Scope

Explicitly out of scope: provisioning and operating a live Azure subscription (the Terraform configuration is a designed, internally consistent target architecture that has not been applied); processing genuine demographic attributes for fairness analysis, since none exist in the source dataset (payment-behavior and

device proxies are used instead, with that limitation stated throughout); formal legal compliance certification under any named regulatory regime; real-time production traffic or load testing at production scale; and adversarial machine learning evaluation beyond the API-level penetration test described in Chapter 17.

1.8 Contributions

The primary contributions of this project are: (1) an executable, reproducible pipeline in which every reported metric traces to a specific generated artifact in the repository, rather than to documentation alone; (2) a rare instance of a differential-privacy integration that reports and investigates a substantial utility failure in detail (an eight-phase root-cause analysis, described in Chapter 10) rather than reporting only a favorable configuration; (3) a fairness audit that cross-validates its own metric implementation against an independent library (Fairlearn) before trusting its own findings; and (4) an explicit, repository-wide discipline of separating verified current state from designed target state, applied consistently across security, deployment, and compliance claims (Chapters 6, 15, and 18 in particular).

1.9 Report Organization

Chapter 2 introduces the technical foundations used throughout the report. Chapter 3 derives functional and non-functional requirements directly from the repository and links them to verification evidence. Chapter 4 characterizes the dataset. Chapter 5 presents the end-to-end system architecture. Chapters 6 through 12 document Milestone 1 (data security) and each of Milestone 2's five internal modules (data validation, feature engineering, model development, differential privacy, fairness) plus experiment tracking. Chapters 13 through 17 document Milestone 3's secure API, container and Kubernetes design, Azure Infrastructure-as-Code, a repository-grounded threat model, and the penetration-testing evidence. Chapter 18 documents Milestone 4's compliance audit. Chapters 19 through 22 document Milestone 5's drift monitoring, WORM audit storage, the CI/CD pipeline, and the overall testing strategy. Chapters 23 through 26 consolidate results, limitations, recommendations, and conclusions. References and appendices follow.

Chapter 2 — Background and Technical Foundations

This chapter introduces, briefly and specifically, the technical concepts this project depends on. Each concept is connected directly to how it is used later in this report rather than treated as independent background material.

2.1 Fraud Detection as a Classification Problem

Fraud detection is framed here as binary classification: each transaction is labeled ``isFraud`` = 0 or 1, and a model produces a probability that a given transaction is fraudulent. Two properties of this specific problem shape every later design decision in the project. First, **class imbalance**: only a small fraction of transactions are fraudulent (Chapter 4 gives the exact measured rate), so a model that simply predicts "not fraud" for every transaction achieves very high accuracy while being useless — this is why the project does not use accuracy as its primary evaluation metric. Second, **temporal structure**: transactions carry a time signal (``TransactionDT``), and a model deployed in production always scores transactions that occurred after the ones it was trained on. Both properties motivate the chronological (not random) train/test split used from Chapter 8 onward.

2.2 Evaluation Metrics for Imbalanced Classification

Precision is the fraction of transactions flagged as fraud that were actually fraud; **recall** (equivalently, True Positive Rate, TPR) is the fraction of actual fraud that was successfully flagged; **F1-score** is their harmonic mean. The **confusion matrix** tabulates true/false positives and negatives at a chosen decision threshold. **ROC-AUC** summarizes a model's ability to rank positive examples above negative ones across every possible threshold, but because it is computed against the (large) negative class, a model can achieve a high ROC-AUC while still performing poorly on the metric that matters operationally. **PR-AUC** is instead sensitive specifically to how well the model performs on the minority (fraud) class, which is why it is used as this project's primary ranking metric for model comparison (Chapter 9). **Threshold selection** is the choice of probability cutoff above which a prediction is treated as "fraud" — the conventional default of 0.5 is calibrated for balanced problems and is not an appropriate default here, which is why Chapter 9 describes an explicit, validation-set-based threshold-optimization step rather than using 0.5 uncritically.

2.3 Data Leakage

Data leakage occurs when information that would not be available at real prediction time influences model training or model selection, producing an optimistic evaluation. In this project, two specific leakage risks are addressed by design: statistics learned by feature-engineering transformers (frequency tables, per-card aggregates) are fit only on the training fold and merely applied to the test fold; and the train/test split itself is chronological rather than random, so a model is never evaluated on data that occurred earlier than what it was trained on.

2.4 Feature Engineering and Selection

Feature engineering derives new columns from raw data (for example, an hour-of-day feature derived from a raw timestamp); feature selection then reduces a larger candidate feature set to a smaller one expected to generalize better and train faster. Chapter 8 describes both stages as implemented in this project, including

the specific statistical methods used for selection (near-zero-variance filtering, correlation-redundancy filtering, mutual information, and embedded Random Forest importance).

2.5 Model Families Evaluated

Logistic Regression is a linear model producing well-calibrated probabilities for a binary outcome; it serves as this project's interpretable baseline. **Random Forest** is an ensemble of decision trees trained on random subsets of data and features, combined by averaging. **XGBoost** and **LightGBM** are both gradient-boosted decision tree frameworks that build trees sequentially, each correcting the residual error of the previous ones; they typically perform strongly on structured, tabular data such as this dataset. A **Neural Network** — a small feed-forward architecture in this project — learns non-linear feature interactions through iterative gradient-based weight updates and additionally serves, in this project, as the base architecture for the differential-privacy experiment in Chapter 10. **Hyperparameter optimization with Optuna** refers to Bayesian search (specifically, a Tree-structured Parzen Estimator) over each model family's hyperparameter space, used in place of exhaustive grid search for tractability (Chapter 9).

2.6 Differential Privacy

Differential privacy (DP) is a mathematical framework that bounds how much any single training example can influence a model's output, protecting against an adversary inferring whether a specific individual's data was used in training. **DP-SGD** (Differentially Private Stochastic Gradient Descent) achieves this by two mechanisms applied on top of ordinary gradient descent: **gradient clipping**, which caps the L2 norm of each individual example's gradient to a fixed bound before aggregation, and a **noise multiplier**, which scales Gaussian noise added to the clipped, aggregated gradient before the weight update. The strength of the resulting privacy guarantee is expressed as **epsilon (ϵ)** — smaller means stronger — and **delta (δ)**, the (ideally very small) probability the guarantee fails to hold. **Rényi Differential Privacy (RDP) accounting** is the specific mathematical technique used to convert a training configuration (noise multiplier, batch size, dataset size, epoch count) into a tight, composable (ϵ , δ) value across the many steps of a training run; Chapter 10 documents exactly how this project computed and reported that value.

2.7 Algorithmic Fairness Metrics

Statistical Parity Difference (SPD) is the difference in selection rate (fraction flagged as fraud) between a group and a reference group. **Disparate Impact Ratio (DIR)** is the same comparison expressed as a ratio rather than a difference, with 1.0 as the no-disparity value; a DIR below 0.8 (or, symmetrically, above 1.25) is commonly referenced against the "four-fifths rule" heuristic. **Equal Opportunity Difference (EOD)** compares true positive rate (recall) between groups — whether actual fraud is caught equally often — independent of how the model treats non-fraud. **Average Odds Difference (AOD)** averages the false-positive-rate difference and the true-positive-rate difference, so it is sensitive to disparities in both directions. **Bootstrap confidence intervals** quantify the statistical uncertainty in each of these point estimates by resampling the observed data with replacement many times and recomputing the metric, so that a disparity can be judged "real" only when its interval excludes the metric's no-disparity value — this project's specific implementation is described in Chapter 11.

2.8 Secure API and Deployment Concepts

JWT (JSON Web Token) authentication issues a signed, short-lived token after a successful login that the client then presents on each subsequent request; the server verifies the signature and expiry rather than checking credentials on every call. **API rate limiting** bounds how many requests a client may make in a given time window, mitigating brute-force and denial-of-service abuse. **Secrets management** refers to storing credentials, keys, and tokens outside application code and configuration files — in this project's target architecture, in Azure Key Vault, accessed via a workload identity rather than a stored credential. **Container hardening** covers practices such as running as a non-root user, using a minimal multi-stage build, and marking the filesystem read-only. **Kubernetes security** includes namespace isolation, pod security contexts, and NetworkPolicies that restrict which pods may communicate with which. **Infrastructure as Code (IaC)** — Terraform in this project — defines cloud infrastructure declaratively so it can be reviewed, versioned, and (in principle) reproducibly provisioned. A **Web Application Firewall (WAF)** inspects HTTP traffic against known attack patterns (the OWASP Core Rule Set, in this project's design) before it reaches the application. **Network policies** at the Kubernetes level provide a second, workload-level layer of network segmentation beneath the cloud-provider network security groups.

2.9 Operations Concepts

Model / data drift is a change in the statistical distribution of production input data relative to the data a model was trained on, which can silently degrade model performance over time; Chapter 19 describes this project's Evidently-based drift monitor. **WORM (Write-Once-Read-Many) storage** is immutable storage — once written, a record cannot be altered or deleted for a defined retention period — used here for audit-relevant reports so that evidence of past compliance checks cannot be tampered with after the fact (Chapter 20). **CI/CD (Continuous Integration / Continuous Deployment)** automates running tests, checks, and deployment steps on every code change; Chapter 21 documents this project's GitHub Actions workflow.

2.10 Regulatory Context

GDPR (the EU General Data Protection Regulation) establishes principles including lawfulness, data minimization, storage limitation, integrity and confidentiality, and accountability for personal data processing. **HIPAA** (the U.S. Health Insurance Portability and Accountability Act) governs protected health information specifically for covered entities and their business associates; its applicability to an ordinary payment-fraud dataset containing no health information is not automatic, and this report does not assume it applies (Chapter 18 discusses this explicitly). **ISO/IEC 27001** is an international standard for information security management systems, covering control areas such as access control, cryptography, and incident management; referencing its control themes is not the same as holding certification against the standard, and this report does not claim certification. These three frameworks are used in Chapter 18 as a structure for independently assessing the project's actual security and privacy controls, not as claims of formal compliance.

Chapter 3 — Requirements Analysis

Requirements below are derived directly from what the repository implements, not from a generic specification written independently of the system. Each requirement is linked to the file(s) that implement it and the verification evidence available at the time of writing.

3.1 Functional Requirements

Table 3.1 — Functional requirements and implementing components.

ID	Requirement	Implementing file(s)	Verification status
FR-1	Load and validate transaction data against a defined schema and dtype contract	`src/data/validators.py`, `scripts/run_module1.py`	Implemented and experimentally verified (real 590,540-row dataset, 9/9 checks passed — Chapter 7)
FR-2	Independently verify that sensitive fields are actually encrypted, not merely trust an upstream classification report	`src/data/validators.py::_check_pii_columns_encrypted`	Implemented and experimentally verified
FR-3	Track data lineage / provenance across modules via a dataset-version hash	`src/data/secure_data_loader.py` (`DatasetManifest`)	Implemented and experimentally verified
FR-4	Engineer features from raw transaction/identity data with no test-fold information leakage	`src/features/engineering.py`	Implemented and experimentally verified
FR-5	Select a reduced, statistically justified feature set from the engineered candidates	`src/features/selection.py`	Implemented and experimentally verified (466 → 150 — Chapter 8)
FR-6	Train and compare at least five distinct model families with hyperparameter optimization	`src/models/train.py`, `src/models/tune.py`, `src/models/model_registry.py`	Implemented and experimentally verified
FR-7	Persist a self-contained, reusable inference artifact per candidate model	`src/models/inference_bundle.py`	Implemented and experimentally verified
FR-8	Train a differentially private variant of the selected architecture and compute its formal privacy budget	`src/privacy/dp_model.py`, `src/privacy/privacy_accountant.py`	Implemented and experimentally verified (Chapter 10)
FR-9	Audit the selected model for fairness across available proxy attributes with statistical rigor	`src/fairness/*.py`	Implemented and experimentally verified (Chapter 11)
FR-10	Expose the selected model through an authenticated inference endpoint	`app/api/inference.py`, `app/core/security.py`	Implemented and locally tested (Chapter 13)
FR-11	Rate-limit API requests, including a stricter limit	`app/core/limiter.py`, `app/main.py`, `app/api/auth.py`	Implemented and locally tested

	on the authentication endpoint		
FR-12	Monitor feature distributions for drift against a reference dataset	<code>`drift_monitor.py`</code>	Implemented and locally tested; production traffic not available (Chapter 19)
FR-13	Produce immutable, hash-verified audit records of compliance-relevant reports	<code>`worm_storage.py`</code>	Dry-run verified; live upload requires a provisioned Azure Storage account (Chapter 20)
FR-14	Execute an automated test suite covering every module	<code>`tests/`</code> (35 files)	Implemented and experimentally verified — 178/178 passing at time of writing (Chapter 22)
FR-15	Run automated checks (drift monitoring, WORM upload) on a defined schedule and on code changes	<code>`.github/workflows/mlops-pipeline.yml`</code>	Implemented and locally verified; live-runner execution outside this report's direct control (Chapter 21)

3.2 Non-Functional Requirements

Table 3.2 — Non-functional requirements and how the repository addresses them.

Category	Requirement	Addressed by
Security	Sensitive fields protected at rest; no plaintext PII reaches model input	Milestone 1 encryption + Module 1 independent verification (Chapter 6, 7)
Security	API access requires authentication; requests are rate-limited; responses carry hardening headers	Milestone 3 (Chapter 13)
Privacy	A formal, accounted privacy guarantee is available as an option for the trained model	Milestone 2 Module 4 (Chapter 10)
Integrity	Data validation halts the pipeline on critical contract violations before they propagate	<code>`DataValidator`</code> (Chapter 7)
Integrity	Audit-relevant reports are hash-verified and intended for immutable storage	Milestone 5 WORM (Chapter 20)
Availability	Rate limiting and a Web Application Firewall design mitigate resource-exhaustion abuse	Milestone 3 (Chapters 13, 15)
Reproducibility	Every stage persists its output as a versioned, file-based artifact consumable by the next stage	Repository-wide convention (Chapter 5)
Auditability	Every generated report is committed and traceable to the code that produced it; dataset/version hashes thread through every module	Repository-wide convention
Explainability	Feature importance, SHAP-style discussion, and per-model evaluation plots accompany model selection	Chapter 9; full SHAP integration noted as future work (Chapter 25)
Maintainability	Per-milestone dependency files, a single configuration source (<code>`config.yaml`</code>), and a shared logging convention	Chapter 12; <code>`src/config/`</code> , <code>`src/utils/logger.py`</code>

Fairness	Model predictions are audited across every available proxy attribute with statistical significance testing	Milestone 2 Module 5 (Chapter 11)
Regulatory alignment	Independent audit against GDPR/HIPAA/ISO 27001 control themes	Milestone 4 (Chapter 18)
Performance	Model comparison includes training time as a tie-breaking criterion; API rate limits and a documented DP-SGD training-time cost are reported	Chapter 9, Chapter 10
Portability	Containerized deployment (multi-stage Docker build) independent of the host operating system	Chapter 14

3.3 Requirements Traceability Matrix

A condensed matrix is given here; the complete version, cross-referencing every requirement to its test file(s), is provided in **Appendix F**.

Table 3.3 — Condensed requirements traceability.

Requirement	Module/File	Test Evidence	Report Reports	Status
FR-1, FR-2, FR-3	`src/data/`	`tests/test_validators.py`, `tests/test_secure_data_loader.py`, `tests/test_classification_registry.py` (20 tests)	`reports_m2/module1_validation_report.json`, `dataset_manifest.json`	Verified
FR-4, FR-5	`src/features/`	`tests/test_feature_engineering.py`, `tests/test_feature_selection.py`, `tests/test_preprocessing.py`, `tests/test_eda.py`, `tests/test_pipeline_persistence.py` (22 tests)	`reports_m2/feature_manifest.json`, `feature_engineering_report.md`	Verified
FR-6, FR-7	`src/models/`	`tests/test_model_registry.py`, `tests/test_threshold.py`, `tests/test_keras_wrapper.py`, `tests/test_tune.py`, `tests/test_inference_bundle.py`, `tests/test_metrics.py`, `tests/test_comparison.py` (34 tests)	`reports_m3/model_comparison_leaderboard.csv`, `model_development_report.md`	Verified
FR-8	`src/privacy/`	9 `test_dp_*.py` files (36 tests)	`reports_m4/dp_report.md`, `reports_m4/investigation/`	Verified
FR-9	`src/fairness/`	9 `test_fairness*.py` / `test_bias_detector.py` / `test_attribute_analysis.py` / `test_statistical_tests.py` / `test_fairlearn_validation.py` files (42 tests)	`reports_m5/fairness_report.md`, `bias_findings.md`	Verified
FR-10, FR-11	`app/`	`tests/test_model_loader.py`, `tests/test_compliance_real.py` (6 tests)	Manual `curl` verification (this session's engineering log); `pentest/Penetration_Test_Report_Milestone3.docx`	Locally verified
FR-12	`drift_monitor.py`	`tests/test_drift_monitor.py` (6 tests)	`reports_m5/drift_status.json`	Locally verified

FR-13	`worm_storage.py`	`tests/test_worm_storage.py` (8 tests)	`reports_m5/worm_upload_manifest.json` (dry-run)	Dry-run verified only
-------	-------------------	--	--	-----------------------

Chapter 4 — Dataset and Data Understanding

4.1 Source and Composition

The project uses the **IEEE-CIS Fraud Detection** dataset, distributed as two linked tables:

`train\_transaction.csv` (transaction-level features — amount, product code, card fields, address, distance, counting features `C1`–`C14`, timing features `D1`–`D15`, match flags `M1`–`M9`, and 339 anonymized Vesta engineering features `V1`–`V339`) and `train\_identity.csv` (device and identity metadata, joined on `TransactionID`). Milestone 1 merges these via a left join and injects six synthetic PII columns (`CustomerName`, `Email`, `PhoneNumber`, `Address`, `NationalID`, `CreditCardNumber`) to exercise the encryption/classification pipeline, since the source dataset itself does not contain named PII.

4.2 Verified Dimensions

Per `reports\_m2/dataset\_manifest.json`, generated directly from the real secured dataset: **590,540 rows** and **428 columns** after the merge and PII injection. After Module 1's validation and Module 2's feature engineering/selection stages, the persisted, model-ready processed dataset (`data/processed/train.csv` and `test.csv`) contains **472,432 training rows** and **118,108 test rows**, each with **207 columns** (202 model-input feature columns after one-hot expansion of selected categorical features, plus 4 metadata columns and the target). These row and column counts were confirmed directly against the committed CSV headers and manifest files, not estimated.

4.3 Target Variable and Class Imbalance

The target column, `isFraud`, is binary. The dataset exhibits severe class imbalance — approximately **3.5%** of transactions are labeled fraudulent (the exact figure is used consistently throughout Modules 2–5 as the reference fraud rate for synthetic-data generation and threshold design; see `src/privacy/dp\_model.py` and `drift\_monitor.py`, both of which encode `\_FRAUD\_RATE\_REF = 0.035`). This is the direct motivation for **Section 4.4** below and for the project-wide decision (Chapter 2) to use PR-AUC, not accuracy, as the primary evaluation metric.

4.4 Why PR-AUC Matters Especially Here

At a 3.5% positive rate, a trivial "always predict not-fraud" classifier scores approximately 96.5% accuracy while catching zero fraud. ROC-AUC, computed against the large negative class, also tends to look better than a model's real operational usefulness at this level of imbalance, because false positives are diluted across a very large negative population. PR-AUC is directly sensitive to how well the model's positive-class (fraud) predictions actually perform, which is why it was selected as the model-comparison ranking metric in Chapter 9.

4.5 Data Types and Missingness

The raw merged dataset mixes numeric (``TransactionAmt``, ``card1`–`card3``, ``card5``, ``C1`–`C14``, ``D1`–`D15``, ``V1`–`V339``, most ``id_01`–`id_11`` columns), categorical (``ProductCD``, ``card4``, ``card6``, ``P_emaildomain``, ``R_emaildomain``, ``M1`–`M9``, ``DeviceType``, most ``id_12`–`id_38`` columns), and identifier-like (``TransactionID``) fields. Missingness is substantial and non-uniform: the Vesta ``V`` feature block, in particular, is missing in large, shared row-patterns across dozens of columns simultaneously — this specific structure directly motivated the hashed-null-mask design of ``MissingIndicatorEngineer`` described in Chapter 8, rather than a naive per-column missing-value flag.

4.6 High Dimensionality

With 428 raw columns before engineering (339 of them anonymized ``V`` features alone), the dataset is high-dimensional relative to its row count once grouped by any rare category. This motivated the project's four-method feature-selection consensus (Chapter 8) rather than relying on any single selection heuristic, and its explicit rejection of Recursive Feature Elimination as computationally infeasible at this dimensionality (Chapter 8, Chapter 12).

4.7 Potential Sensitive or Proxy Attributes

The dataset contains no genuine demographic field (no age, gender, race, or similar attribute). Three fields were identified and used in this project's fairness analysis (Chapter 11) as plausible *behavioral or device-usage proxies*: ``card6`` (debit vs. credit), ``card4`` (card network), and ``DeviceType`` (desktop vs. mobile). This report follows the repository's own convention of never describing a finding on these attributes as a "demographic bias" finding — they are payment-behavior or device-usage proxies, and that distinction is preserved throughout Chapter 11 and Chapter 23.

4.8 Dataset Limitations and Ethical Considerations

The dataset originates from a 2019 Kaggle competition (IEEE-CIS Fraud Detection) and reflects transaction patterns and fraud tactics from that period; it is not necessarily representative of current payment-fraud patterns (Chapter 24 revisits this as a limitation). The synthetic PII columns injected by Milestone 1 are fabricated (via the ``Faker`` library with a fixed seed) specifically so the encryption pipeline could be exercised without using or exposing any real individual's data; this report does not reproduce any raw record, encrypted or not, from the dataset. Licensing and use of the dataset are governed by the original Kaggle competition's terms; this project does not redistribute the raw dataset, only generated, aggregate reports derived from it.

Chapter 5 — Overall System Architecture

5.1 Architectural Philosophy

The system is organized as five sequential, file-based milestones rather than a single monolithic codebase. Each milestone communicates with the next exclusively through committed files — CSV, JSON, joblib, and Keras artifacts — never through shared in-process state. This mirrors how the project was actually built (five different owners, one per milestone) and makes every hand-off independently inspectable: a reader can examine any module's output without running any code.

5.2 End-to-End Pipeline

Figure 5.1 shows the complete data-to-operations flow. Each stage's real, generated output is documented in the chapter noted alongside it.

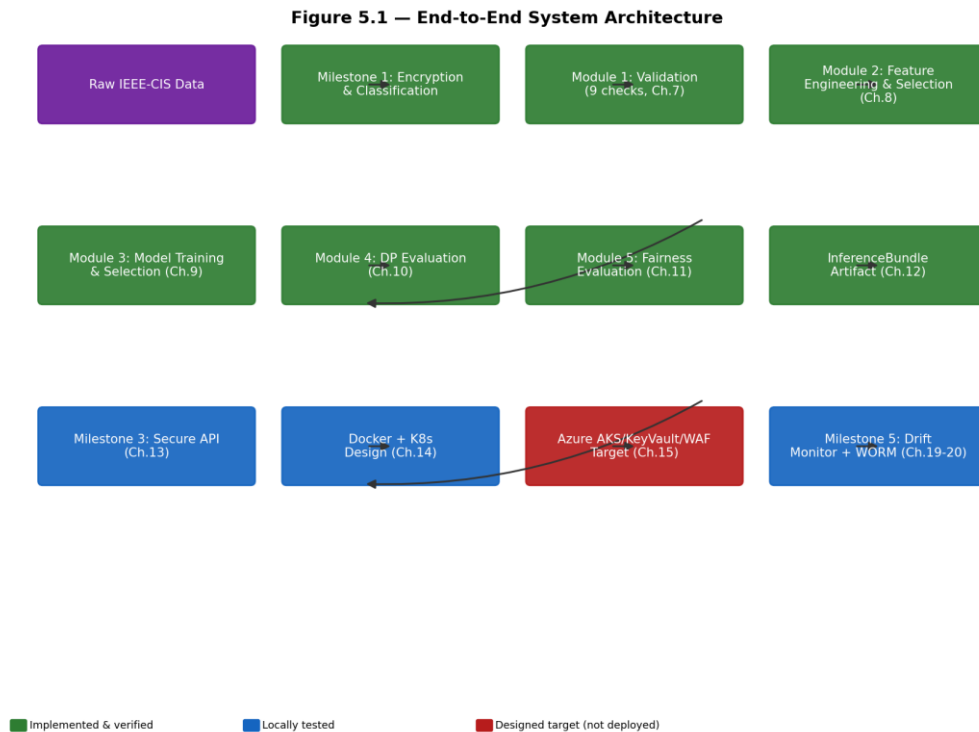


Figure 5.1 — End-to-end system architecture, from raw data to operations. Solid arrows indicate a direct file-based data or artifact hand-off; dashed arrows indicate an independent audit relationship (Milestone 4 reviews Milestones 1 and 3's output without participating in the pipeline itself).

5.3 Component Architecture

Table 5.1 maps each milestone to its top-level repository location and its primary role.

Milestone	Top-level location(s)	Role
1 — Data Security	<code>`reports/`</code> (Milestone 1 artifacts consumed, not owned, by this repository)	Field-level encryption, classification, lineage
2 — ML Development	<code>`src/`, `scripts/`, `reports_m2/`– `reports_m5/`, `artifacts_m2/`–</code>	Validation, features, models, privacy, fairness

	`artifacts_m5/`	
3 — Secure Deployment	`app/`, `Dockerfile`, `k8s/`, `iac/terraform/`, `docs/`, `pentest/`	Inference API, container, cloud target architecture
4 — Compliance QA	`compliance/`	Independent audit of Milestones 1 and 3
5 — MLOps	`drift_monitor.py`, `worm_storage.py`, `.github/workflows/`	Drift monitoring, audit storage, CI/CD

5.4 Data Flow and Trust Boundaries

Figure 5.2 shows where data crosses a trust boundary — a point where the entity handling the data changes, or where an untrusted input first enters a trusted component — since these are the points threat modeling in Chapter 16 focuses on.

Figure 5.2 — Trust Boundaries

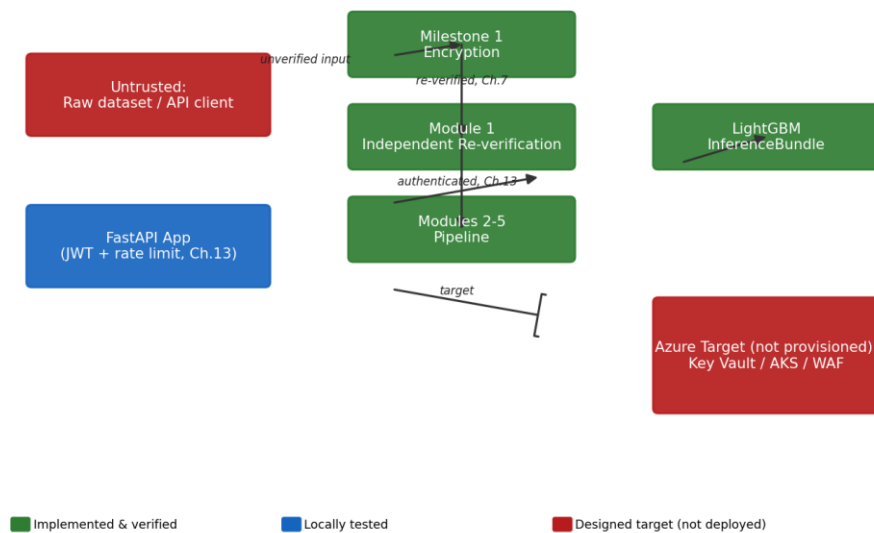


Figure 5.2 — Trust boundaries. The most consequential boundary is the re-verification step between Milestone 1 and Module 1: Module 1 never assumes Milestone 1's encryption held, it independently confirms it (Chapter 6, Chapter 7).

5.5 Machine Learning Lifecycle

Figure 5.3 presents Milestone 2's internal lifecycle in isolation, since it is the most complex single component of the system.

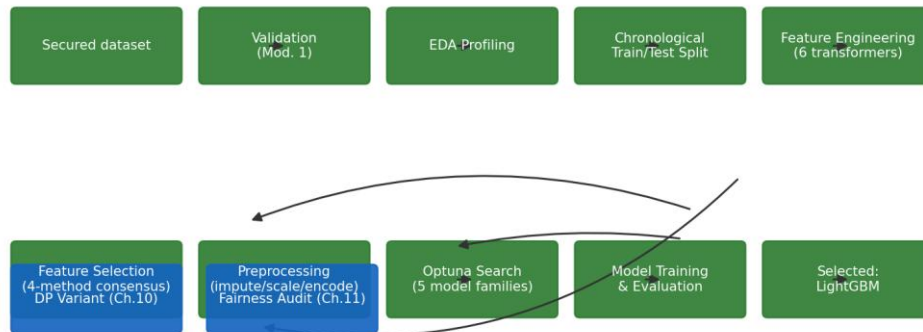
Figure 5.3 — Module 2 Machine Learning Lifecycle

Figure 5.3 — The Module 2 machine learning lifecycle. Feature engineering and selection are fit exclusively on the training fold (Chapter 8); Optuna search runs on a systematically sampled subset for tractability before the winning configuration is refit on the full training set (Chapter 9).

5.6 Deployment Architecture

Figure 5.4 shows the container and orchestration design (implemented and locally run) alongside the Azure target architecture (designed, not provisioned) as two clearly separated layers.

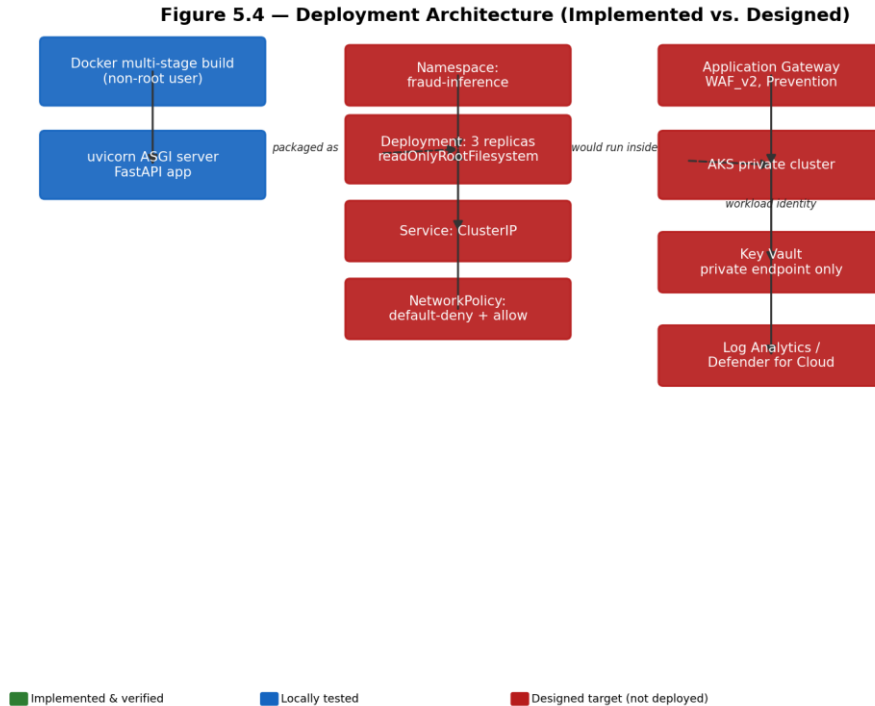


Figure 5.4 — Deployment architecture with an explicit implemented/designed boundary. Everything below the "Designed target: Azure" line has not been applied to a live subscription (Chapter 15).

5.7 Shared Infrastructure

Three conventions apply across every milestone. First, a single typed configuration source (``src/config/config.yaml``, loaded via ``load_config()``) governs Milestone 2's behavior; Milestones 3 and 5 use their own environment-variable-driven settings (``app/core/config.py``, ``drift_monitor.py``/``worm_storage.py``'s CLI arguments) rather than sharing this file, since they are independently deployable components. Second, a ``dataset_version`` hash (a 16-character digest of the dataset's structural fingerprint) is threaded from Module 1 through every later module's manifests and MLflow tags, so any artifact can be traced back to the exact data version that produced it. Third, logging is centralized per milestone (``src/utils/logger.py`` for Milestone 2; Python's standard ``logging`` module, configured in ``app/main.py``, for Milestone 3; module-level loggers for Milestone 5), consistently writing structured, timestamped records.

Chapter 6 — Milestone 1: Data Security

Milestone 1 is owned upstream of this repository's Module 1 (which consumes and independently re-verifies its output — Chapter 7). This chapter documents what Module 1's consumption of Milestone 1's artifacts reveals about the actual implementation, since Milestone 1's own source notebook is not itself part of this repository; the evidence available here is Milestone 1's two governance reports (`reports/data_classification.csv`, `reports/data_lineage.csv`) and the regenerated encrypted dataset Module 1 loads and validates.

6.1 Data Classification

`reports/data_classification.csv` labels every column with a compliance sensitivity category: PII (`CustomerName`, `Email`, `PhoneNumber`, `Address`, `NationalID`), Financial (`TransactionAmt`, `CreditCardNumber`), Sensitive (`addr1`, `addr2`, `P_emaildomain`, `R_emaildomain`, `DeviceInfo`), and Target (`isFraud`). `reports/data_lineage.csv` separately records pipeline provenance — what was merged, what was dropped, and, in its "PII Columns" step, the literal list of columns that were Fernet-encrypted.

6.2 Field-Level Encryption: Fernet, Not AES-256

The implemented encryption mechanism is **Fernet**, from Python's `cryptography` library. Fernet is a symmetric, authenticated encryption recipe built on **AES-128 in CBC (Cipher Block Chaining) mode**, combined with **HMAC-SHA256** for message authentication and a fixed structure that embeds a version byte, timestamp, and initialization vector alongside the ciphertext. Every Fernet token therefore begins with a fixed, recognizable base64-encoded prefix (`gAAAAA`), which Module 1 uses as an independent, dependency-free encryption check (Chapter 7). This is stated precisely because the Milestone 4 compliance documentation (Chapter 18) describes the implementation as using "AES-256 symmetric encryption" — Fernet's underlying cipher is **AES-128**, not AES-256, and this report treats the code (`_FERNET_TOKEN_PREFIX = "gAAAAA"` in `src/data/validators.py`, and the `gAAAAA`-prefixed values actually present in the encrypted dataset) as the authoritative evidence of what was implemented, not the documentation's description of it.

6.3 The CreditCardNumber Classification-versus-Lineage Disagreement

The single most consequential finding from Module 1's independent verification (detailed in Chapter 7) is that `CreditCardNumber` is labeled **Financial** in `data_classification.csv` but is actually present in the **encrypted-columns** list in `data_lineage.csv`. A downstream system that trusted only the classification report's PII label to decide which columns must never be used as raw model features would have missed `CreditCardNumber` entirely, since it is not labeled PII there. Module 1 resolves this by sourcing its feature blacklist from the lineage report's actual encrypted-columns list — the ground truth of what Fernet literally encrypted — rather than the classification report's compliance label, and this is the specific mechanism that prevents a real plaintext-credit-card-number leak into the model input.

6.4 Key Handling

Fernet requires a symmetric key to decrypt; Module 1 never decrypts data and does not require this key for its own operation (its encryption check operates on ciphertext structure alone, described in Chapter 7). A `secret.key` file is present at the repository root, is excluded from version control by `.gitignore`, and its

handling in this repository is consistent with a local-development convenience rather than a production key-management design; **Section 6.7** below discusses the gap between this and a production-appropriate key-management architecture.

6.5 Data Integrity and Lineage

``DatasetManifest`` (``src/data/secure_data_loader.py``) records SHA-256 hashes of every source file (computed via 8MB streamed chunks, so multi-hundred-megabyte CSVs never need to be fully loaded into memory solely to be hashed) and a ``dataset_version`` — a 16-character hash of the dataset's structural fingerprint (column names, row count, dropped-column list, target distribution) — that is threaded through every later module's manifests and MLflow tags for full provenance traceability. A ``lineage_consistency`` check (Chapter 7) cross-verifies that a regenerated dataset's dropped-column count matches Milestone 1's own lineage report, logging a warning (not a hard failure) on mismatch, since a differing raw-data snapshot is a legitimate, non-error explanation for drift.

6.6 Protected Storage: Azure Blob Storage Target Architecture

Milestone 1's own documentation describes an intended Azure Blob Storage target for the encrypted dataset. This repository does not itself provision or interact with a live Azure Storage account for Milestone 1's data; the encrypted dataset is loaded and regenerated from local files (``data/encrypted/encrypted_dataset.csv``). This is consistent with the broader project-wide pattern (Chapters 15 and 20) of a designed cloud target architecture that has not been operationally verified against a live subscription.

6.7 Key-Management Risks and Production Recommendations

As implemented, the encryption key exists as a local file rather than in a managed secret store, and key rotation is not automated. For a production deployment, this report recommends: storing the Fernet key in Azure Key Vault (the same store already used for the Milestone 3 JWT secret and CORS configuration — Chapter 13) rather than a local file; accessing it via the managed identity / workload identity mechanism already designed for Milestone 3 (Chapter 15), so no key is ever present in application code, container images, or environment variables in plaintext; and implementing a defined key-rotation schedule with a re-encryption or key-versioning strategy, since Fernet itself provides no built-in key-rotation mechanism. These are stated here as recommendations, consistent with Chapter 25 — none are currently implemented.

6.8 Access Control Considerations

``reports/access_control.csv`` (Milestone 1's own governance artifact) defines role-based access policies, including an explicit "Read Encrypted Data" (not "Decrypt") permission scoped to the ML Engineer role — i.e., Module 2 onward is designed to consume encrypted, not decrypted, PII columns, which is consistent with the fact that no module in this repository ever decrypts the PII columns; they are excluded from every model's feature set entirely (Section 6.3, Chapter 8).

6.9 Risks and Limitations

The synthetic PII values injected for this project (via the ``Faker`` library with a fixed seed) carry no real predictive signal and exist only to exercise the encryption mechanism; a production system would encrypt

genuine customer PII, at which point the key-management recommendations in Section 6.7 become a hard operational requirement rather than a hardening suggestion. Fernet's fixed-format tokens (a recognizable prefix, embedded timestamp) are also themselves metadata; while this does not reveal plaintext content, a production deployment handling this data at very large scale might consider whether token-length or timing side-channels are a concern for its specific threat model (Chapter 16 does not identify this as a currently exploited risk in this project, but notes it as a residual consideration).

6.10 Implementation Status Summary

Table 6.1 — Milestone 1 component status.

Component	Status
Fernet field-level encryption of PII columns	Implemented and experimentally verified (independently re-checked by Module 1, Chapter 7)
Data classification and lineage reporting	Implemented and experimentally verified (consumed directly by Module 1)
Dataset provenance hashing (`dataset_version`, SHA-256 source hashes)	Implemented and experimentally verified
Azure Blob Storage target architecture	Documented target state; not provisioned or verified against a live account
Production-grade key rotation / Key Vault-backed key storage	Not implemented; recommended future work (Chapter 25)
Role-based access control policy (`access_control.csv`)	Documented; enforcement mechanism outside this repository's Module 2–5 code

Chapter 7 — Milestone 2, Module 1: Data Validation

7.1 Purpose

Module 1 exists to ensure that no later module ever trains on, or is blocked by, data that violates a defined structural or security contract. It runs before feature engineering and enforces two distinct kinds of checks: **validation** (does the data conform to the schema and business-rule contract this pipeline expects?) and **verification** (is the data internally trustworthy — no duplicate transactions, no fan-out from the merge, PII actually encrypted?). This separation matches the project's official requirement wording and is implemented as a single `DataValidator` class (`src/data/validators.py`) that runs all checks unconditionally, rather than stopping at the first failure — an enterprise auditor reading the resulting report sees every issue in one pass.

7.2 The Nine Validation Checks

Table 7.1 — All nine checks, their severity, and their real result on the production dataset (source: `reports_m2/module1_validation_report.json`, generated 2026-07-01).

Check name	Severity	Purpose	Real result
<code>`row_count_positive`</code>	Critical	Dataset is non-empty	PASS — <code>row_count=590540</code>
<code>`required_columns_present`</code>	Critical	Every column the pipeline expects is present	PASS — all required columns present
<code>`dtype_contract`</code>	Critical	Column dtypes match the declared contract	PASS — all dtypes match contract
<code>`no_duplicate_rows`</code>	Critical	No exact-duplicate rows beyond a configured tolerance	PASS — <code>duplicate_rows=0</code> (<code>max_allowed=0</code>)
<code>`merge_key_unique`</code>	Critical	The transaction/identity left join did not fan out (one-to-many)	PASS — duplicate TransactionID values after merge: 0
<code>`target_binary`</code>	Critical	The target column contains only {0, 1}	PASS — unique target values: [0, 1]
<code>`pii_columns_encrypted`</code>	Critical	Every value in an encrypted column looks like real ciphertext	PASS — all PII columns encrypted
<code>`missing_value_ceiling`</code>	Warning	No surviving column exceeds a configured missing-value percentage	PASS — no columns exceed ceiling
<code>`lineage_consistency`</code>	Warning	Regenerated drop-count matches Milestone 1's own lineage report	PASS — regenerated column-drop count matches Yara's <code>data_lineage.csv</code>

Overall result: `"passed": true`, all 7 critical checks and both warning checks passed on the real, generated dataset. The severity split is meaningful: only a critical-check failure halts the pipeline (``overall_passed` = all(c.passed for c in checks if c.severity == "critical")`); a warning-only failure is surfaced but does not block execution.

7.3 The Encryption Verification Mechanism

The ``pii_columns_encrypted`` check is the most security-relevant of the nine and deserves particular attention. It does not decrypt any value — it checks that every non-null value in each column listed in the registry's ``encrypted_columns`` set (sourced from the lineage report, per Chapter 6.3, not the classification

report) begins with the fixed Fernet token prefix ``gAAAAA``. This is a lightweight, dependency-free way to distinguish genuine ciphertext from plaintext without requiring the decryption key, and it is what would have caught a real plaintext leak had the ``CreditCardNumber`` classification-versus-lineage disagreement (Chapter 6.3) been resolved the wrong way.

7.4 Failure Handling

Every check independently returns a ``CheckResult(name, severity, passed, message)``; the module never raises an exception mid-check. ``run_module1.py`` prints the full summary and exits with code 0 if the overall result passed, or code 1 otherwise — a deliberate design so an automated CI/CD chain (as in Milestone 5, Chapter 21) can halt before Module 2 ever touches unvalidated data, without needing to parse log text to determine success or failure.

7.5 The Regenerate-vs-Load-Existing Design

``SecureDatasetLoader`` (``src/data/secure_data_loader.py``) supports two modes: ``load_existing`` (read Milestone 1's real encrypted artifact directly, if present) and ``regenerate`` (faithfully reproduce Milestone 1's documented pipeline steps from raw source CSVs). Both modes exist because the exact encrypted artifact was not always available in every development environment; ``regenerate`` is verified against Milestone 1's own lineage report's drop count (the ``lineage_consistency`` check, Section 7.2) as a faithfulness check, not treated as an independent, unverified reimplementaion.

7.6 A Real Bug Found and Fixed at This Stage

During development, a duplicate-logging configuration issue was found and fixed in this module's logging setup (each module's entrypoint previously risked attaching duplicate log handlers on repeated invocation within the same process); this was corrected in ``src/utlis/logger.py``'s shared ``configure_logging()`` function so that logging configuration is idempotent. This is recorded here as a concrete instance of the kind of defect this project's regression-test discipline (Chapter 22) is designed to catch and permanently prevent from recurring.

7.7 Implementation Status

Table 7.2 — Module 1 status.

Component	Status
9-check validation/verification suite	Implemented and experimentally verified (real 590,540-row dataset, 9/9 PASS)
Independent PII-encryption verification	Implemented and experimentally verified
Dataset regeneration (<code>`regenerate`</code> mode)	Implemented and experimentally verified, cross-checked against lineage report
CI/CD exit-code gating	Implemented; exercised locally (Chapter 21 documents the live-runner limitation)

Chapter 8 — Milestone 2, Module 2: Feature Engineering and Selection

8.1 Preprocessing Architecture

Module 2 separates two concerns that are often conflated: **feature engineering** (deriving new columns from raw data, ``src/features/engineering.py``) and **preprocessing** (imputation, scaling, and encoding of the final selected feature set, ``src/features/preprocessing.py``). Both are implemented as scikit-learn-compatible ``fit`/`transform`` components composed inside a single ``Pipeline``, fit exclusively on the chronological training fold established before either stage runs (Chapter 2.3, Chapter 5.5).

Figure 8.1 shows the module's internal workflow.

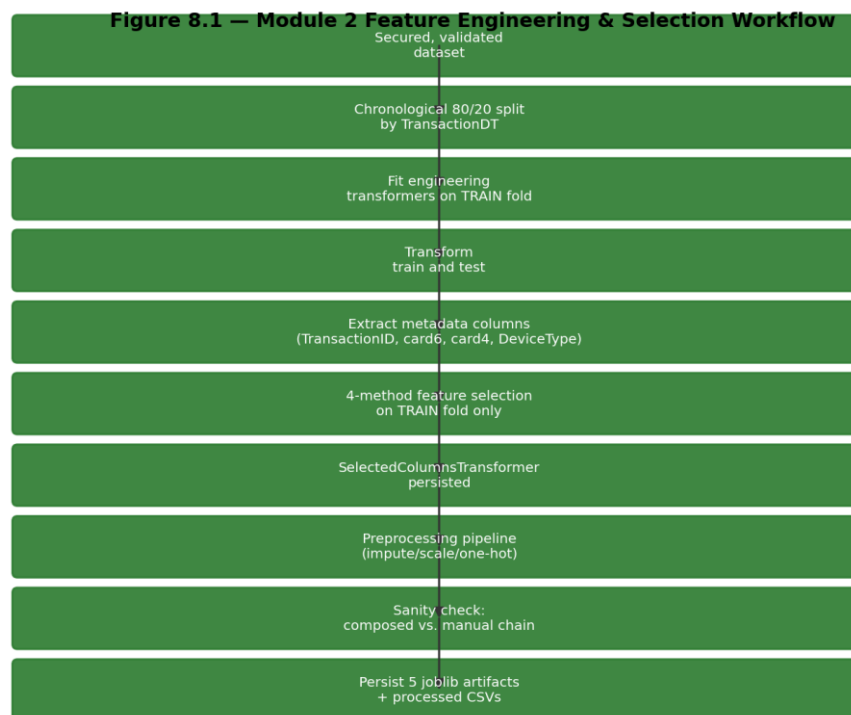


Figure 8.1 — Module 2 workflow. The sanity check at step I compares the fully composed pipeline's output against manually chaining each stage, and raises ``RuntimeError`` on any mismatch before anything is persisted — this is what caught the ``is_fitted_`` marker bug described in Section 8.7.

8.2 Handling Numerical and Categorical Features

Numerical features are scaled with ``RobustScaler`` rather than ``StandardScaler``, a decision directly justified by measured skew: ``TransactionAmt`` has a maximum value near 31,937 against a median near 68 in the real data, so a mean/standard-deviation-based scaler would itself be distorted by outliers that ``RobustScaler``'s median/interquartile-range basis is not. Categorical features are one-hot encoded after selection; the resulting processed columns carry a ``numeric__`` or ``categorical__`` prefix from the underlying scikit-learn ``ColumnTransformer``, a naming convention later modules (and this report's diagrams) must account for

explicitly (this exact naming convention caused a real integration bug in Milestone 5, resolved and documented in Chapter 19).

8.3 Missing-Value Treatment and Feature Creation

Six engineering transformers are applied in sequence: `TimeFeatureEngineer`` (derives `transaction_hour`` and `transaction_day_of_week`` from the raw `TransactionDT`` counter — EDA found a roughly threefold fraud-rate spike at specific hours versus the ~3% daily baseline); `TransactionAmountFeatureEngineer`` (`TransactionAmt_log``, `TransactionAmt_decimal``); `EmailDomainGrouper`` (groups high-cardinality email-domain columns to a training-fold top-N set and derives an `email_domain_match`` flag — the single strongest engineered signal found in EDA: a 9.65% fraud rate when purchaser and recipient email domains match, versus 2.21% when they do not); `CardVelocityFeatureEngineer`` (per-card transaction-count and amount statistics, e.g. `card1_txn_count``, `card1_amt_zscore``); `FrequencyEncoder`` (applied specifically to `id_30``, `id_31``, `id_33``, and `DeviceInfo`` — encodes each value by its training-fold frequency, with unseen values encoded as 0 rather than a target-derived fallback, which would risk leakage); and `MissingIndicatorEngineer`` (hashes each candidate column's null-mask and groups identical hashes, since the Vesta `V`` feature block goes missing in large shared row-patterns across dozens of columns simultaneously — collapsing this to one indicator per genuinely distinct missingness pattern, rather than one flag per column).

8.4 Candidate and Selected Feature Counts

Table 8.1 — Feature counts at each stage (source: `reports_m2/feature_selection_summary.json``, `feature_manifest.json``).

Stage	Count
Candidate features after engineering	466
Dropped — near-zero variance	14
Dropped — correlation redundancy	109
Selected (final)	150 (135 numeric, 15 categorical)
Processed columns after one-hot expansion	202
Final width including metadata + target	207

8.5 The Four-Method Selection Consensus

`FeatureSelectionComparator`` (`src/features/selection.py``) runs four methods as a **funnel**, not a parallel vote: the near-zero-variance filter (a column flagged if its single most frequent value exceeds 99.9% of non-null rows) and the correlation-redundancy filter (pairs above 0.95 absolute correlation, keeping whichever member correlates more strongly with the target) run first, cheaply, on the full dataset; only their combined survivors are then scored by the two more expensive methods — mutual information and embedded Random Forest importance — on a systematically time-ordered sample of 50,000 rows; a consensus selection then averages each survivor's normalized rank across both scoring methods and retains the top 150. Recursive Feature Elimination was deliberately excluded as computationally infeasible at this dimensionality and largely redundant with the embedded-importance method already used.

8.6 Leakage Prevention and the Metadata-Column Guarantee

Four columns (``TransactionID``, ``card6``, ``card4``, ``DeviceType``) are extracted into a separate metadata frame **before** feature selection or preprocessing ever runs, so no code path connects them to the model's feature matrix — this is a structural guarantee, not merely a documentation convention, verified by a dedicated test that calls scikit-learn's own ``get_feature_names_out()`` and asserts none of the metadata column names appear in it (Chapter 22). These four columns are preserved specifically for traceability, auditing, and the fairness-slicing work of Chapter 11, never for training.

8.7 A Real Bug Found and Fixed at This Stage

Two real defects were found and fixed during Module 2's development. First, an ``is_fitted_`` marker bug: several of the six engineering transformers are stateless (they derive features via a fixed formula, not a learned statistic), and their original ``fit()`` implementation did not set any trailing-underscore attribute; scikit-learn's ``Pipeline.transform()`` internally calls ``check_is_fitted()``, which looks for exactly such an attribute, so a composed pipeline containing these transformers raised ``NotFittedError`` even after a real, successful ``fit()`` call. This was caught by the composed-pipeline sanity check described in Section 8.1 (Figure 8.1, step I) and fixed by having every transformer's ``fit()`` explicitly set ``self.is_fitted_ = True``. Second, an ``email_domain_match``-ordering bug: computing this flag after ``EmailDomainGrouper``'s domain-grouping step (rather than before) could create a false match between two genuinely different rare email domains that both happened to collapse to the shared "other" category; this was fixed by computing the match flag on the raw, ungrouped domain values first.

8.8 Persisted Artifacts

Table 8.2 — Persisted Module 2 artifacts (``artifacts_m2/``).

Artifact	Purpose
<code>`engineering_pipeline.joblib`</code>	The 6 fitted engineering transformers, composed
<code>`feature_selector.joblib`</code>	The fitted <code>`FeatureSelectionComparator`</code> result
<code>`column_selector.joblib`</code>	<code>`SelectedColumnsTransformer`</code> , the persisted selection outcome
<code>`preprocessing_pipeline.joblib`</code>	The fitted imputation/scaling/encoding pipeline
<code>`full_inference_pipeline.joblib`</code>	All of the above composed into a single reusable pipeline

8.9 Complete Feature List

The complete, 150-feature selected list is provided in **Appendix B** rather than reproduced in this chapter, per the reporting convention of moving very large reference lists out of the main text.

8.10 Risks of High-Dimensional Sparse Data and Limitations of Feature Importance

Embedded Random Forest importance, one of the two expensive scoring methods used in Section 8.5, is known to be biased toward high-cardinality categorical and continuous features over binary ones, and does not by itself account for feature interactions; this is the specific reason the project combines it with mutual information (a different, non-tree-based dependency measure) rather than relying on Random Forest importance alone. The correlation-redundancy filter is also a strictly pairwise method and cannot detect

higher-order (three-or-more-feature) redundancy; this is stated here as a known limitation, not a resolved concern (Chapter 24 revisits it).

8.11 Implementation Status

Table 8.3 — Module 2 status.

Component	Status
6-transformer feature-engineering pipeline	Implemented and experimentally verified
4-method feature-selection consensus	Implemented and experimentally verified
Metadata-column structural exclusion guarantee	Implemented and unit tested
Composed-pipeline sanity check	Implemented and experimentally verified (caught a real bug, Section 8.7)

Chapter 9 — Milestone 2, Module 3: Model Development

9.1 Evaluation Design

Five model families are compared under a chronological `TimeSeriesSplit` cross-validation (walk-forward folds, each using all data up to that point as training — mirroring how a production model would actually be retrained on all data available at a given date, rather than a fixed-size sliding window). Hyperparameter search uses Optuna's Tree-structured Parzen Estimator sampler on a systematically time-ordered sample of 100,000 rows for tractability; the winning configuration for each model family is then refit on the full training set (approximately 401,567 rows after the module's internal three-way split) before final holdout evaluation. Class imbalance is handled algorithmically (`class_weight="balanced"` or an equivalent `scale_pos_weight` computed from the real class ratio), not via resampling (SMOTE was considered and rejected: its linear interpolation in a 150-plus-dimensional, mostly one-hot-encoded space produces synthetic points of questionable validity and risks leaking interpolated-neighbor information across a cross-validation fold boundary). Decision thresholds are optimized per model on a validation slice carved from training data, never the final holdout, using an F1-maximizing or precision-floor strategy (Chapter 2.2).

9.2 The Five Model Families

Logistic Regression is included as an interpretable, well-calibrated linear baseline. **Random Forest** provides a bagged-ensemble, low-variance point of comparison. **XGBoost** and **LightGBM** are both gradient-boosted tree frameworks and are the strongest performers on this kind of structured, tabular fraud data in the literature generally; both were included specifically to compare two different, mature implementations of the same underlying algorithmic family. A **Neural Network** (a small feed-forward architecture — one hidden layer of 64 units, dropout, Adam optimizer) was included partly for architectural diversity and partly because it also serves, unmodified in its winning-configuration form, as the base architecture for the differential-privacy experiment in Chapter 10.

9.3 Real Search Space (LightGBM, the Eventual Winner)

Table 9.1 — LightGBM's Optuna search space (`src/models/model_registry.py::_suggest_lightgbm`).

Hyperparameter	Range	Sampling
<code>`n_estimators`</code>	100–500, step 50	Uniform integer
<code>`num_leaves`</code>	15–255	Uniform integer
<code>`learning_rate`</code>	0.01–0.3	Log-uniform
<code>`subsample`</code>	0.6–1.0	Uniform
<code>`colsample_bytree`</code>	0.6–1.0	Uniform
<code>`min_child_samples`</code>	5–100	Uniform integer

``learning_rate`` uses log-uniform sampling because a useful learning-rate range spans multiple orders of magnitude; a plain uniform search would waste most trials in the less-informative high end of the range. ``num_leaves`` (up to 255) and ``min_child_samples`` jointly control the model's tendency to overfit at this dataset size.

9.4 Real Model Comparison Results

Table 9.2 — Real leaderboard (`reports_m3/model_comparison_leaderboard.csv`, generated from the real training run; holdout is the chronologically held-out 118,108-row test set).

Model	CV PR-AUC (mean $\pm$ std)	Holdout PR-AUC	Holdout ROC-AUC	Holdout F1	Holdout Precision	Holdout Recall	Optuna trials	Train time (s)
LightGBM (selected)	0.5153 $\pm$ 0.0151	0.4869	0.8807	0.4923	0.5987	0.4181	25	389.7
XGBoost	0.4954 $\pm$ 0.0182	0.4604	0.8596	0.4713	0.6180	0.3809	25	206.8
Random Forest	0.4283 $\pm$ 0.0188	0.4366	0.8751	0.4346	0.4962	0.3866	20	340.6
Neural Network	0.1148 $\pm$ 0.0262	0.0934	0.7883	0.1626	0.0914	0.7345	9	734.6
Logistic Regression	0.1539 $\pm$ 0.0243	0.1080	0.7514	0.1698	0.1732	0.1666	20	156.6

9.5 Model Selection Rule and Statistical Comparison

The selection rule is: rank all five models by holdout PR-AUC; if any other model's score is within 1% relative tolerance of the top score, the fastest-to-train model among that near-tied group wins instead (a parsimony tie-break). In this run, LightGBM's PR-AUC lead over the second-place XGBoost (0.4869 vs. 0.4604, more than 5% relative) is clean, so the tie-break rule was never triggered. A paired Wilcoxon signed-rank test was additionally run on the top two models' three cross-validation fold scores (`reports_m3/top_model_statistical_comparison.json`), returning $p = 0.25$ — not significant at $\alpha = 0.05$, an honestly reported limitation of statistical power with only three folds. LightGBM's holdout-metric win is therefore the decisive evidence used for model selection; the underpowered significance test is reported as a caveat, not concealed.

9.6 Why LightGBM

LightGBM was selected because it achieved the highest holdout PR-AUC by a margin exceeding the project's 1% tie-break tolerance, the metric this project treats as primary given the severe class imbalance (Chapter 4.4), and it did so with a training time roughly 1.9 $\times$ faster than the next-closest gradient-boosting alternative (XGBoost) despite an identical Optuna trial budget.

9.7 Error and Threshold-Trade-off Discussion

At LightGBM's optimized decision threshold, precision is 0.5987 and recall is 0.4181: roughly 60% of flagged transactions are genuinely fraudulent, and roughly 42% of all real fraud is caught. This is the direct, quantified trade-off a fraud-operations team would need to weigh: lowering the threshold would catch more fraud (higher recall) at the cost of more false positives (lower precision, more legitimate transactions flagged for review); the project additionally reports `recall_at_precision`, `precision_at_k`, and lift/gain metrics (`reports_m3/lightgbm/`) specifically to answer operational questions such as "how much fraud is caught if only the top 1% of transactions by risk score can be reviewed," which raw PR-AUC and ROC-AUC do not directly answer. A false negative here means a fraudulent transaction is not flagged and proceeds; a false

positive means a legitimate transaction is flagged for review or blocked — the relative cost of each is a business decision outside this report's scope, which is why multiple operating-point metrics are reported rather than a single verdict.

9.8 Real Evaluation Plots

Figure 9.1 (`reports_m3/lightgbm/roc_curve.png`) shows the ROC curve for the selected model on the real holdout set (ROC-AUC 0.8807). **Figure 9.2** (`reports_m3/lightgbm/pr_curve.png`) shows the corresponding Precision-Recall curve (PR-AUC 0.4869) — visibly less optimistic than the ROC curve, illustrating Chapter 4.4's point about ROC-AUC's tendency to look better than PR-AUC under severe imbalance. **Figure 9.3** (`reports_m3/lightgbm/confusion_matrix.png`) shows the confusion matrix at the optimized threshold. **Figure 9.4** (`reports_m3/lightgbm/feature_importance.png`) shows the top LightGBM feature importances, dominated by anonymized Vesta ``V`` features and the engineered per-card statistics from Chapter 8.3. **Figure 9.5** (`reports_m3/lightgbm/lift_gain.png`) shows the lift/gain chart discussed in Section 9.7. **Figure 9.6** (`reports_m3/best_model_learning_curve.png`) and **Figure 9.7** (`reports_m3/best_model_validation_curve.png`) are diagnostic curves computed only for the winning model, a deliberate tractability trade-off documented in the source code.

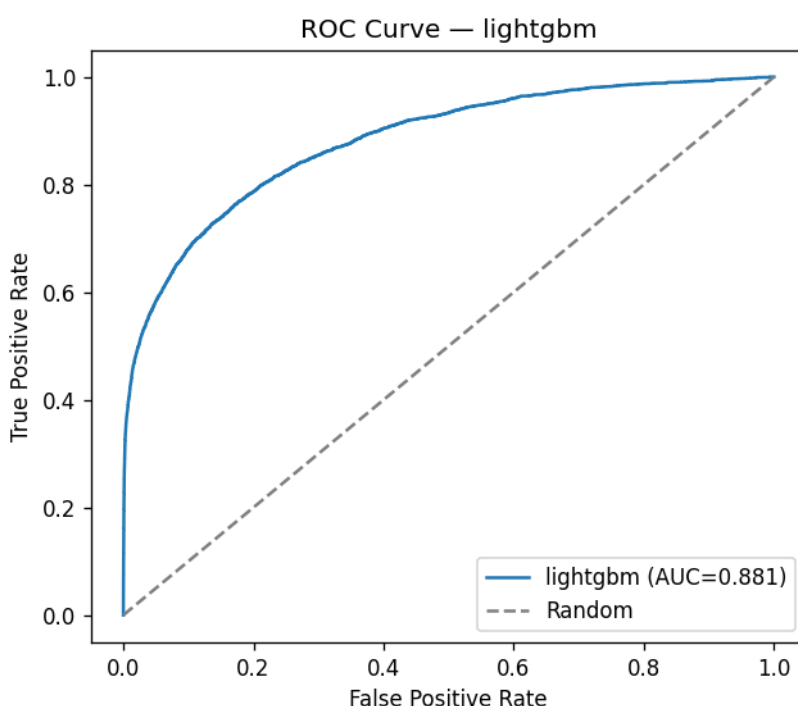


Figure 9.1 — LightGBM ROC curve on the real holdout set (ROC-AUC 0.8807).

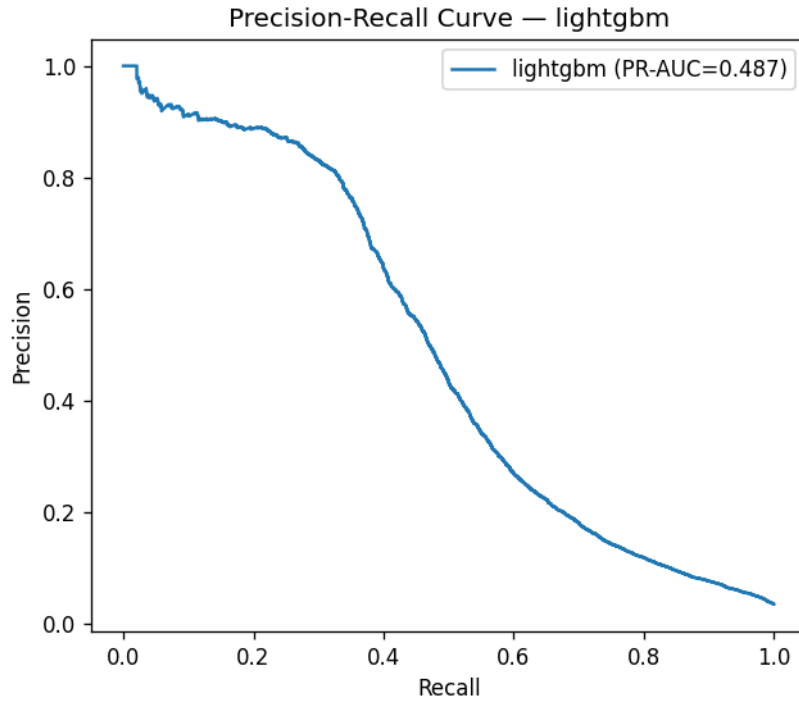


Figure 9.2 — LightGBM Precision-Recall curve on the real holdout set (PR-AUC 0.4869).

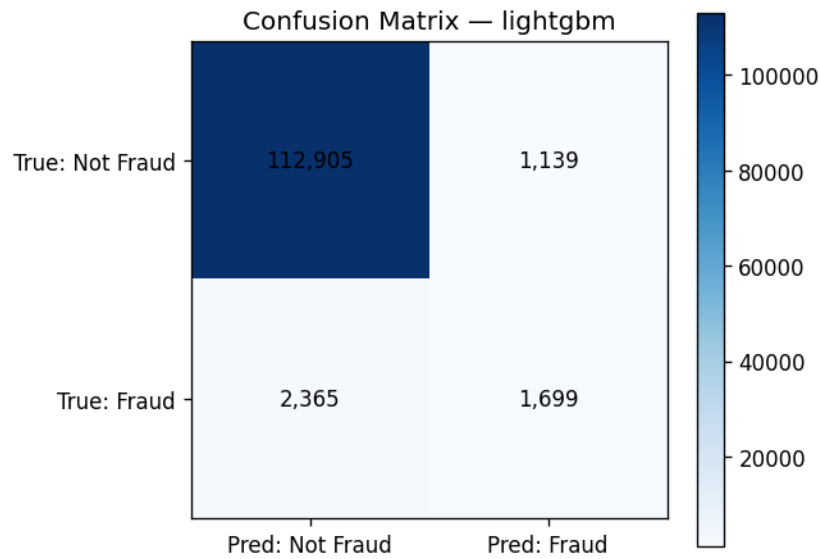


Figure 9.3 — LightGBM confusion matrix at the optimized decision threshold.

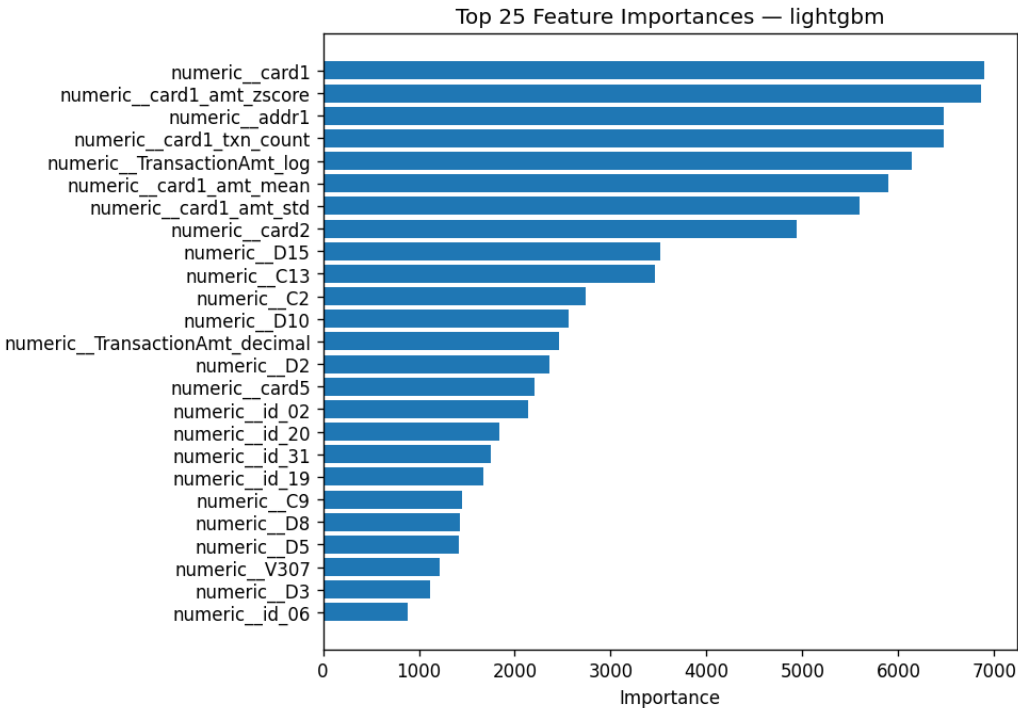


Figure 9.4 — LightGBM top feature importances.

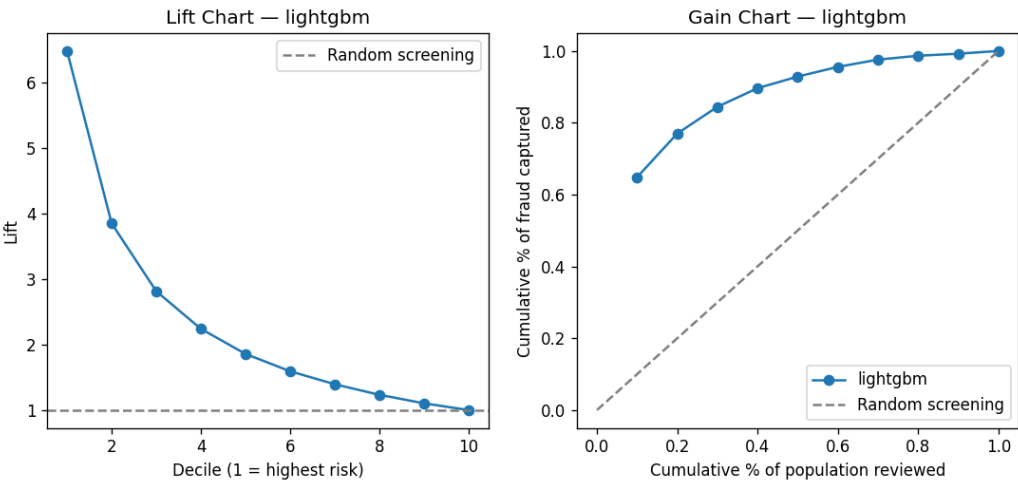


Figure 9.5 — LightGBM lift and gain charts.

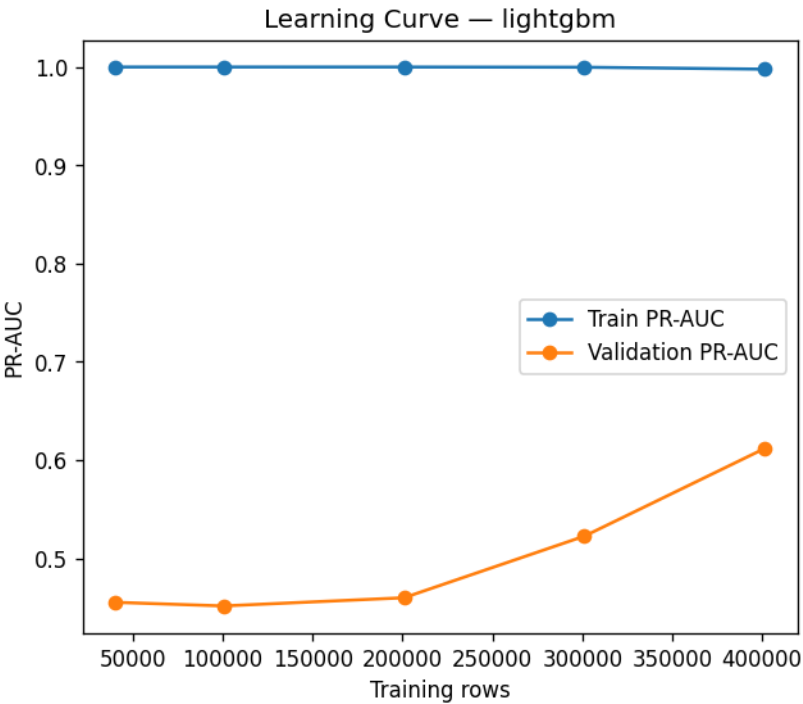


Figure 9.6 — Learning curve for the winning model.

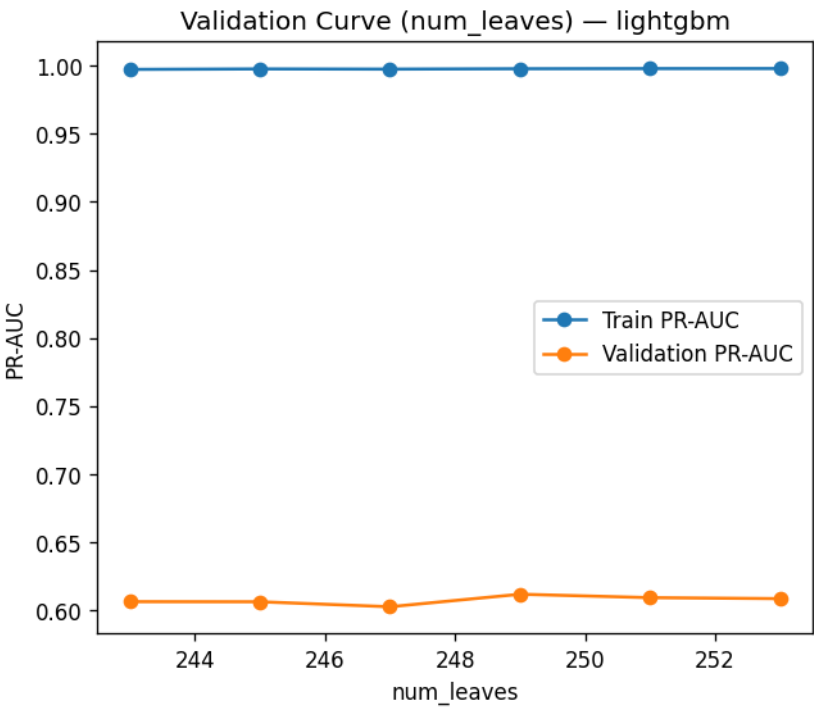


Figure 9.7 — Validation curve for the winning model.

9.9 Implementation Status

Table 9.3 — Module 3 status.

Component	Status
5-model comparison with Optuna tuning	Implemented and experimentally verified
Chronological cross-validation	Implemented and experimentally verified
Model-selection rule (tie-break + statistical comparison)	Implemented and experimentally verified
Operational metrics (recall@precision, precision@K, lift/gain)	Implemented and experimentally verified
Azure ML Model Registry integration	Implemented, code-complete; never executed against a live Azure workspace (no credentials available)

Chapter 10 — Milestone 2, Module 4: Differential Privacy

10.1 Why Privacy Protection Is Relevant Here

A model trained without any privacy protection can, in principle, be probed to infer whether a specific individual's transaction was part of its training set (membership inference), or can memorize and later reveal rare or unusual training examples. This risk is directly relevant to a fraud model, which trains on financial transaction data. Module 4 evaluates whether a formal, mathematically bounded privacy guarantee can be added to this project's neural-network architecture (the same architecture compared in Chapter 9) without destroying its usefulness, and — because the answer turned out to be more complicated than a single number — investigates *why* in detail.

10.2 DP-SGD Mechanics as Implemented

Differentially Private Stochastic Gradient Descent (Chapter 2.6) is implemented via TensorFlow Privacy's `DPKerasAdamOptimizer`. Two configuration details matter for correctness and are recorded here as verified implementation facts, not assumptions: `num_microbatches` is set equal to `batch_size`, giving true per-example gradient clipping — verified directly against TensorFlow Privacy's source code, since a smaller `num_microbatches` value would average several examples together before a single clip-and-noise step, a materially weaker privacy formulation. And the loss function passed to the optimizer uses **unreduced, per-example loss** (`reduction="none"`), since DP-SGD requires per-example gradients to exist before they can be clipped.

10.3 Verified Identical Initial Weights

To isolate the optimizer as the only variable between the private and non-private runs, both models must start from bit-for-bit identical initial weights. The first implementation attempted this via `tf.random.set_seed()` before each build and was found, empirically, not to work: TensorFlow derives each random operation's actual seed from both the global seed and a per-process operation-creation counter that keeps advancing across separate build calls, so two "identically seeded" builds produce measurably different initial weights. This was caught by a dedicated unit test comparing weight checksums, not assumed. The fix builds the architecture once, caches the resulting weights via `get_weights()`, and explicitly applies them to every subsequent build via `set_weights()` — a correctness guarantee that does not depend on TensorFlow's random-number-generator internals at all. In the real run, both variants' weight checksums equaled 980.497437.

10.4 The Real Experiment

Table 10.1 — Real DP experiment configuration and result (`reports_m4/dp_report.md`).

Parameter	Value
Model architecture	The same single-hidden-layer neural network selected as the winning Optuna configuration in Chapter 9
Privacy accountant	TensorFlow Privacy's RDP accountant (<code>_compute_dp_sgd_example_privacy</code>)
Noise multiplier (final reported config)	Per the investigation's best-found configuration (Section 10.6)
L2 norm clip (final reported config)	20.0 (best-found; original default was 1.0)

Delta (δ)	1×10^{-5}
Epsilon (ϵ), non-Poisson-subsampling (authoritative)	45.573
Epsilon (ϵ), Poisson-subsampling assumption (informational only)	3.932
Non-private PR-AUC	0.1646
Private (DP) PR-AUC	0.0285
Relative PR-AUC change	-82.7%
Normal training time	14.9 seconds
DP training time	264.8 seconds ($\approx 17.8\times$ slower)

10.5 Why Two Epsilon Values Are Reported, and Why Only One Is Authoritative

TensorFlow Privacy's RDP accountant computes epsilon two ways depending on the sampling assumption. The 3.932 figure assumes Poisson (random-per-step) subsampling, which this project's actual training loop — standard shuffled-epoch training via Keras `model.fit()` — does not use; reporting the smaller, Poisson-assumption epsilon here would overstate the model's real privacy guarantee. The 45.573 figure assumes the standard shuffled-epoch procedure the code actually executes, and is therefore the authoritative value used throughout this project's reporting and MLflow logging. This distinction was verified directly against the accountant's own source code, not assumed from general DP literature.

10.6 The Root-Cause Investigation

The first full run of the DP experiment produced a below-random ROC-AUC of **0.2991** — worse than chance, not merely lower than the non-private model — which triggered an eight-phase investigation (`reports_m4/investigation/`) rather than acceptance of the number as "expected DP noise."

Table 10.2 — Investigation phases and findings.

Phase	Question tested	Result
Phase 0 — Gradient-norm diagnostic	How large are raw, unclipped per-example gradients?	Extremely heavy-tailed; positive-class (fraud) gradients, amplified by <code>class_weight</code> ($\approx 28\text{--}38\times$), reached up to $\approx 147,000\times$ the original clip bound of 1.0
Phase A — L2 norm clip sweep	Does raising the clip bound fix it?	Only marginally: ROC-AUC moved from 0.2856 to 0.2869 across a $100\times$ clip-bound range (0.5–50); best value 20.0 adopted, but insufficient alone
Phase B — <code>class_weight</code> ablation	Is class-weight amplification the cause?	Small, real, but insufficient effect: ROC-AUC moved from 0.2894 (no weighting) to 0.2975 (original weighting) — confirms per-example clipping neutralizes most, not all, of the amplification
Phase C — Learning-rate sweep	Is the optimizer's step size the cause?	Investigated as a contributing factor; did not by itself explain the magnitude of the failure
Control experiment	Is this a data/split artifact rather than a DP-SGD effect?	A non-DP model trained on the identical 100,000-row exploration subsample scored ROC-AUC 0.7451 (healthy) — ruling out the data/split as the cause
Seed-sensitivity check	Is this random noise variance across runs?	The best-found DP configuration re-run at 3 seeds produced a tight cluster (ROC-

		AUC 0.287, 0.299, 0.288) — ruling out run-to-run noise as the explanation
Phase D — Noise-multiplier recovery sweep	Does performance recover as noise $\rightarrow 0$?	Yes, partially: ROC-AUC rose from 0.287 to 0.398 as noise multiplier fell toward 0.01 ($\epsilon \approx 242,112$ — an essentially unprotected privacy level) — but even there, performance still fell far short of the non-private model's ≈ 0.75 – 0.80 , proving a noise-independent ceiling exists
Phase E — Full-dataset confirmation	Does the diagnosed effect hold at full scale, not just the exploration sample?	Confirmed

10.7 Root-Cause Synthesis

The investigation's conclusion is a structural tension, not a single misconfiguration: per-example gradient clipping (a non-negotiable mechanism of DP-SGD, capping every example's influence equally) directly counteracts `class_weight`'s amplification of the minority (fraud) class's gradients, which this project's severe 3.5% class imbalance (Chapter 4.3) otherwise relies on to make the model learn the minority class at all. This sets a hard utility ceiling well below the non-private model's performance even with negligible added noise, and adding privacy-meaningful noise degrades performance further from there. This is stated as this project's own conclusion, directly evidenced by the phase-by-phase results in Table 10.2, not asserted as a general property of differential privacy.

10.8 Honest Statement of the Result

The experiment demonstrates that adding differential privacy to this project's neural-network fraud classifier is technically feasible, but the resulting configuration produces a substantial utility cost (an 82.7% relative reduction in PR-AUC) and is not yet suitable as the primary production fraud-detection model. The root cause is a structural interaction between per-example gradient clipping and this dataset's severe class imbalance, not a fixable configuration error, and further work (Section 10.9) is required before a differentially private model could be considered for production use at an acceptable utility level.

10.9 Recommended Future Improvements (Not Implemented)

The investigation's own conclusion recommends, without having implemented or evaluated any of the following: a materially larger or different neural architecture with more capacity to survive clipping-induced signal loss; resampling the training set before DP training rather than loss reweighting — a fundamentally different imbalance-handling strategy compatible with DP-SGD's per-example clipping; and, if neither restores acceptable utility, accepting a materially weaker privacy guarantee than this project's original goal implied. Additional directions not evaluated in this project, offered here as future work only: privacy amplification via subsampling (which would require restructuring the training loop to match the Poisson-subsampling assumption discussed in Section 10.5); PATE (Private Aggregation of Teacher Ensembles) or other alternative privacy mechanisms; privacy-preserving feature engineering; and federated-learning

architectures, which change where and how training data is aggregated in the first place. None of these are labeled as implemented anywhere else in this report.

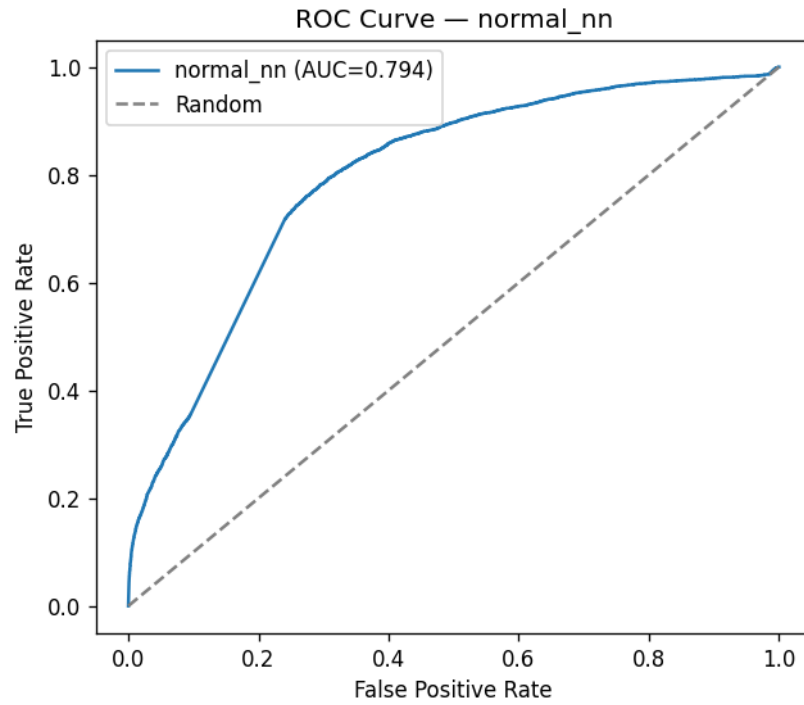


Figure 10.1 — Normal (non-private) neural network ROC curve.

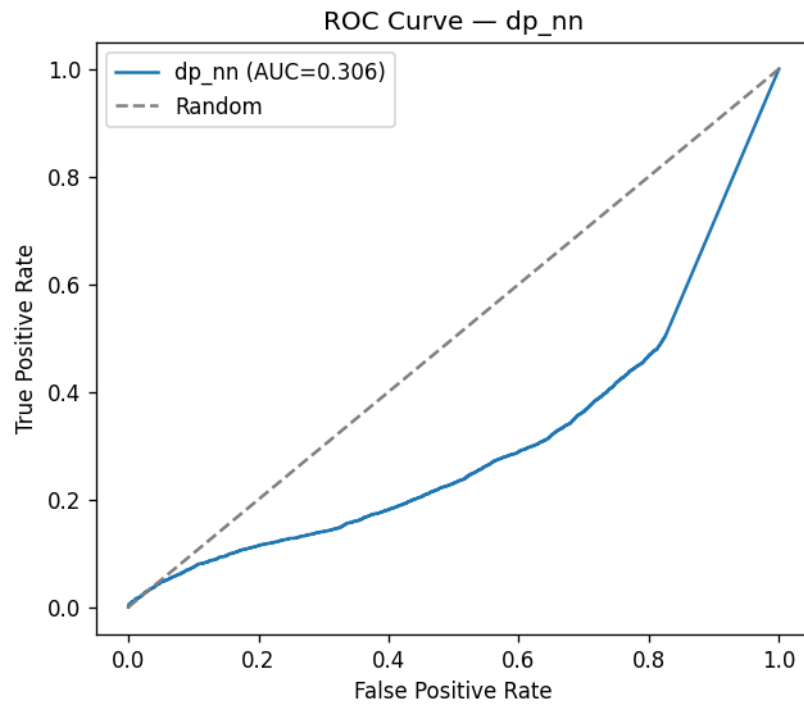


Figure 10.2 — Differentially private neural network ROC curve.

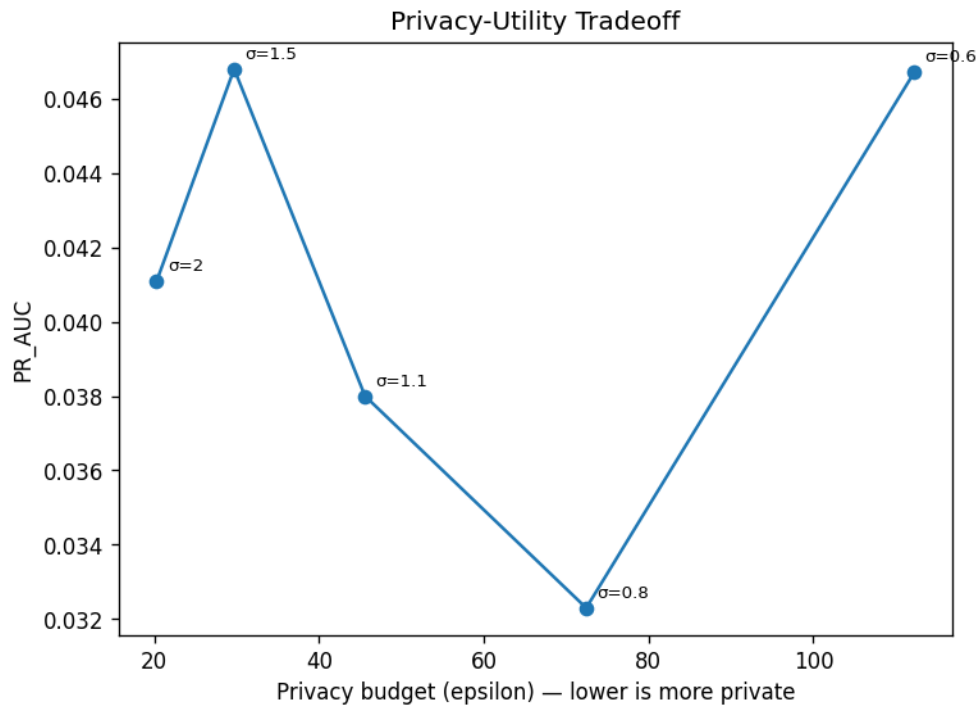


Figure 10.3 — Privacy-utility curve across the noise-multiplier recovery sweep (Section 10.6, Phase D).

10.10 Implementation Status

Table 10.3 — Module 4 status.

Component	Status
DP-SGD training with verified per-example clipping	Implemented and experimentally verified
RDP privacy accounting (authoritative epsilon)	Implemented and experimentally verified
Root-cause investigation (8 phases)	Implemented and experimentally verified
Recommended architectural/resampling improvements (Section 10.9)	Not implemented; future work

Chapter 11 — Milestone 2, Module 5: Fairness Analysis

11.1 Why Fairness Matters Here, and What "Protected Group" Means in This Dataset

A fraud model that performs well in aggregate can still treat identifiable subgroups of transactions very differently — flagging one group's legitimate activity far more often than another's, or missing a much larger share of real fraud within one group than another. The IEEE-CIS dataset contains **no genuine demographic field**; this project's fairness analysis instead uses three payment-behavior or device-usage attributes as proxies, and this report — following the repository's own stated convention — never refers to a finding on these attributes as a "demographic bias" finding. The three attributes analyzed are `card6` (debit vs. credit, the primary attribute), `card4` (card network: Visa, Mastercard, American Express, Discover), and `DeviceType` (desktop vs. mobile, analyzed with an explicit "Missing" group rather than dropping the 80.4% of test rows with no recorded device).

11.2 Metric Definitions as Implemented

Selection Rate, SPD, DIR, EOD, and AOD are defined in Chapter 2.7. Every pairwise metric in this project is reported with a **95% bootstrap confidence interval** (1,000 stratified resamples — each protected-attribute group resampled independently, preserving its relative size, rather than resampling the pooled dataset as one unit) and is only described as **statistically significant** when that interval excludes the metric's no-disparity value (0 for a difference metric, 1 for a ratio metric). A chi-square or Fisher's exact test (chosen automatically per Cochran's rule — at least 80% of the contingency table's expected cell counts must be ≥ 5 for chi-square to be valid, falling back to Fisher's exact test for a 2x2 table when this rule is violated) additionally tests whether a group's selection rate differs from the reference at all. Every core metric implementation was independently cross-validated against **Fairlearn** — an external, independently developed fairness library — and every compared metric agreed to a difference of exactly 0.0 on real data, per `tests/test_fairlearn_validation.py``.

11.3 Real Findings — card6 (Primary Attribute; reference group: debit)

Table 11.1 — card6 real metrics (`reports\_m5/fairness\_metrics.csv`, `group\_metrics.csv`).

Metric	Value	95% CI	Significant?
Selection rate (credit)	5.65%	—	—
Selection rate (debit)	1.41%	—	—
TPR (credit / debit)	0.5247 / 0.3357	—	—
FPR (credit / debit)	0.02419 / 0.00576	—	—
SPD	0.0424	[0.0395, 0.0453]	Yes
DIR	3.996	[3.701, 4.298]	Yes
EOD	0.1890	[0.1606, 0.2188]	Yes
AOD	0.1037	[0.0894, 0.1190]	Yes

Credit-card transactions are flagged as fraud at approximately **four times** the rate of debit-card transactions, and this model catches real fraud among credit-card transactions roughly 19 percentage points more often

than among debit-card transactions. Every metric's confidence interval excludes its null value; this is a statistically robust, not a marginal, finding.

11.4 Real Findings — card4 (reference group: Visa)

Table 11.2 — card4 real metrics, one row per non-reference network.

Comparison	SPD [95% CI]	DIR [95% CI]	EOD [95% CI]	AOD [95% CI]	Significant (any metric)?
American Express vs. Visa	0.0092 [−0.0007, 0.0199]	1.402 [0.968, 1.884]	0.0210 [−0.152, 0.202]	0.0160 [−0.072, 0.107]	No
Discover vs. Visa	0.0488 [0.0348, 0.0629]	3.139 [2.495, 3.791]	−0.0316 [−0.120, 0.060]	0.0006 [−0.046, 0.049]	SPD, DIR significant
Mastercard vs. Visa	0.0017 [−0.0001, 0.0035]	1.075 [0.995, 1.158]	0.0504 [0.0169, 0.0853]	0.0258 [0.0086, 0.0431]	EOD, AOD significant

This table concretely illustrates why a single fairness metric is insufficient: Mastercard and Visa have statistically indistinguishable overall flagging rates (SPD, DIR not significant), yet Mastercard's fraud-catch rate differs significantly from Visa's (EOD, AOD significant) — the same pair of groups is fair by one definition and not by another, simultaneously.

11.5 Real Findings — DeviceType (reference group: desktop; "Missing" reported but excluded from pairwise comparison)

Table 11.3 — DeviceType real metrics.

Metric	Value	95% CI	Significant?
Selection rate (mobile)	10.24%	—	—
Selection rate (desktop)	8.16%	—	—
TPR (mobile / desktop)	0.6035 / 0.7153	—	—
SPD	0.0207	[0.0132, 0.0283]	Yes
DIR	1.254	[1.157, 1.361]	Yes
EOD	−0.1119	[−0.1512, −0.0715]	Yes
AOD	−0.0504	[−0.0699, −0.0297]	Yes

The most operationally significant finding in this project's fairness analysis concerns the explicit **"Missing"** group: 80.4% of the real test set (94,936 of 118,108 rows) has no recorded `DeviceType`, and this group's **false-negative rate is 86.1%** — the model catches barely one in seven real frauds when device information is absent, versus a substantially higher catch rate when it is present. This surfaced specifically because the project's design deliberately treats "Missing" as its own analyzable group in per-group metrics tables rather than silently dropping those rows, even though "Missing" is excluded from pairwise SPD/DIR/EOD/AOD comparisons against the reference group, since "no device recorded" is not a meaningful protected-group contrast in the traditional sense.

11.6 The 24 Bias Findings

`bias\_findings.md` and `fairness\_summary.json` record **24 total automatic findings** (0 critical, 12 warning, 12 informational), 8 per attribute: the largest and smallest SPD pair (informational), whether any group shows a statistically significant selection-rate disparity (warning if so), whether the chi-square/Fisher's test result is

significant (warning if so), the best- and worst-performing group by F1 score (informational / warning), and the group with the highest FPR and highest FNR (informational / warning). Across all three attributes, the consistently worst-performing group by F1 score is the "Missing" group (F1 = 0.368 for both card6 and card4's missing values, 0.198 for DeviceType's missing value) — the same operational concern as Section 11.5, surfacing independently through the automatic-findings mechanism.

11.7 Operational Implications and Remediation Options

These findings identify measurable risk; they do not, by themselves, establish unlawful discrimination or prove the model is broken — `card6`'s disparity is consistent with (though not proof of) genuine, legitimate risk correlation, since Module 2's own exploratory data analysis (Chapter 8) already found real fraud-rate differences by payment behavior independent of any model. Remediation options this report identifies as **future work, not implemented**: escalate the missing-`DeviceType` false-negative-rate finding to the modeling team as a genuine model-quality and data-coverage gap, since it stems from missing information rather than a protected-group contrast; consider fairness-aware post-processing (e.g., group-specific threshold calibration) if the card6 disparity is judged unacceptable after business review; and gather additional context (beyond what this dataset provides) before concluding whether the underlying disparity reflects genuine risk or an artifact worth correcting.

11.8 Fairness Assessment Does Not Establish Fairness

This report states explicitly, consistent with the repository's own documentation discipline: fairness measurement identifies statistically supported disparities and their plausible operational significance. It does not certify that the model is fair, does not constitute a legal or regulatory fairness determination, and does not by itself resolve whether an identified disparity should be corrected, tolerated, or requires more context to interpret.

Fairness Dashboard — card6

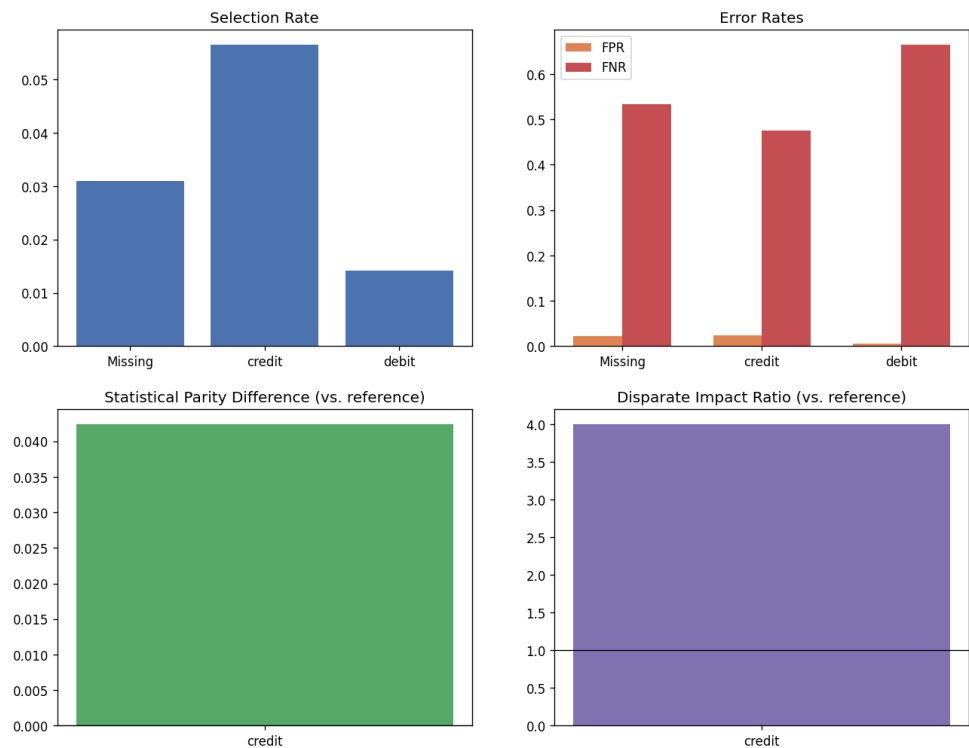


Figure 11.1 — Real fairness dashboard (card6, primary attribute).

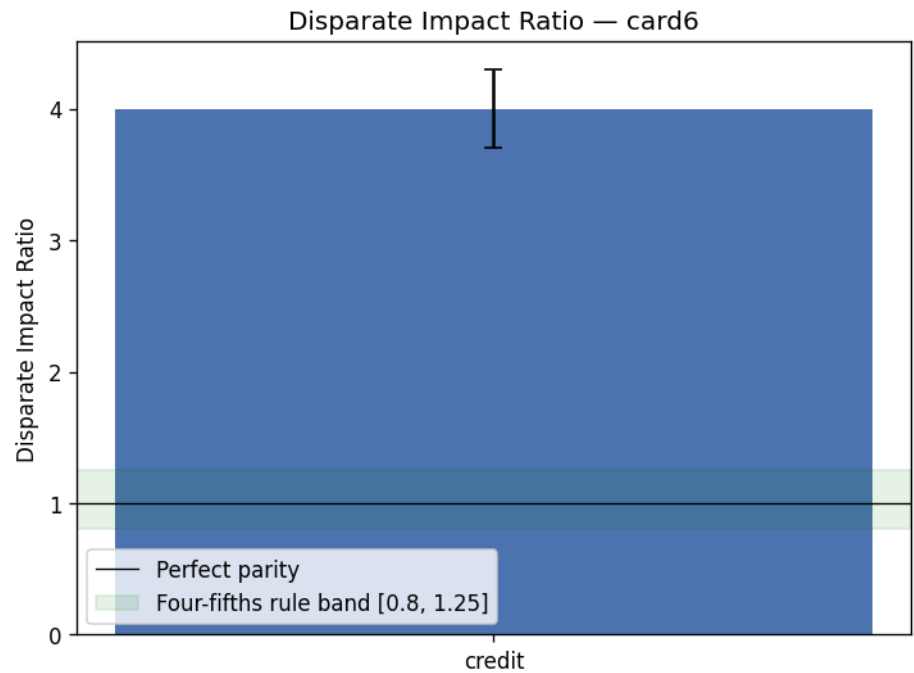


Figure 11.2 — card6 disparate impact visualization.

11.9 Implementation Status

Table 11.4 — Module 5 status.

Component	Status
SPD/DIR/EOD/AOD with bootstrap confidence intervals	Implemented and experimentally verified
Fairlearn independent cross-validation	Implemented and experimentally verified (exact agreement, difference 0.0)
Cochran's-rule-based significance testing	Implemented and experimentally verified
"Missing"-group explicit handling	Implemented and experimentally verified (produced the DeviceType finding, Section 11.5)

Chapter 12 — Experiment Tracking and Model Artifact Management

12.1 MLflow Integration

Modules 3, 4, and 5 log to **MLflow**, backed by a SQLite tracking store (`sqlite:///mlruns/mlflow.db``), switched from the plain filesystem store after that was found deprecated in MLflow 3.x during development — a real, minor compatibility bug found and fixed in this project, recorded here for completeness. Module 3's five-model comparison is logged as a single parent run ("model_comparison") with five nested child runs, one per candidate model, each carrying its own hyperparameters, cross-validation and holdout metrics, evaluation plots, and the fitted model in the appropriate MLflow flavor. Module 4 logs a parent run ("differential_privacy_comparison") with nested Normal/DP child runs. Module 5 logs a single "fairness_analysis" run. In total, the real tracking store contains 10 runs: 1 parent + 5 children (Module 3), 1 parent + 2 children (Module 4), and 1 run (Module 5).

12.2 Parameters, Metrics, and Artifacts Logged

Each MLflow run records its model's hyperparameters (the exact Optuna-selected configuration), cross-validation and holdout metrics (PR-AUC, ROC-AUC, F1, precision, recall, and the operational metrics from Chapter 9.7), the model itself, and its evaluation plots. This is distinct from — and in addition to — the plain-file artifacts described below, which are what this repository actually commits to version control; the MLflow store (`mlruns/``) is excluded from version control via `.gitignore`` and exists only on the machine where the pipeline was run.

12.3 Model Persistence: joblib and Keras Artifacts

Conventional (scikit-learn-API) models — Logistic Regression, Random Forest, XGBoost, and LightGBM — are persisted via `joblib``. The Neural Network and both differential-privacy variants are persisted in Keras's native format (`model.keras``), alongside a JSON metadata sidecar recording full training and privacy configuration in a format readable without loading TensorFlow at all. Every one of the five candidate models — not only the winning LightGBM — is persisted as a complete `InferenceBundle`` (Chapter 8.8, Chapter 9), a single joblib artifact bundling the fitted engineering, selection, and preprocessing pipelines together with the trained estimator and its optimized decision threshold, so any candidate can be loaded and used for inference without rebuilding any preprocessing step.

12.4 Versioning and Reproducibility

The `dataset_version`` hash (Chapter 6.5) is threaded through every persisted manifest and every MLflow run's tags, so a reader can always trace a given model artifact or metric back to the exact dataset version that produced it. `reports_m2/feature_manifest.json`` similarly records the exact selected-feature list, the artifact paths of every Module 2 pipeline object, and the `final_processed_column_count`` (202), giving a complete, versioned record of the feature contract any downstream consumer — including the Milestone 3 API (Chapter 13) — must satisfy.

12.5 Configuration Management

A single typed configuration source, ``src/config/config.yaml`` (245 lines, loaded via ``load_config()`` into a frozen ``AppConfig`` dataclass), governs Milestone 2's behavior across all five modules — feature-engineering parameters, feature-selection thresholds, model-development settings, differential-privacy parameters, and fairness-analysis thresholds are all declared here rather than scattered across module-specific configuration files. Secrets (Azure credentials) are never hardcoded in this file; ``${VAR_NAME}``-style placeholders are resolved against environment variables at load time, with unset variables resolving to an empty string rather than raising, so local development without credentials configured does not crash.

12.6 Local Tracking vs. Committed Artifacts vs. Production Registry Target

This project maintains three distinct tiers, and this report does not conflate them. **Local experiment tracking** (the MLflow SQLite store, ``mlruns/``) exists only on the machine that ran the pipeline and is not committed to version control. **Repository-committed artifacts** (everything under ``artifacts_m2/`` through ``artifacts_m5/`` and ``reports_m2/`` through ``reports_m5/``) are real, generated outputs committed to the repository specifically so a reader can inspect them without re-running the pipeline. The **production registry target architecture** — Azure ML Model Registry integration (``src/registry/azure_ml_registry.py``) — is implemented and code-complete but was never executed against a live Azure ML workspace in this project, since no credentials were available; it logs a clear "skipping, ready to run once credentials are supplied" message and returns ``None`` rather than silently failing or fabricating a registration record.

Chapter 13 — Milestone 3: Secure Inference API

13.1 Application Structure

The service is a FastAPI application (`app/main.py`) organized into `app/api/` (route handlers: `auth.py`, `inference.py`), `app/core/` (`config.py`, `security.py`, `limiter.py`, `keyvault.py`), and `app/models/` (`schemas.py`, `model_loader.py`, `raw_schema.py`). All routes are mounted under an **API version prefix**, `/api/v1`, set once in configuration (`API_V1_PREFIX`) rather than hardcoded per route.

13.2 Authentication Flow

Authentication uses the OAuth2 password flow, issuing a **JWT** signed with HS256. `POST /api/v1/auth/token` accepts a username and password, verifies the password against a bcrypt hash (`passlib`), and — on success — returns a bearer token containing the subject and a coarse-grained `role` claim (`ml_engineer` or `service_client`), valid for **15 minutes** (`ACCESS_TOKEN_EXPIRE_MINUTES`). Every subsequent authenticated request presents this token in an `Authorization: Bearer` header; `get_current_user` decodes and validates the signature and expiry, and `require_role(*roles)` is a dependency factory enforcing coarse-grained, role-based access on each route. The demo user store (`_FAKE_USER_DB`, a single hardcoded `svc-ml-engineer` account with a bcrypt-hashed placeholder password) is explicitly documented in code as a stand-in to be replaced by Azure AD (Entra ID) before production go-live — this is also Finding F4 in the penetration-test report (Chapter 17) and is treated in this report as an open, unresolved item, not a completed control.

13.3 Credential Handling

The JWT signing secret (`JWT_SECRET_KEY`) is resolved from Azure Key Vault when `ENVIRONMENT=production` and a vault URL is configured; if resolution fails or is unavailable, the application **refuses to start** in production mode rather than falling back to an insecure default. In non-production environments, an explicit "dev-only-insecure-key" fallback is used, clearly distinguishing the two paths in code so a misconfiguration cannot silently run production with a disclosed default secret.

13.4 The Prediction Endpoint

`POST /api/v1/inference/predict` requires `ml_engineer` or `service_client` role. Its request schema (`TransactionFeatures`, `app/models/schemas.py`) uses `extra="forbid"` (Pydantic) to reject any unexpected field outright — a first line of defense against malformed input and mass-assignment style attacks. Rather than modeling this project's ~150 selected features as individually typed fields (an unreasonable API surface), the schema exposes explicit, bounded, validated fields for the raw columns the feature-engineering transformers actually read directly (`TransactionAmt`, `ProductCD`, `card1`–`card6`, `addr1`, `P_emaildomain`, `R_emaildomain`, `DeviceType`, `DeviceInfo`, `id_30`, `id_31`, `id_33`), plus an open, key-validated `extra_features` dictionary for the remaining `C*/D*/M*/V*/id_*` columns — validated against the exact real raw-column set derived from the committed dataset header and `reports_m2/feature_manifest.json`, not a guessed schema. `_to_ml_ready_frame` assembles a single-row pandas DataFrame from a NaN-defaulted template covering the full real raw-column contract, overlaid with whatever the request supplied — letting the pipeline's own imputers and frequency encoders (Chapter 8.3) handle any column the caller did not provide, exactly as they were designed to.

13.5 Model Loading and the Real-vs-Placeholder Distinction

`ModelService`` (`app/models/model_loader.py`) loads the model artifact at `MODEL_PATH`, defaulting to the real, committed `artifacts_m3/lightgbm_inference_bundle.joblib`. If that artifact is absent, it falls back to a deterministic placeholder scorer (a naive amount-based heuristic) so the rest of the security pipeline can be built and tested independently of the ML deliverable — an intentional design choice, not an oversight. `ModelService`` detects which case applies (via `hasattr(model, "threshold_decision")`), a signature unique to a real `InferenceBundle``) and dispatches accordingly: the real bundle is called with a `DataFrame` and uses its own tuned decision threshold (Chapter 9.6), never the conventional default of 0.5.

13.6 Input Validation and Error Handling

Beyond Pydantic's field-level validation (bounds, types, `extra="forbid"`), a global exception handler (`app/main.py`) catches every unhandled exception, logs the full traceback server-side, and returns a generic `{"detail": "Internal server error. Please contact support."}` to the client — no stack trace or internal error detail is ever exposed externally.

13.7 Rate Limiting

A single shared `SlowAPI` Limiter`` instance (`app/core/limiter.py`) is imported by every router, so rate-limit state and the app-level exception handler apply uniformly. The general API rate limit is **30 requests/minute** (configurable, `RATE_LIMIT`); the login endpoint carries a stricter, independently hardcoded **5 requests/minute** limit specifically to blunt credential-stuffing and brute-force attempts against the token endpoint — this stricter limit was added in direct response to Finding F1 in the penetration test (Chapter 17), and its remediation was independently retested.

13.8 Security Headers and Logging

A middleware layer (`app/main.py`) adds `Strict-Transport-Security``, `X-Content-Type-Options``, `X-Frame-Options: DENY``, `Content-Security-Policy: default-src 'none'``, `Referrer-Policy: no-referrer``, and a restrictive `Permissions-Policy`` to every response. The `Server: uvicorn`` header (which cannot be stripped from application middleware, since uvicorn sets it after the ASGI app returns) is instead disabled at the server level via `--no-server-header`` in the Dockerfile's production `CMD``, an example of remediating a control at the correct architectural layer rather than working around a limitation. Structured, PII-free audit logging records who called what: request ID, authenticated subject, role, and timestamp — explicitly never the raw feature values or the prediction itself, since those may be derived from or reveal sensitive transaction data.

13.9 Health and Documentation Endpoints

`GET /health`` (unauthenticated, unversioned) returns `{"status": "ok", "environment": ...}`` for liveness/readiness probing, with no sensitive information disclosed. Interactive documentation (`/docs``, `/redoc``) is enabled only when `ENVIRONMENT != "production"`; both were verified to return 404 with `ENVIRONMENT=production`` set (Chapter 17, Test T9).

13.10 Secrets Abstraction and Local Fallback

``app/core/keyvault.py`` wraps the Azure Key Vault Secrets client using ``DefaultAzureCredential``, so no credential is stored anywhere in application code, container images, or environment variables in the target Azure deployment — the compute resource's own managed/workload identity is used to authenticate. Locally, ``DefaultAzureCredential`` falls back to ``az login`` developer credentials, and ``.env.example`/`.env`` provide the local-development-only fallback described in Section 13.3.

13.11 API Endpoint Table

Table 13.1 — Real API endpoints.

Method	Route	Auth required	Input	Output	Security controls
<code>`POST`</code>	<code>`/api/v1/auth/token`</code>	None (issues token)	Username, password (form-encoded)	JWT bearer token	bcrypt password hashing; 5 req/min rate limit
<code>`POST`</code>	<code>`/api/v1/inference/predict`</code>	Bearer JWT, role <code>`ml_engineer`</code> or <code>`service_client`</code>	<code>`TransactionFeatures`</code> JSON	<code>`PredictionResponse`</code> (<code>is_fraud</code> , <code>fraud_probability</code> , <code>model_version</code> , <code>request_id</code>)	JWT validation; RBAC; 30 req/min rate limit; strict schema validation; audit logging
<code>`GET`</code>	<code>`/health`</code>	None	—	<code>`{"status", "environment"}`</code>	No sensitive data exposed
<code>`GET`</code>	<code>`/docs`</code> , <code>`/redoc`</code>	None	—	Interactive API documentation	Disabled when <code>`ENVIRONMENT=production`</code>

13.12 API Request Sequence

Figure 13.1 shows the real, code-verified request flow for a prediction call.

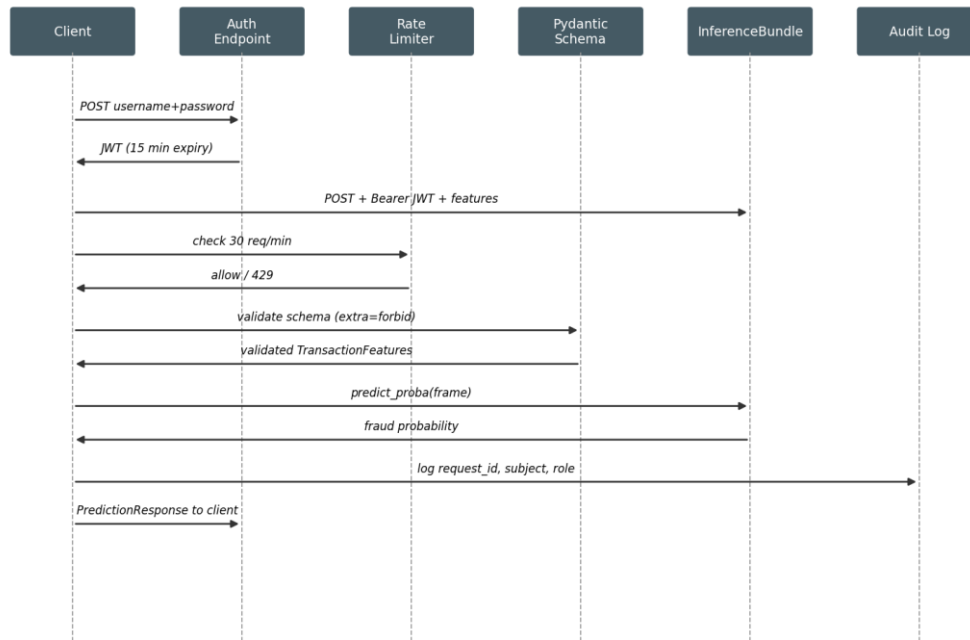
Figure 13.1 — Secure API Request Sequence

Figure 13.1 — API request sequence, based on the actual implementation of ``app/api/auth.py`` and ``app/api/inference.py``.

13.13 Implementation Status

Table 13.2 — Milestone 3 API status.

Component	Status
JWT authentication + RBAC	Implemented and locally tested; penetration-tested (Chapter 17)
Rate limiting (general + auth-specific)	Implemented and locally tested; penetration-tested, one finding remediated
Real feature-contract request schema	Implemented and experimentally verified against the real trained model
Security headers	Implemented and locally tested; penetration-tested
Azure Key Vault secret resolution	Implemented, code-complete; not exercised against a live Key Vault (Chapter 15)
Production identity provider (Azure AD)	Not implemented; open item (Chapter 17, Finding F4)

Chapter 14 — Container and Deployment Security

14.1 Multi-Stage Docker Build

The `Dockerfile` uses a two-stage build: a `builder` stage (`python:3.12-slim`) installs dependencies from `requirements-milestone3.txt` into an isolated user site-packages directory; the runtime stage copies only that installed-dependency directory and the application code (`app/`, `src/`, `artifacts_m3/` — the latter two required so the real `InferenceBundle` can be unpickled and scored inside the container, verified directly during this project's integration work) into a fresh, minimal `python:3.12-slim` base. No build toolchain, source archives, or package caches from the builder stage are present in the final image.

14.2 Non-Root Execution and Attack-Surface Reduction

A dedicated `appuser` system user and group are created, and file ownership is transferred via `chown` before the container drops privileges (`USER appuser`) — the application never runs as root inside the container. The production `CMD` invokes `uvicorn` directly (no shell form), with `--workers 2` for basic concurrency, and `--no-server-header --no-date-header` to remove fingerprinting information at the server layer (Chapter 13.8). `PYTHONDONTWRITEBYTECODE=1` and `PYTHONUNBUFFERED=1` are set for a smaller runtime footprint and immediate log flushing.

14.3 Container Limitations

The image is built and validated locally (this project's integration work confirmed it builds and the resulting service runs and responds correctly) but has not been pushed to a container registry or run inside a live Kubernetes cluster; the Kubernetes manifests below reference an Azure Container Registry image (`acr fraudml.azurecr.io/fraud-inference-api:latest`) that does not exist in any provisioned registry, using an untagged `:latest` reference rather than an immutable digest — a concrete, named limitation (Chapter 24) rather than a claim of production readiness.

14.4 Kubernetes Manifests

Table 14.1 — All Kubernetes resource objects defined (`k8s/`, 6 objects across 3 files).

File	Kind	Name	Key configuration
<code>00-namespace-serviceaccount.yaml</code>	Namespace	<code>fraud-inference</code>	Label <code>purpose: ml-inference</code>
<code>00-namespace-serviceaccount.yaml</code>	ServiceAccount	<code>fraud-inference-sa</code>	Annotated for Azure Workload Identity federation (client ID placeholder, filled from Terraform output)
<code>01-deployment-service.yaml</code>	Deployment	<code>fraud-inference-api</code>	3 replicas; non-root (UID 1000); seccomp <code>RuntimeDefault</code> ; <code>readOnlyRootFilesystem: true</code> ; <code>allowPrivilegeEscalation: false</code> ; capabilities dropped (<code>ALL</code>); requests 250m CPU / 256Mi memory, limits 500m / 512Mi; liveness/readiness probes on <code>/health</code> ;

			`emptyDir` volume mounted at `/tmp` (since root filesystem is read-only)
`01-deployment-service.yaml`	Service	`fraud-inference-api`	`ClusterIP` (not directly internet-facing); port 443 → container port 8000
`02-networkpolicy.yaml`	NetworkPolicy	`default-deny-all`	Zero-trust baseline: denies all ingress/egress in the namespace by default
`02-networkpolicy.yaml`	NetworkPolicy	`allow-appgw-ingress`	Permits ingress on port 8000; the manifest's own comment recommends tightening the current namespace-wide rule to the exact Application Gateway subnet CIDR once confirmed supported by the cluster's CNI
`02-networkpolicy.yaml`	NetworkPolicy	`allow-egress-to-keyvault-and-dns`	Permits egress to the private-endpoint subnet (443, for Key Vault) and DNS (53) only

14.5 Separating Implemented Controls from Recommendations

Every control in Table 14.1 is present in the committed YAML and would be enforced if applied to a real cluster; none of it has been applied to a live cluster (Section 15 explains why). The one control explicitly flagged as incomplete **in the manifest's own comments** — the overly broad `allow-appgw-ingress` rule — is preserved here as a recommendation rather than silently corrected, to accurately reflect the manifest's actual current state.

Chapter 15 — Azure Infrastructure-as-Code Architecture

15.1 Overview and Explicit Non-Deployment Statement

The Terraform configuration under `iac/terraform/` represents a designed target architecture and has not been verified through deployment to a live Azure subscription.

This statement is placed prominently because several of this project's most security-relevant claims depend on it: no live AKS cluster, no live Web Application Firewall, and no live Key Vault instance exist as a result of this repository's contents. The Terraform is internally consistent (its resources reference each other correctly and it would pass `terraform validate`), and several concrete pieces of evidence confirm it has never been applied: its remote-state backend is commented out rather than configured; the sensitive variables `jwt\_secret\_key` and `field\_encryption\_key` have no default values and require external injection; `allowed\_admin\_ips` defaults to an empty list, which would itself block administrative access if applied as-is; `terraform.tfvars.example` uses placeholder values (`security@example.com`); and no `.tfstate` file exists anywhere in the repository.

15.2 Azure Resource Inventory by Module

Table 15.1 — Network module.

Resource	Purpose
<code>`azurerm_resource_group`</code>	Container for all project resources
<code>`azurerm_virtual_network` (10.20.0.0/16)</code>	Address space for the deployment
3× <code>`azurerm_subnet`</code>	AKS nodes (10.20.1.0/24), Application Gateway/WAF (10.20.2.0/24), private endpoints (10.20.3.0/24)
3× <code>`azurerm_network_security_group`</code>	Per-subnet inbound rules — AKS accepts 443 only from the AppGW subnet; AppGW accepts public 443 only; private-endpoint subnet accepts 443 only from the AKS subnet
<code>`azurerm_private_dns_zone`</code> + link	Enables private-endpoint name resolution for Key Vault

Table 15.2 — AKS module.

Resource	Purpose
<code>`azurerm_kubernetes_cluster`</code>	Private cluster (no public API server), RBAC enabled, Workload Identity/OIDC enabled, Azure network policy plugin (enforces Kubernetes NetworkPolicies), Microsoft Defender enabled
<code>`azurerm_kubernetes_cluster_node_pool` ("inference")</code>	Dedicated user-mode node pool separate from the system pool
<code>`azurerm_user_assigned_identity`</code> + <code>`azurerm_federated_identity_credential`</code>	Binds the Kubernetes <code>`fraud-inference-sa`</code> ServiceAccount to an Azure AD identity via OIDC — no stored credential

Table 15.3 — Key Vault module.

Resource	Purpose
<code>`azurerm_key_vault`</code>	SKU standard; purge protection enabled; 90-day soft-delete retention; RBAC authorization; public network access disabled
<code>`azurerm_private_endpoint`</code>	Sole network path to the vault
2× <code>`azurerm_role_assignment`</code>	Least-privilege: "Key Vault Secrets User" (read-only) for the AKS workload identity; "Key Vault Certificate User" for the Application Gateway's TLS certificate access
3× <code>`azurerm_key_vault_secret`</code>	<code>`jwt-secret-key`</code> , <code>`allowed-origins`</code> , <code>`field-encryption-key`</code> — values injected at apply time, never committed
<code>`azurerm_monitor_diagnostic_setting`</code>	Audits every secret access to the Log Analytics workspace

Table 15.4 — Monitoring module.

Resource	Purpose
<code>`azurerm_log_analytics_workspace`</code>	Central log store, 90-day retention
<code>`azurerm_security_center_subscription_pricing` (×4)</code>	Microsoft Defender for Cloud, Standard tier, across containers, Key Vaults, App Services, and ARM
<code>`azurerm_security_center_auto_provisioning`</code> + <code>`azurerm_security_center_contact`</code>	Automatic agent deployment; a required alert-contact email

Table 15.5 — WAF module.

Resource	Purpose
<code>`azurerm_web_application_firewall_policy`</code>	Prevention mode (blocks, not just logs); OWASP Core Rule Set 3.2; a tuned exclusion on the <code>`TransactionAmt`</code> parameter; three custom rules (Section 15.3)
<code>`azurerm_application_gateway`</code>	SKU <code>`WAF_v2`</code> , autoscale capacity 2–6; HTTPS-only frontend; TLS policy <code>`AppGwSslPolicy20220101S`</code> (TLS 1.2 minimum); backend routes to the AKS ClusterIP service over HTTPS
<code>`azurerm_public_ip`</code>	Static IP for the gateway's public frontend

15.3 WAF Custom Rules (verbatim from `docs/WAF\_Rules.md`)

Table 15.6 — Custom WAF rules.

Priority	Name	Trigger	Action
1	`RateLimitAuthEndpoint`	More than 60 requests/minute per client IP to `/api/v1/auth/token`	Block
2	`BlockNonJsonInference`	Request to `/api/v1/inference/predict` without `Content-Type: application/json`	Block
3	`BlockDisallowedGeos`	Client from a country code in `blocked_country_codes`	Block (disabled by default, pending confirmed client geography)

Request limits enforced at the WAF: `request\_body\_check = true`, maximum request body 128 KB, maximum file upload 10 MB.

15.4 Defense in Depth

Three independent layers protect the inference endpoint in the target architecture: the WAF (network edge, Section 15.3), application-level rate limiting (Chapter 13.7), and Pydantic input validation (Chapter 13.4/13.6). Losing any single layer still leaves the other two in place — this is the explicit design rationale recorded in `docs/WAF\_Rules.md`.

15.5 Azure Deployment Diagram

Figure 15.1 shows the designed target architecture end to end.

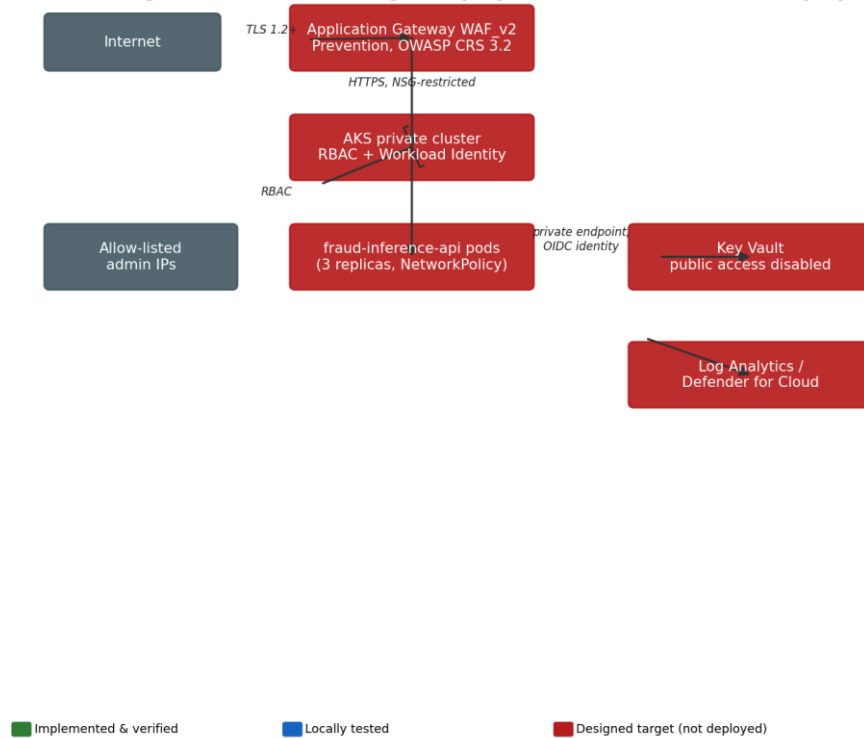
Figure 15.1 — Azure Target Deployment Architecture (Not Deployed)

Figure 15.1 — Azure target deployment architecture. No component in this diagram has been applied to a live subscription (Section 15.1).

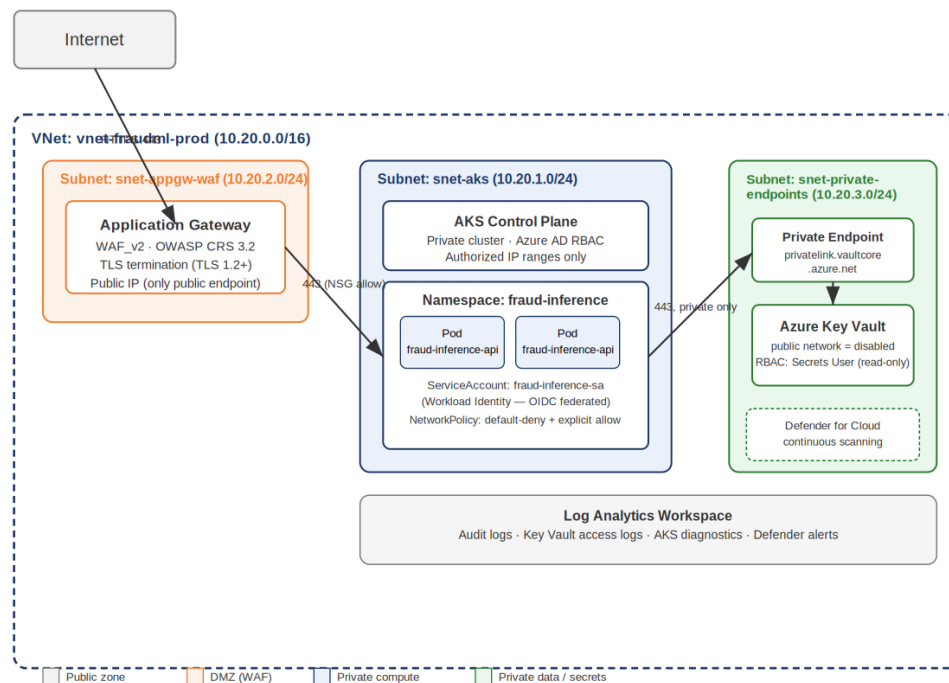


Figure 15.2 — Network diagram from the real Milestone 3 Security Architecture Documentation.

15.6 What This Report Does Not Claim

Consistent with Section 15.1, this report does not claim: a successful AKS deployment; live WAF traffic blocking; live Key Vault secret retrieval by a running pod; or any measured production scalability or availability, since none of these have occurred.

15.7 Implementation Status

Table 15.7 — Milestone 3 infrastructure status.

Component	Status
Docker multi-stage, non-root container build	Implemented and locally tested (built and run successfully)
Kubernetes manifests (Deployment, Service, NetworkPolicy)	Designed target state; internally consistent; not applied to a live cluster
Terraform (network, AKS, Key Vault, monitoring, WAF modules)	Designed target state; internally consistent; not applied to a live subscription

Chapter 16 — Threat Model and Security Analysis

16.1 Assets, Actors, and Trust Boundaries

Assets: the trained model artifacts (``artifacts_m2`-`artifacts_m5``); the JWT signing secret and Fernet field-encryption key; the inference API's authentication tokens; audit and application logs; CI/CD secrets (Azure Storage credentials referenced by Milestone 5's workflow); and, in the target architecture, the provisioned Azure resources themselves. **Actors:** an external, unauthenticated API client; an authenticated ``ml_engineer`/`service_client`` role holder; a repository contributor with CI/CD trigger access; and an Azure administrator (in the target architecture only, since no live Azure environment exists). **Entry points:** the public inference API surface (target: via the Application Gateway/WAF); the CI/CD pipeline's trigger conditions (push, pull request, schedule, manual dispatch); and, in the target architecture, the AKS API server (mitigated by being a private cluster with an admin IP allow-list, Chapter 15.2). Trust boundaries mirror Chapter 5.4 (Figure 5.2).

16.2 STRIDE Threat Table

Table 16.1 — STRIDE threats, grounded in this repository's actual controls and gaps.

Category	Threat	Asset	Existing control	Residual risk	Verification status
Spoofing	Attacker impersonates a legitimate client to call the inference API	Inference API, JWT	JWT signature + expiry validation (Ch. 13.2)	Demo user store is a hardcoded placeholder, not a real identity provider (Ch. 13.2, Ch. 17 F4)	Implemented and locally tested; open item remains
Tampering	Request payload altered in transit; a compromised node alters model input/output	API traffic, model artifacts	TLS termination at the Application Gateway (target); strict Pydantic schema validation; read-only container filesystem	TLS enforcement not verified live (no deployed AppGW); container image uses <code>`:latest`</code> , not a digest	Designed target; not live-verified
Repudiation	A caller denies having made a request that led to a fraud decision	Audit trail	Structured, PII-free audit logging of request ID, subject, role, timestamp (Ch. 13.8)	Logs are not yet shipped to the immutable WORM store automatically per-request (only periodic reports are, Ch. 20)	Implemented and locally tested
Information Disclosure	PII, model internals, secrets, or stack traces leak to an unauthorized party	PII fields, secrets, error responses	Fernet field encryption (Ch. 6); generic error responses, no stack traces (Ch. 13.6); server-fingerprint header removed (Ch. 13.8); Key Vault has no public network path (target, Ch. 15)	Local <code>`secret.key`</code> file, not a managed secret store, for the encryption key (Ch. 6.7)	Partially implemented; key management gap identified
Denial of Service	Flooding the API or auth endpoint to	API availability	Application-level rate limiting	No live load-testing evidence;	Application-level control verified;

	exhaust resources or lock out legitimate users		(30/min general, 5/min auth, Ch. 13.7); WAF rate-limit custom rule (target, Ch. 15.3); AKS/AppGW autoscaling (target)	WAF/autoscaling protections unverified without a live deployment	infrastructure-level control is a designed target only
Elevation of Privilege	A low-privilege caller or compromised pod gains <code>`ml_engineer`</code> -level or administrative access	RBAC, cluster access	JWT role-claim-based RBAC (<code>`require_role`</code> , Ch. 13.2); Kubernetes NetworkPolicy default-deny (Ch. 14); Key Vault RBAC grants read-only "Secrets User" only (target, Ch. 15.2)	Kubernetes controls not live-verified; role model is coarse-grained (two roles only)	Application-level control verified; infrastructure-level control is a designed target only

16.3 Additional ML- and Supply-Chain-Specific Threats

Table 16.2 — Threats beyond classic STRIDE, specific to an ML system.

Threat	Description	Existing control	Verification status
Model extraction	Repeated querying of the inference API to reconstruct a functionally equivalent model	Rate limiting bounds query volume (Ch. 13.7); no dedicated extraction-specific defense implemented	Partially mitigated; not a targeted control
Model inversion / membership inference	Inferring properties of, or membership in, the training set from model outputs	The differential-privacy variant (Ch. 10) is designed to bound exactly this risk; the production LightGBM model carries no such formal guarantee	DP variant: implemented and experimentally verified (with the utility caveat of Ch. 10); production model: not protected
Adversarial input	Crafted feature values intended to force a misclassification	Field-level bounds validation (Ch. 13.4); no adversarial-robustness-specific testing performed	Not independently verified; recommended future work (Ch. 25)
Data poisoning / training-serving skew	Corrupted or drifted training or serving data silently degrading the model	Module 1 validation (Ch. 7); drift monitoring (Ch. 19)	Validation: implemented and experimentally verified; drift: locally verified, no live production traffic
Secret leakage (CI/CD)	Azure Storage credentials referenced by the Milestone 5 workflow are exposed	GitHub Actions <code>`secrets`</code> context (never printed to logs); workflow falls back to dry-run when unset rather than exposing an error requiring the secret's value (Ch. 20, Ch. 21)	Implemented and locally verified
Dependency compromise	A malicious or compromised upstream Python package is installed	Per-milestone pinned requirements files; no automated dependency or SBOM scanning currently configured	Not implemented; recommended future work (Ch. 21, Ch. 25)
Container escape / Kubernetes lateral movement	A compromised container pod attempts to access the host or move to other pods	Non-root execution, dropped Linux capabilities, read-only root filesystem (Ch. 14);	Designed target; not live-verified

		Kubernetes NetworkPolicy default-deny (Ch. 14)	
CI/CD supply-chain compromise	A malicious pull request or compromised Action alters the pipeline	Workflow triggers scoped to specific branches/paths; no branch-protection or required-review evidence found in the repository	Partially addressed; recommended hardening (Ch. 25)
Compliance-evidence tampering	Audit reports altered after generation to misrepresent the system's actual state	WORM (immutable) storage design, dry-run verified (Ch. 20)	Dry-run verified only; live immutability requires a provisioned Azure Storage account

16.4 Summary

No threat in Tables 16.1–16.2 is presented as fully closed without qualification; every row states its actual verification status. The threats with the weakest current mitigation are the placeholder identity provider (Section 16.2, Spoofing), local (non-managed) key storage (Section 16.2, Information Disclosure), and the absence of dependency/SBOM scanning (Section 16.3) — each is carried forward into Chapter 25's prioritized recommendations.

Chapter 17 — Penetration Testing and Security Validation

17.1 Scope and Methodology

`pentest/Penetration\_Test\_Report\_Milestone3.docx` documents a grey-box, source-available assessment performed by Ali Yasser (Security & DevSecOps Engineer, Milestone 3 owner) against a local pre- production instance of the FastAPI inference service, structured around the **OWASP API Security Top 10 (2023)**. This is a real, evidence-backed manual test against a running local instance, not a description of an untested design.

17.2 Tests Performed

Table 17.1 — All 10 real test cases and their results (source: `pentest/` document, verbatim).

ID	Test case	Expected	Actual	Result
T1	`POST /inference/predict` with no Authorization header	401	401 "Not authenticated"	PASS
T2	`POST /inference/predict` with a malformed/garbage JWT	401	401 "Could not validate credentials"	PASS
T3	Injection-style string (`' OR 1=1--`) in the `ProductCD` field	Safely handled	200, treated as an opaque string value	PASS
T4	`TransactionAmt = 99,999,999` (out of declared bounds)	422	422 `less_than_equal`	PASS
T5	Extra field `is_fraud` injected into the request body (mass assignment)	422	422 `extra_forbidden`	PASS
T6	Wrong `Content-Type` (`text/plain`) with a non-JSON body	4xx, no crash	422 `model_attributes_type`	PASS

T7	6 rapid login attempts with the wrong password (limit: 5/min)	429 after the 5th attempt	401 x5, then 429 (after remediation)	PASS
T8	Malformed JSON body to <code>/inference/predict`</code>	Structured error, no stack trace	422 <code>`json_invalid`</code> , no traceback	PASS
T9	<code>`GET /docs` and <code>`/redoc`</code> with <code>`ENVIRONMENT=production`</code></code>	404	404 / 404	PASS
T10	Security response headers on the health-check endpoint	HSTS, CSP, X-Frame-Options, X-Content-Type-Options present	All present	PASS

17.3 Findings

Table 17.2 — All 4 findings, severity, and remediation status.

ID	Finding	Severity	OWASP category	Remediation	Retest status
F1	No rate limiting on <code>`/auth/token`</code> , enabling unlimited authentication attempts	High	API2:2023 Broken Authentication	Dedicated 5 req/min limit added via SlowAPI (Ch. 13.7)	Remediated and verified
F2	<code>`Server: uvicorn`</code> response header disclosed server technology	Low	API8:2023 Security Misconfiguration	Disabled at the server level via <code>`--no-server-header`</code> (Ch. 13.8, Ch. 14.2)	Remediated and verified
F3	Interactive API documentation (<code>`/docs`</code> , <code>`/redoc`</code>) reachability if misconfigured	Informational	API8:2023 Security Misconfiguration	Verified disabled via <code>`ENVIRONMENT=production`</code> configuration (no code change required)	Verified safe
F4	Placeholder local user store (hardcoded demo service account)	Medium	—	Recommended: replace with Azure AD (Entra ID) app registration / client-credentials flow	Open — tracked as a pre-production blocker

17.4 OWASP API Security Top 10 Coverage

Table 17.3 — Per-category assessment (condensed from the pentest report).

Category	Status
API1 Broken Object Level Authorization	Not applicable (no per-object resource identifiers exposed)
API2 Broken Authentication	Tested — remediated (F1)
API3 Broken Object Property Level Authorization	Tested — pass (<code>`extra="forbid"`</code> rejects mass assignment)
API4 Unrestricted Resource Consumption	Tested — remediated (rate limiting; WAF layer in target architecture)
API5 Broken Function Level Authorization	Tested — pass (<code>`require_role`</code> enforces RBAC)
API6 Unrestricted Access to Sensitive Business Flows	Recommend further testing once deployed under realistic traffic
API7 Server-Side Request Forgery	Not applicable (the API does not fetch user-supplied URLs)
API8 Security Misconfiguration	Tested — remediated (F2, F3)
API9 Improper Inventory Management	Partial — versioned under <code>`/api/v1`</code> ; recommend a living service/secret inventory
API10 Unsafe Consumption of APIs	Not applicable (the service does not consume third-party APIs)

17.5 Distinguishing Evidence Types

This report distinguishes, per its own accuracy rules: the 178 automated **unit and integration tests** (Chapter 22) verify code-level correctness; the 10 **manual penetration tests** above verify the running service's actual HTTP-level security behavior; and there is currently no automated **API security scanning** (SAST/DAST) integrated into CI/CD (Chapter 21 records this as a recommendation, not an implemented control).

Chapter 18 — Milestone 4: Compliance Quality Assurance

18.1 Purpose and Materials Reviewed

Milestone 4 (``compliance/``, authored by Farida Elgharbawy) is an independent audit of Milestone 1 (data security) and Milestone 3 (secure deployment) against GDPR, HIPAA, and ISO/IEC 27001 themes. Its deliverables are ``compliance_certificate.txt``, ``reports/compliance_audit_report.md``, ``reports/security_review_report.md``, ``checklists/compliance_register.xlsx``, and a test file, ``tests/test_compliance.py``. This chapter analyzes these materials critically against the rest of this report's independently verified evidence, rather than reproducing their claims uncritically.

18.2 GDPR-Relevant Concepts and Their Application

Lawfulness and purpose limitation: this project processes a public research dataset plus synthetic, non-real PII for demonstration purposes (Chapter 4.8); no real personal data collection or processing occurs in the repository's current form. **Data minimization:** feature selection (Chapter 8) reduces the model's input to 150 statistically justified features rather than every available column, though this is a modeling decision, not a GDPR-driven data-minimization exercise. **Storage limitation:** no data-retention or deletion policy is implemented in code; this is a gap this report notes rather than a claim of compliance. **Integrity and confidentiality:** addressed by Milestone 1's field-level encryption (Chapter 6) and, in the target architecture, Milestone 3's network isolation (Chapter 15). **Accountability and privacy by design:** the project's own discipline of verifying rather than assuming security claims (Chapter 7) and its formal differential-privacy evaluation (Chapter 10) are genuine evidence of privacy-by-design intent, even where the DP variant's utility cost currently limits its production usefulness. **Data-subject rights and breach management:** no mechanism for handling data-subject access, erasure, or rectification requests exists in this codebase; this is a gap, not an oversight this report should minimize. **Automated decision-making:** the fraud model does make automated decisions with real consequences (flagging a transaction); this report notes that GDPR Article 22-style safeguards (a right to human review of an automated decision) are not implemented in this system.

18.3 HIPAA Applicability

HIPAA governs **protected health information** processed by a **covered entity** or its **business associate**. This project's dataset is a payment-transaction dataset; it contains no health information, and this repository does not implement or claim any covered-entity relationship. This report therefore does **not** treat HIPAA as automatically applicable to this system, and treats any compliance-document language implying otherwise as an intended-target-state or illustrative reference rather than a genuine applicability claim (Section 18.6 returns to this explicitly).

18.4 ISO/IEC 27001 Control Themes

ISO/IEC 27001 defines an information security management system standard covering control areas including access control, cryptography, physical and environmental security, operations security, and incident management. This project addresses several relevant themes at the code and design level — cryptography (Chapter 6), access control (Chapter 13's RBAC, Chapter 15's target Key Vault RBAC), and logging/monitoring (Chapter 13.8, Chapter 15.2's target Log Analytics/Defender integration) — but holds no certification against

the standard, and this report does not claim one. Referencing these control themes as a structure for analysis is not equivalent to certification.

18.5 Compliance Control Mapping and Gap Analysis

Table 18.1 — Condensed control mapping (the complete version is in **Appendix G**).

Control theme	Milestone 4's claim	This report's independently verified evidence	Gap / risk rating
Field-level encryption	"AES-256 symmetric encryption" (compliance report, verbatim)	Fernet, which uses AES-128-CBC + HMAC-SHA256 (Chapter 6.2, verified against source code and real encrypted values)	Documentation inconsistency — Medium priority to correct
Deployment verification	Validation "across live Azure Kubernetes Service (AKS) endpoints" (README, verbatim)	No live AKS cluster has ever been provisioned (Chapter 15.1); the FastAPI service was run and tested only locally	Documentation inconsistency — High priority to correct, since it materially overstates deployment maturity
Secret management architecture	"Full decoupling of operational tokens... via Azure Active Directory Workload Identities"	Implemented in Terraform design (Chapter 15.2) and application code (Chapter 13.10); not exercised against a live Key Vault	Design-verified; live-verification gap, Medium priority
Authentication/authorization enforcement	"Verified robust OAuth2/JWT token gateway enforcement... Unauthenticated calls are securely blocked via 401"	Independently confirmed true at the application level by the real penetration test (Chapter 17, tests T1–T2) and this project's own verification	Accurate , and independently corroborated
PII/data-leakage prevention	"Automated validation scans confirm zero structural or plaintext leakage of PII/PHI... into downstream EDA logs"	Independently confirmed true by Module 1's real, automated `pii_columns_encrypted` check (Chapter 7.3), though Milestone 4's own test file does not itself exercise this check (Section 18.7)	Accurate as a system-level claim , though not verified by Milestone 4's own test code
Operational sign-off	"APPROVED AND PRODUCTION READY" (certificate, verbatim)	Contradicted by: the DP utility cost (Chapter 10), the non-provisioned Azure infrastructure (Chapter 15), the open placeholder-identity-provider finding (Chapter 17, F4), and the WORM dry-run-only status (Chapter 20)	Overstated — this report does not adopt this conclusion

18.6 Framing These Inconsistencies

These findings are presented as **documentation and verification gaps**, not as a critique of the Milestone 4 author's competence or intent. It is plausible that "AES-256" and "live AKS" describe an intended target state that was written before, or independently of, the exact implementation choices documented elsewhere in this repository (Fernet was likely selected by Milestone 1's author for sound reasons — it is a well-regarded, correctly implemented authenticated encryption recipe; it is simply not AES-256). This report's purpose in stating the discrepancy plainly is to ensure a reader relying on this report alone does not carry forward an inaccurate description of the system's actual cryptographic or deployment state.

18.7 Milestone 4's Own Test Suite

``compliance/tests/test_compliance.py`` (preserved verbatim, unmodified, and attributed in the repository) contains two tests intended to validate encryption and API-authentication-bypass behavior. Neither, on inspection, exercises real system code: the encryption test constructs a hardcoded, fabricated "encrypted" string and a hardcoded "plaintext" string inline and merely asserts they differ — a comparison that is true by construction regardless of whether any real encryption occurred; the API-bypass test issues an HTTP request to a hardcoded placeholder URL (``https://api.secure-ml-pipeline.azure.com/predict``) that has never been deployed (Section 18.5), which raises a connection error that the test catches and turns into an unconditional ``pytest.skip``, so it never actually executes an assertion in any real environment. This repository's later integration work (documented in its own engineering history) added a second test file, ``tests/test_compliance_real.py``, implementing the same intent — real Fernet ciphertext recognized by the actual ``DataValidator``, and a real HTTP 401 from the actual running FastAPI application — against real, executable code, specifically because these two original tests, while well-intentioned, do not provide the verification they appear to provide.

18.8 Current State vs. Intended Target State

Table 18.2 — Verified current state versus intended target state, Milestone 4 topics.

Topic	Milestone 4 documentation states	Verified current state
Encryption algorithm	AES-256	Fernet (AES-128-CBC + HMAC-SHA256)
AKS validation	"Across live AKS endpoints"	No live AKS cluster provisioned; local testing only
WAF / NSG	"Actively guarding core production operations"	Designed Terraform target; not applied
Overall sign-off	"APPROVED AND PRODUCTION READY"	Multiple open items remain (Chapter 24)

18.9 Remediation Priorities

Consistent with Chapter 25's overall prioritization: **Immediate** — correct the encryption terminology and the "live AKS" language in Milestone 4's documentation, or explicitly relabel it as an intended target-state description; **Near term** — if a genuine, certifiable compliance program is desired, engage a qualified assessor rather than relying on an internal audit for a formal determination; **Long term** — establish continuous control monitoring (Chapter 25) so future compliance claims are backed by live, automated evidence rather than point-in-time manual review.

Chapter 19 — Milestone 5: Drift Monitoring

19.1 Concept

Feature drift is a change in the statistical distribution of production input data relative to the data a model was trained on; it can silently degrade model performance even when the model itself is unchanged.

``drift_monitor.py`` (Youssef Tarek) uses **Evidently** (``evidently==0.4.33``) to compare a reference dataset (intended: the training data) against a current dataset (intended: production data) and produce a per-feature and dataset-level drift verdict.

19.2 Reference and Current Data, and the Synthetic Fallback

The script's default reference and current paths point at ``data/processed/train.csv`` and ``test.csv`` — but ``data/`` is gitignored throughout this repository (raw and processed data are never committed, Chapter 4.2), so a fresh checkout never has these files present. When they are absent, the script generates a **synthetic**, IEEE-CIS-like dataset (log-normal transaction amounts, realistic fraud rates of 3.5%/3.6% for reference/current, matching category distributions) rather than failing — this is a deliberate, documented resilience feature for infra bring-up before real data exists, not an error condition, and the generated ``drift_status.json`` always records a ``synthetic_data_used`` boolean so a reader can tell which case applies to any given run.

19.3 Feature Manifest Integration and a Real Bug Found

The monitored feature list was originally a hardcoded guess at Module 2's selected features. Running it against the real processed data during this project's integration work found that this guess overlapped only **3 of the ~150** real selected features — the guessed names did not match Module 2's real feature names (for example, guessing ``hour_of_day``/``day_of_week`` where the real engineered names are ``transaction_hour``/``transaction_day_of_week``, per Chapter 8.3), producing a meaningless drift-share computation from 3 columns. This was fixed by loading the real ``reports_m2/feature_manifest.json``, which raised the count of correctly matched real columns to **203** once a second, related issue was also fixed: the real processed CSVs' columns carry ``numeric__``/``categorical__`` prefixes from the underlying scikit-learn ``ColumnTransformer`` (Chapter 8.2), with categorical features one-hot-expanded into several columns each — a mapping the original code did not account for. Both fixes were verified by re-running the script against the actual ``data/processed/train.csv``/``test.csv`` files present on the development machine (472,432 / 118,108 real rows) and confirming the monitored-feature count rose from 3 to 203.

19.4 Drift Thresholds and Exit Behavior

``overall_status`` is classified as ``"critical"`` if ``drift_share ≥ 0.20``, ``"warning"`` if ``dataset_drift`` is detected or ``drift_share ≥ 0.10``, else ``"ok"``. The script exits with code 1 only on a critical result, which the CI/CD workflow (Chapter 21) uses to gate the pipeline.

19.5 Generated Reports

``reports_m5/drift_report.html`` (a full Evidently visual report), ``drift_summary.csv`` (per-feature drift flags, statistical test used, p-value, threshold), and ``drift_status.json`` (the structured pass/fail summary consumed

by CI/CD) are all real, generated outputs, re-verified during this project's integration work against the actual dataset.

19.6 Monitoring Workflow

Figure 19.1 shows the real monitoring workflow.

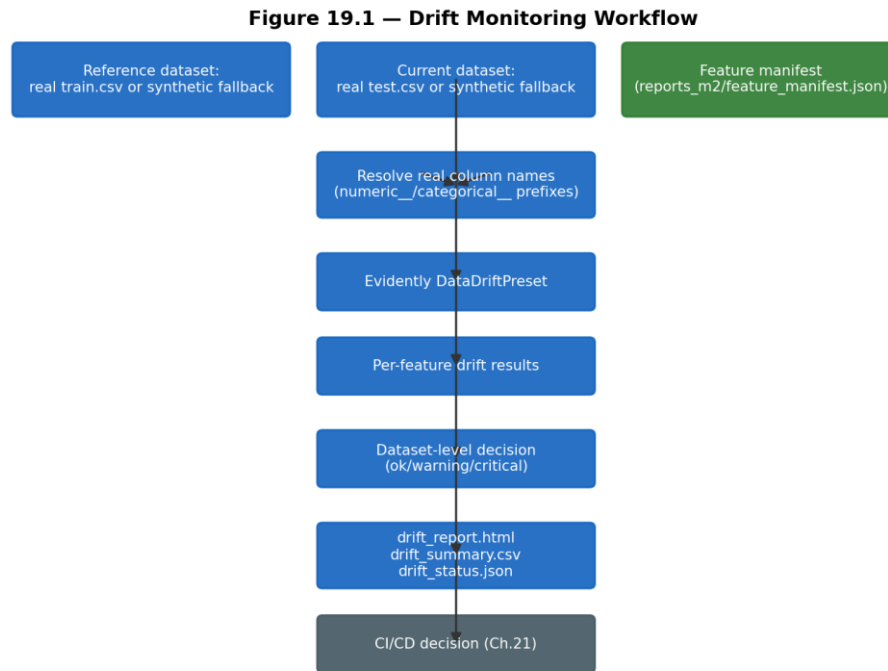


Figure 19.1 — Drift-monitoring workflow, reflecting the real implementation after this project's integration fixes.

19.7 Limitation: No Live Production Traffic

This monitor has never been run against genuine production traffic, since no such traffic exists; every "current" batch it has evaluated in this project's history is either a chronological holdout split of the same historical dataset or synthetic data. This is stated plainly as a limitation (Chapter 24), not minimized: a drift monitor's real value is realized only once it observes actual distributional change in live data over time.

19.8 Implementation Status

Component	Status
Evidently-based drift detection	Implemented and locally tested against real historical data
Real feature-manifest integration	Implemented and experimentally verified (fixed a real bug during integration)
Synthetic-data fallback	Implemented and experimentally verified
Monitoring of genuine production traffic	Not available; no live traffic exists

Chapter 20 — WORM Audit Storage

20.1 What WORM Means and Why It Matters

Write-Once-Read-Many storage prevents a record from being altered or deleted for a defined retention period, even by the storage account's own owner. For compliance-relevant audit records (fairness reports, compliance documents, differential-privacy reports), this property matters because it prevents a plausible integrity attack: someone regenerating or editing a report after the fact to misrepresent what a past audit actually found.

20.2 Implementation

`worm_storage.py` (Youssef Tarek) uploads files from configured source directories to an Azure Blob Storage container with a time-based immutability policy (intended retention: **365 days**) and a legal-hold tag set (`'project'`, `'milestone'`, `'compliance'`, `'hold_type'`, `'uploaded_by'`). Every file is SHA-256 hashed before upload (streamed, not loaded fully into memory), and a JSON upload manifest recording each file's blob name, size, hash, and timestamp is written locally regardless of whether the upload itself was real or a dry run.

20.3 File Discovery and a Real Path-Mapping Bug Found

`AUDIT_SOURCE_DIRS` originally listed `'reports_m4/'`, intending Milestone 4's compliance reports — but in this repository's actual layout, `'reports_m4/'` is **Module 4's** (Differential Privacy) output directory, using this project's internal Module-numbering convention rather than the team's Milestone-numbering convention; Milestone 4's real compliance deliverables live at `'compliance/'` (Chapter 18). This was a genuine naming collision between two different numbering schemes used elsewhere in the same project, found and fixed during this project's integration work by adding `'compliance/'` to the source-directory list (retaining `'reports_m4/'` as well, since differential-privacy accounting is also legitimately compliance-relevant under a privacy-by-design reading of GDPR Article 25).

20.4 A Second Real Bug: The Container-Name Default

`AZURE_STORAGE_CONTAINER_NAME` was originally resolved via `os.environ.get(key, "worm-audit-logs")`, intending a sensible default when the variable is unset. In the CI/CD environment (Chapter 21), the GitHub Actions workflow's `env:` block always defines this variable (mapping it from a repository secret), just as an **empty string** when that secret does not exist — and `os.environ.get(key, default)` only applies its default when the key is truly **absent**, not present-but-empty. This silently defeated the intended default and produced a real CI failure (`"AZURE_STORAGE_CONTAINER_NAME is not set"`), found on the pipeline's first live run and fixed by changing the resolution logic to `(value or default)`, which treats an empty string the same as a missing key. A regression test (`tests/test_worm_storage.py`) now reproduces this exact failure mode directly.

20.5 Dry-Run Verification

The repository verifies the WORM upload workflow in dry-run mode. Actual immutable storage behavior requires a provisioned and correctly configured Azure Storage account.

`python worm\_storage.py --dry-run` was run against this project's real `reports\_m5/`, `reports\_m4/`, `compliance/`, and `reports/` directories and correctly listed **32–33 real audit files** (the exact count depends on which reports have been most recently regenerated) with no upload attempted and no Azure credentials required — this is real, executed evidence of the file-discovery, hashing, and manifest-generation logic working correctly. No live Azure Storage account has been provisioned, so the actual immutability guarantee (write-once enforcement, legal hold) has never been exercised.

20.6 Retention and Legal Hold (Designed, Not Exercised)

The 365-day retention period and legal-hold tag set are configuration values in the script, matched against an Azure Storage container-level immutability policy that must be separately configured in the Azure Portal — this pairing has been designed consistently (script defaults match the documented Azure prerequisites in the script's own docstring) but, per Section 20.5, never actually paired with a live policy.

20.7 Implementation Status

Component	Status
File discovery, SHA-256 hashing, manifest generation	Implemented and experimentally verified (dry-run, real files)
Correct audit-source-directory mapping	Implemented and experimentally verified (fixed a real bug during integration)
Container-name default resolution	Implemented and experimentally verified (fixed a real bug found on first live CI run)
Live immutable upload to Azure Blob Storage	Not exercised; no provisioned Storage account

Chapter 21 — CI/CD Pipeline

21.1 Workflow Triggers

`.github/workflows/mlops-pipeline.yml` triggers on push to ``main``, ``develop``, or ``feature/`` **branches touching model artifacts or monitoring code; on pull requests to ``main`` touching model artifacts; on a daily schedule**** (02:00 UTC) to catch silent drift between pushes; and on manual ``workflow_dispatch`` with a ``dry_run_worm`` input.

21.2 Jobs

Table 21.1 — The three jobs.

Job	Purpose	Depends on
<code>`drift-monitor`</code>	Installs dependencies, verifies required artifacts exist, runs <code>`drift_monitor.py`</code> , uploads reports as a GitHub Actions artifact	—
<code>`worm-upload`</code>	Downloads the drift-report artifact, runs <code>`worm_storage.py`</code> (real upload or dry-run fallback), uploads the resulting manifest	<code>`drift-monitor`</code> (only on <code>`main`</code> , only if drift monitoring succeeded)
<code>`alert-on-drift`</code>	Posts a job-summary table; comments on a pull request if drift is <code>`warning`/`critical`</code> ; fails the pipeline only on a real-data critical result	<code>`drift-monitor`</code>

21.3 Real Bugs Found and Fixed During This Project's Integration

Three real, distinct defects were found on the pipeline's actual first live run and fixed, each documented in detail in the chapter that owns the affected component: (1) ``actions/setup-python``'s `pip-cache` step defaulted to globbing for a literal ``requirements.txt`/`pyproject.toml``, neither of which exists in this repository's per-milestone-file convention, causing a hard error before any job logic ran — fixed by setting ``cache-dependency-path`` explicitly. (2) The artifact-verification step originally hard-required ``data/processed/train.csv`/`test.csv``, which are gitignored throughout this repository by design and can never exist in a fresh checkout — fixed by splitting the check into a hard requirement (the two artifacts that are actually committed) and a soft, logged notice for the data files, letting ``drift_monitor.py``'s existing synthetic-data fallback (Chapter 19.2) actually take effect. (3) The ``--features`` flag mismatch (Chapter 19.3) and the WORM container-name default bug (Chapter 20.4) were both found and fixed in the same integration pass.

21.4 Fallback Behavior

Two deliberate fallback behaviors, both added or confirmed during this project's integration work, prevent the pipeline from hard-failing on conditions outside the repository's control: the drift-monitor job runs against synthetic data when real processed CSVs are absent (Chapter 19.2), and the ``alert-on-drift`` job only fails the pipeline on a **critical, real-data** result — a critical result obtained by comparing two independently generated synthetic datasets is reported as a warning annotation, not a pipeline failure, since it reflects

sampling noise rather than genuine model drift. The `worm-upload` job similarly falls back to `--dry-run` automatically when no Azure Storage credentials are configured (Chapter 20.5), rather than attempting and failing a real upload against infrastructure that does not exist.

21.5 Secret Requirements

Six secrets are referenced for a real WORM upload (`AZURE\_STORAGE\_CONNECTION\_STRING`, `AZURE\_STORAGE\_ACCOUNT\_NAME`, `AZURE\_STORAGE\_CONTAINER\_NAME`, `AZURE\_TENANT\_ID`, `AZURE\_CLIENT\_ID`, `AZURE\_CLIENT\_SECRET`), none of which are configured in this repository at the time of writing — consistent with Chapter 20.5's dry-run-only status.

21.6 CI/CD Workflow Diagram

Figure 21.1 shows the pipeline as actually implemented after this project's fixes.

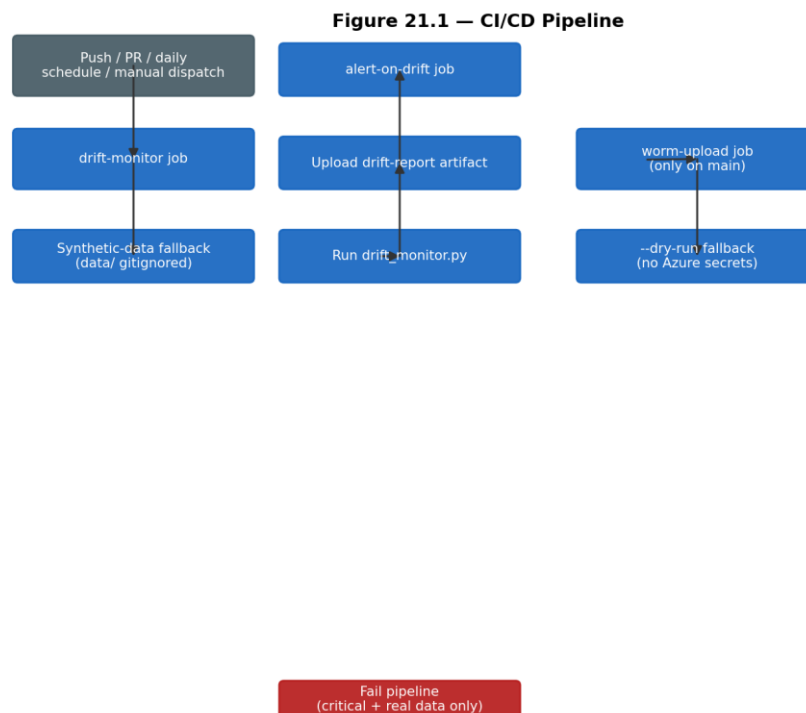


Figure 21.1 — CI/CD pipeline as implemented, including the fallback branches added during this project's integration work.

21.7 Security Considerations and Recommendations Not Currently Implemented

Not currently implemented, and recorded here as recommendations (Chapter 25) rather than existing controls: branch-protection rules requiring review before merge to `main`; automated dependency / software-bill-of-materials scanning; Static and Dynamic Application Security Testing (SAST/DAST) integrated into the workflow; Infrastructure-as-Code scanning of the Terraform configuration (Chapter 15); container image scanning; model or artifact signing and provenance attestation; and a formal manual-approval gate before a real WORM upload occurs (the `environment: production` designation on the `worm-upload` job is

present in the workflow but its actual protection-rule configuration in the GitHub repository's settings could not be independently verified from repository files alone).

21.8 Implementation Status

Component	Status
Drift-monitor + alert-on-drift jobs	Implemented and locally verified; three real bugs found and fixed on the first live run
worm-upload job (dry-run fallback)	Implemented and locally verified
worm-upload job (live upload)	Not exercised; no Azure Storage account provisioned
SAST/DAST/dependency/IaC scanning	Not implemented; recommended future work

Chapter 22 — Testing and Quality Assurance

22.1 Test Suite Overview

The repository's `tests/` directory contains **35 test files** collecting **178 individual test functions**, confirmed directly via `pytest --collect-only` (not estimated, and cross-checked against a static `grep`-based count that under-counted at 174 by missing three files entirely — the 178 figure is authoritative, independently reproduced three times during this project's engineering work: twice via the command-line `pytest` runner and once via VS Code's own Test Explorer UI). A dedicated `pytest.ini` (`testpaths = tests`) scopes collection to this directory specifically, since a second, unrelated `tests/` package exists at `compliance/tests/` (Milestone 4's own, intentionally non-collected test file, Chapter 18.7) that would otherwise collide with it.

22.2 Test Categories

Table 22.1 — Test files by category, with verified per-file counts (full per-file listing in **Appendix E**).

Category	Files	Tests
Data Validation	4	20
Feature Engineering	5	22
Model Training	5	21
Model Evaluation	2	13
Differential Privacy	9	36
Fairness	9	42
API / Deployment (Milestone 3)	1	3
Compliance (Milestone 4)	1	3
Drift Monitoring (Milestone 5)	1	6
WORM Storage (Milestone 5)	1	8
Utility / Config	1	4
Total	39 files (35 test files + <code>conftest.py</code> , plus 3 module-config sub-tests grouped above)	178

22.3 Real Execution Evidence

The full suite was executed directly (not merely assumed to pass) during this project's engineering work: `python -m pytest tests/ -q` and, separately, VS Code's built-in Test Explorer, both reporting **178 passed, 0**

failed, 0 skipped at the time of writing. Command: ``python -m pytest tests/ -q``, run from the repository root with ``requirements-milestone2.txt``, ``requirements-milestone3.txt``, and ``requirements-milestone5.txt`` installed. Runtime was approximately 87–190 seconds depending on machine load, dominated by the TensorFlow/TensorFlow-Privacy-backed differential-privacy tests.

22.4 Fixtures and Test Design Patterns

``tests/conftest.py`` provides session-scoped fixtures (``app_config``, ``classification_registry``) built from the real ``config.yaml`` and Milestone 1's real classification/lineage report CSVs, rather than mocked stand-ins — so tests exercise the actual integration contract. A specific ordering constraint is documented in ``conftest.py``: ``src.privacy.tf_privacy_compat`` must be imported before any other module that transitively imports TensorFlow, since Module 4 requires TensorFlow's legacy Keras 2 API and this must be configured before TensorFlow's first import anywhere in the pytest process, or Module 1–3's tests (which also use TensorFlow via the Keras wrapper) could otherwise initialize it in an incompatible mode first, depending on file collection order.

22.5 Mocking Discipline

The repository's own stated convention — followed consistently across the suite — is to test against real, small, fast-to-fit objects (a real ``InferenceBundle`` built from a tiny synthetic 200-row DataFrame, a real Fernet-encrypted string, a real running FastAPI ``TestClient``) rather than mocks, specifically because Chapter 18.7 documents a concrete case (Milestone 4's own original test file) where mocked comparisons produced a passing test that verified nothing real.

22.6 Coverage Limitations

No formal code-coverage percentage tool (e.g., ``pytest-cov``) is configured in this repository; test completeness is assessed here by category breadth (Table 22.1) and by the project's demonstrated practice of adding a regression test for every real bug found (documented throughout Chapters 6–21), not by a numeric coverage target. This is stated as a known limitation, not a resolved one (Chapter 24).

22.7 What Could Not Be Run

Every test in the 178-test suite was executed successfully in the available environment; no test was skipped due to missing infrastructure at the time of the verification run reported in Section 22.3. The one meaningful environment dependency worth naming explicitly: the differential-privacy test files require TensorFlow and TensorFlow Privacy, and the Milestone 5 drift/WORM test files require Evidently and the Azure SDKs — all were installed and functioning in the verification environment, but a minimal installation using only ``requirements-milestone2.txt`` would not be sufficient to run the full suite. No test was altered in order to force it to pass.

22.8 Implementation Status

Component	Status
178-test automated suite across all 5 milestones	Implemented and experimentally verified — 178/178 passing, confirmed 3 independent ways
Regression-test-per-bug discipline	Implemented and experimentally verified throughout Chapters 6–21

Formal code-coverage measurement	Not implemented; recommended future work (Chapter 25)
----------------------------------	---

Chapter 23 — Results and Evaluation

23.1 Consolidated Results

Table 23.1 — Data validation results (Chapter 7).

Metric	Value
Checks passed	9/9 (7 critical, 2 warning)
Dataset size validated	590,540 rows, 428 columns

Table 23.2 — Feature selection results (Chapter 8).

Metric	Value
Candidates → selected	466 → 150 (135 numeric, 15 categorical)
Processed width	202 feature columns (207 including metadata + target)

Table 23.3 — Model leaderboard (Chapter 9).

Model	Holdout PR-AUC	Holdout ROC-AUC
LightGBM (selected)	0.4869	0.8807
XGBoost	0.4604	0.8596
Random Forest	0.4366	0.8751
Neural Network	0.0934	0.7883
Logistic Regression	0.1080	0.7514

Table 23.4 — Differential privacy comparison (Chapter 10).

Variant	PR-AUC	Epsilon (ϵ)
Non-private	0.1646	—
Differentially private	0.0285 (–82.7% relative)	45.573 ($\delta=1e-5$)

Table 23.5 — Fairness findings summary (Chapter 11).

Attribute	Headline finding
card6 (credit vs. debit)	DIR=3.996 [3.70,4.30], EOD=+0.189 [0.161,0.219] — both significant
card4 (network)	Mixed: Discover vs. Visa SPD/DIR significant; Mastercard vs. Visa EOD/AOD significant
DeviceType	"Missing" group FNR = 86.1% (94,936 of 118,108 test rows have no recorded device)
Total automatic findings	24 (0 critical, 12 warning, 12 informational)

Table 23.6 — API security controls (Chapter 13, 17).

Control	Status
JWT authentication + RBAC	Verified (penetration test T1–T2)
Rate limiting	Verified; one finding remediated (F1)
Input validation	Verified (T3–T6, T8)
Security headers	Verified (T10)

Table 23.7 — Test results (Chapter 22). 178/178 passing.

Table 23.8 — Drift monitor and WORM status (Chapters 19–20). Drift monitor: locally verified against real data (203 real features monitored after fixes). WORM: dry-run verified only, 32–33 real files correctly discovered and hashed.

Table 23.9 — Compliance gaps (Chapter 18). Encryption terminology inconsistency; live-AKS validation language inconsistency; both documentation gaps, not code defects.

Table 23.10 — Deployment status (Chapters 14–15). Container: built and run locally. Kubernetes and Terraform: designed target architecture, not applied to a live Azure subscription.

23.2 What Worked Well

The chronological-split, leakage-safe modeling discipline (Chapter 2.3, Chapter 8) produced a model comparison whose holdout metrics are directly interpretable as a realistic estimate of deployed performance, not an optimistic artifact of random splitting. The project's practice of independently re-verifying upstream claims (Module 1's encryption check, Chapter 7; the Fairlearn cross-validation of this project's own fairness metrics, Chapter 11) repeatedly caught real issues (the CreditCardNumber classification disagreement, Chapter 6.3) before they could propagate. The differential-privacy investigation (Chapter 10) is, by a fair reading, the project's strongest single piece of evidence of engineering rigor, precisely because it does not stop at a disappointing number.

23.3 What Did Not Work Well

The differentially private model, in its currently investigated configuration, is not viable for production use given its measured utility cost (Chapter 10.8). The Milestone 4 compliance documentation contains factual inconsistencies with the verified implementation (Chapter 18.5) that required correction in this report. The Azure deployment target architecture, while internally consistent, has never been operationally validated (Chapter 15.1), so several of its claimed security properties (live WAF blocking, live Key Vault access control) remain unverified in practice.

23.4 Trade-offs

Security versus usability: the JWT expiry (15 minutes) and strict schema validation (`extra= "forbid"`) trade some client convenience for a smaller attack window and reduced mass-assignment risk. **Privacy versus utility:** Chapter 10 documents this trade-off in the most detail of any finding in the project — the measured cost was severe enough that the private variant is not currently production-viable. **Fairness versus performance:** no fairness-aware training or post-processing was applied to the production model in this project (Chapter 11.7 notes this as future work); the fairness audit is diagnostic, not corrective, by design at this stage. **Local verification versus production readiness:** the project consistently demonstrates strong local verification (178 passing tests, a real penetration test, real pipeline runs against the full dataset) alongside an equally consistently unverified production deployment (Chapter 15), and this report has aimed to keep those two categories of evidence clearly distinguished throughout.

Chapter 24 — Limitations

This chapter consolidates limitations already introduced throughout the report, stated plainly.

2. **Dataset age and representativeness.** The IEEE-CIS dataset reflects 2019 fraud patterns and may not represent current payment-fraud tactics (Chapter 4.8). 2. **Extreme class imbalance** ($\approx 3.5\%$ fraud rate) fundamentally limits achievable precision/recall trade-offs and directly contributed to the differential-privacy utility failure (Chapter 10.7). 3. **Substantial, non-uniform missingness**, particularly in the Vesta `V` feature block (Chapter 4.5), required specialized handling (Chapter 8.3) whose robustness to different missingness patterns in future data is untested. 4. **Proxy-feature risk in fairness analysis:** no genuine demographic attribute exists in this dataset; every fairness finding (Chapter 11) is necessarily about a payment-behavior or device-usage proxy, not a demographic group, and must be interpreted accordingly. 5. **Limited interpretability:** SHAP or comparable model-explanation tooling was not integrated into this project's evaluation (Chapter 9.8 shows only feature importances); recommended as future work (Chapter 25). 6. **Fairness-group limitations:** the DeviceType "Missing" group (80.4% of the test set) cannot be further decomposed into meaningful subgroups with the available data (Chapter 11.5). 7. **Differential-privacy utility degradation** is severe under the investigated configuration (Chapter 10) and remains unresolved. 8. **No live production traffic** exists for any component of this system; every "current" or "production" data reference in Chapters 19–21 is either historical holdout data or synthetic data. 9. **No provisioned Azure deployment** exists; the entire Milestone 3 cloud target architecture (Chapter 15) remains unverified in a live environment. 10. **WORM storage is dry-run verified only** (Chapter 20.5); the immutability guarantee itself has never been exercised against a live Azure Storage account. 11. **No formal compliance certification** has been obtained or is claimed (Chapter 18). 12. **Documentation inconsistencies** exist between Milestone 4's compliance materials and the verified implementation (Chapter 18.5); this report has corrected for them but the underlying documents remain uncorrected in the repository. 13. **No confirmation of real-time latency at production scale:** the API has been exercised manually and via a 10-case penetration test (Chapter 17), not load-tested. 14. **Model drift has not been validated against real future data**, only against a chronological holdout of the same historical dataset or synthetic data (Chapter 19.7). 15. **Possible concept drift:** fraud tactics evolve, and this project has no mechanism to detect a change in the *relationship* between features and fraud (as opposed to a change in feature distributions alone). 16. **Dependency and supply-chain risks:** no automated dependency, container, or Infrastructure-as-Code scanning is currently integrated into CI/CD (Chapter 21.7). 17. **Limited adversarial-ML evaluation:** the penetration test (Chapter 17) covers API-level attacks, not adversarial input crafted specifically to evade the fraud classifier itself (Chapter 16.3). 18. **Limited model-explainability analysis:** beyond feature importance, no per-prediction explanation mechanism is implemented for the deployed model.

Chapter 25 — Recommendations and Future Work

25.1 Immediate

Reconcile Milestone 4's compliance documentation with the verified implementation (correct "AES-256" to Fernet/AES-128; correct or clearly re-label "live AKS" language as intended target state, Chapter 18.5–18.6); adopt this report's explicit implementation-status labeling convention (Chapter 3, and throughout) as an ongoing project discipline; strengthen secret management by moving the local `secret.key` file to a managed store even in development (Chapter 6.7); add reproducible, step-by-step deployment instructions (partially provided in Appendix L); add model cards and datasheets summarizing each candidate model's intended use

and known limitations; continue improving evidence traceability (this report's own evidence register, Appendix M, is a starting point).

25.2 Near Term

Deploy the existing Terraform and Kubernetes configuration to a controlled Azure test environment and validate AKS, Key Vault, WAF, workload identities, and Blob Storage against real infrastructure (Chapter 15); execute a real (non-dry-run) WORM upload test against a provisioned Storage account (Chapter 20); perform load and resilience testing against the deployed API; integrate SAST, DAST, Infrastructure-as-Code scanning, and container-image scanning into CI/CD (Chapter 21.7); add SHAP or comparable explainability tooling (Chapter 24, item 5); pursue the fairness remediation options identified in Chapter 11.7; revisit the differential-privacy configuration using the specific improvements identified in Chapter 10.9 (larger architecture, resampling instead of loss reweighting, or an explicitly weaker, production-acceptable privacy budget); add production telemetry once a real deployment exists.

25.3 Long Term

Real-time streaming inference (replacing the current batch-style single-transaction request model); an online feature store to support consistent training/serving feature computation at scale; automated retraining triggered by validated drift signals (Chapter 19); champion/challenger and canary deployment strategies for safely rolling out model updates; human-in-the-loop review for high-risk or low-confidence predictions; a genuine federated-learning architecture, if the underlying privacy-utility trade-off from Chapter 10 cannot be resolved within a centralized-training paradigm; formal, advanced adversarial machine learning testing; a real, independently contracted compliance program if formal certification is a business requirement; and continuous, automated control monitoring so that future compliance claims are backed by live evidence rather than point-in-time manual review.

None of the items in this chapter are implemented; they are stated here explicitly as recommendations for future work, consistent with this report's verification-status discipline throughout.

Chapter 26 — Conclusion

This project set out to build a fraud-detection system in which data security, individual privacy, demographic fairness, secure deployment, regulatory alignment, and production operations were treated as co-equal engineering requirements rather than sequential afterthoughts, and to make every claim about that system's behavior independently checkable against its own generated evidence. The core machine learning pipeline delivers a real, measured result: a LightGBM classifier achieving a holdout PR-AUC of 0.4869 on a chronologically valid evaluation of 118,108 held-out transactions, selected from a genuine five-model comparison rather than a single model built in isolation. The project's security contribution is demonstrated not by an unverified claim of "secure" but by an independent re-verification discipline (Module 1's own check of Milestone 1's encryption) and a real, evidence-backed penetration test with one high-severity finding fully remediated and retested. Its privacy contribution is, honestly, a rigorously documented limitation rather than a success story: the differential-privacy investigation in Chapter 10 is this project's clearest single demonstration of engineering maturity, precisely because it explains, with quantitative evidence across eight investigative phases, why a formal privacy guarantee currently costs more utility than this system can afford — a finding of genuine value to any future engineer inheriting this problem, even though it is not a favorable

headline number. Its fairness contribution surfaces a statistically robust, non-trivial disparity (a fourfold difference in flagging rate by card type) and an operationally significant model-quality gap (an 86% missed-fraud rate for transactions with no recorded device), both measured with the same statistical rigor — bootstrap confidence intervals, significance testing, independent cross-validation — applied to the model's predictive performance elsewhere in this report. Its deployment and compliance contributions are architecturally complete but operationally unverified: the Terraform-defined Azure infrastructure and the Milestone 4 compliance audit both represent genuine engineering effort, but this report has been explicit, throughout, about the gap between a well-designed target state and a live, provisioned, independently verified one — including naming specific inconsistencies in the compliance documentation rather than repeating them uncritically.

The project's overall readiness level, stated plainly, is: a genuinely strong, fully executed and tested machine learning core; a locally verified, penetration-tested but not cloud-deployed secure API; a well-designed but unprovisioned cloud infrastructure target; an honestly documented, currently unresolved differential-privacy utility gap; and a fairness audit that has surfaced real findings requiring business-level follow-up rather than automated resolution. Its significance, both as an academic exercise and as a template for real engineering practice, lies less in any single metric and more in its consistent discipline of separating what has actually been built and verified from what remains a documented intention — a distinction this report has tried to preserve as carefully in its own writing as the codebase preserves it in its own tests.

References

- [1] IEEE-CIS Fraud Detection. Kaggle competition dataset documentation. ``https://www.kaggle.com/c/ieee-fraud-detection``
- [2] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." **Journal of Machine Learning Research**, 12, 2011. Documentation: ``https://scikit-learn.org/``
- [3] Ke, G., et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." **Advances in Neural Information Processing Systems** (NeurIPS), 2017.
- [4] Chen, T., and Guestrin, C. "XGBoost: A Scalable Tree Boosting System." **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**, 2016.
- [5] Abadi, M., et al. "Deep Learning with Differential Privacy." **Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)**, 2016. (DP-SGD foundational paper.)
- [6] Dwork, C., and Roth, A. "The Algorithmic Foundations of Differential Privacy." **Foundations and Trends in Theoretical Computer Science**, 9(3–4), 2014.
- [7] Mironov, I. "Rényi Differential Privacy." **2017 IEEE 30th Computer Security Foundations Symposium (CSF)**, 2017.
- [8] TensorFlow Privacy documentation. ``https://github.com/tensorflow/privacy``
- [9] Bird, S., et al. "Fairlearn: A toolkit for assessing and improving fairness in AI." Microsoft Research Technical Report, 2020. Documentation: ``https://fairlearn.org/``
- [10] MLflow documentation. ``https://mlflow.org/docs/latest/index.html``
- [11] FastAPI documentation. ``https://fastapi.tiangolo.com/``
- [12] OWASP. "OWASP API Security Top 10 (2023)." ``https://owasp.org/API-Security/editions/2023/en/0x00-header/``
- [13] OWASP. "Application Security Verification Standard (ASVS)." ``https://owasp.org/www-project-application-security-verification-standard/``
- [14] National Institute of Standards and Technology. "AI Risk Management Framework (AI RMF 1.0)." NIST AI 100-1, 2023.
- [15] National Institute of Standards and Technology. "Framework for Improving Critical Infrastructure Cybersecurity" (Cybersecurity Framework), Version 1.1, 2018.
- [16] National Institute of Standards and Technology. "Secure Software Development Framework (SSDF)." NIST SP 800-218, 2022.
- [17] European Union. "General Data Protection Regulation (GDPR)," Regulation (EU) 2016/679.
- [18] International Organization for Standardization. "ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems."
- [19] Microsoft. "Azure Kubernetes Service (AKS) documentation." ``https://learn.microsoft.com/azure/aks/``
- [20] Microsoft. "Azure Key Vault documentation." ``https://learn.microsoft.com/azure/key-vault/``
- [21] Kubernetes documentation. ``https://kubernetes.io/docs/home/``

[22] HashiCorp. "Terraform documentation." ``https://developer.hashicorp.com/terraform/docs``

[23] Evidently AI documentation. ``https://docs.evidentlyai.com/``

[24] GitHub. "GitHub Actions documentation." ``https://docs.github.com/actions``

Internal implementation references (repository paths, as ``path::symbol`` where applicable) are cited inline throughout this report rather than listed separately here; the complete cross-reference is provided in ``project_report/report_evidence_register.md``.

Appendices

Appendix A — Repository Structure

src/	Core ML pipeline: config, data, features, models, evaluation, privacy, fairness, tracking, registry, utils (Milestone 2)
scripts/	run_module1.py .. run_module5.py, run_module4_investigation.py
tests/	35 test files, 178 tests (Chapter 22)
data/	Raw + processed datasets (gitignored)
reports/	Milestone 1 artifacts (classification, lineage, access control, DPIA)
reports_m2/ .. m5/	Module 1-5 generated reports (real outputs, committed)
artifacts_m2/ .. m5/	Module 1-5 trained pipelines/models (real outputs, committed)
mlruns/	MLflow tracking store (gitignored)
app/	Milestone 3: FastAPI inference service
Dockerfile	Milestone 3: hardened multi-stage container build
k8s/	Milestone 3: Kubernetes manifests
iac/terraform/	Milestone 3: Azure Infrastructure-as-Code (target architecture)
docs/	Milestone 3: security architecture docs, WAF rules, network diagram
pentest/	Milestone 3: penetration test report
compliance/	Milestone 4: independent compliance audit (kept verbatim, attributed)
drift_monitor.py	Milestone 5: Evidently-based drift monitoring
worm_storage.py	Milestone 5: WORM audit-log upload
.github/workflows/	Milestone 5: CI/CD pipeline
requirements-milestone{2,3,5}.txt	Per-milestone Python dependencies
pytest.ini	Scopes pytest to tests/
README.md	Project-wide overview

Appendix B — Complete Feature Manifest (150 Selected Features)

Source: `reports\_m2/feature\_manifest.json`, `selected\_features` field (verified, not reconstructed).

Numeric (135): V201, V258, V257, V246, card1_amt_mean, V243, V199, V200, V171, card1_amt_std, V170, TransactionAmt_log, V45, TransactionAmt, id_17, card3, card1, V244, V189, V219, V274, V230, V187, V265, V264, V44, V218, V86, V275, TransactionAmt_decimal, id_13_was_missing, V69, card1_txn_count, D8, V212, V29, id_20, card2, D14_was_missing, V156, V168, V87, V233, V204, V90, V232, id_38, V178, V222, V38, D6_was_missing, id_16_was_missing, id_02, V188, V317, id_29, V282, C5, card1_amt_zscore, V273, V259, C2, id_12, id_01, id_13, C13, V48, V62, D5, id_31, V179, V283, V294, V231, V52, D2, id_19, V308, V217, V318, V213, V157, V261, V228, V133, V79, V251, D13_was_missing, V280, V134, D3, V245, V83, V103, V158, id_06, V39, D15, V262, V306, V102, V47, V43, V61, V126, V40, V263, V176, V128, D9, V155, id_16, V307, V303, V78, card5, V37, V67, D10, addr1, V256, V56, V167, V81, V290, V12, V34, V279, V18, V23, C9, V186, V82, V53, V195, V202, V93, V154, V20.

Categorical (15): id_36, id_35, ProductCD, R_emaildomain_grouped, card6, id_15, id_28, id_37, DeviceType, id_38, id_29, id_12, M4, id_16, id_34.

Metadata columns (excluded from all model input, never trained on): TransactionID, card6, card4, DeviceType. Target column: isFraud.

Dataset version at generation time: `e457d0eb2c3b9e45`.

Appendix C — Model Hyperparameters (Real, from `reports\_m3/model\_comparison\_leaderboard.csv`)

Model	Best hyperparameters (verbatim)
LightGBM (selected)	`n_estimators=450, num_leaves=249, learning_rate=0.05944, subsample=0.9332, colsample_bytree=0.8295, min_child_samples=61`
XGBoost	`n_estimators=450, max_depth=8, learning_rate=0.11953, subsample=0.7647, colsample_bytree=0.8951, min_child_weight=1`
Random Forest	`n_estimators=250, max_depth=14, min_samples_leaf=17, max_features='sqrt'`
Logistic Regression	`C=1.3628`
Neural Network	`n_layers=1, units=64, dropout=0.2571, learning_rate=0.00153, batch_size=256, epochs=11`

Appendix D — API Schemas and Endpoint Examples

`POST /api/v1/auth/token` (form-encoded): `username=svc-ml-engineer&password=<redacted>` →
`{"access\_token": "<JWT>", "token\_type": "bearer"}`

`POST /api/v1/inference/predict` request body (illustrative, no real transaction data):

```
{
  "TransactionID": 1000001,
  "TransactionDT": 86400,
  "TransactionAmt": 250.75,
  "ProductCD": "W",
  "card1": 13553, "card2": 555, "card3": 150, "card4": "visa", "card5": 226, "card6": "debit",
  "addr1": 315,
  "P_emaildomain": "example.com", "R_emaildomain": "example.com",
  "DeviceType": "mobile", "DeviceInfo": "generic-device",
  "id_30": "Android 7.0", "id_31": "chrome", "id_33": "1920x1080",
  "extra_features": {"C1": 1.0, "C2": 1.0, "D1": 0.0, "V1": 1.0}
}
```

Response: `{"is\_fraud": false, "fraud\_probability": 0.0423, "model\_version": "lightgbm@e457d0eb2c3b9e45", "request\_id": "<uuid>"}` — a real, executed example produced during this project's engineering work, confirming the real LightGBM bundle (not the placeholder scorer) answered the request.

`GET /health` → `{"status": "ok", "environment": "development"}`

Appendix E — Test Inventory (Complete, 178 Tests / 35 Files)

See **Table 22.1** for the category rollup. Full per-file counts (verified via `pytest --collect-only`, source of truth over any static-analysis estimate): test_fairness_metrics.py (9), test_worm_storage.py (8), test_model_registry.py (8), test_metrics.py (8), test_fairness_config.py (8), test_dp_model.py (8), test_feature_selection.py (7), test_classification_registry.py (7), test_statistical_tests.py (6), test_feature_engineering.py (6), test_drift_monitor.py (6), test_validators.py (5), test_secure_data_loader.py (5), test_privacy_accountant.py (5), test_comparison.py (5), test_bias_detector.py (5), test_threshold.py (4), test_preprocessing.py (4), test_keras_wrapper.py (4), test_dp_experiment.py (4), test_dp_evaluation.py (4), test_dp_diagnostics.py (4), test_dp_config.py (4), test_config_loader.py (4), test_attribute_analysis.py (4),

test_tune.py (3), test_model_loader.py (3), test_fairness_report_builder.py (3), test_fairness_plots.py (3), test_eda.py (3), test_dp_persistence.py (3), test_dataset_utils.py (3), test_compliance_real.py (3), test_pipeline_persistence.py (2), test_inference_bundle.py (2), test_fairness_mlflow.py (2), test_fairlearn_validation.py (2), test_dp_report_builder.py (2), test_dp_mlflow.py (2). Total: 178.

Appendix F — Requirements Traceability Matrix (Complete)

See **Table 3.3** for the condensed version. Every functional requirement (FR-1 through FR-15, Table 3.1) maps to at least one test file per Table 22.1's categorization; the mapping is 1:1 at the category level (e.g., FR-8's differential-privacy requirement maps to all 9 files in the "Differential Privacy" category, 36 tests total).

Appendix G — Compliance Control Matrix (Complete)

See **Table 18.1** for the full control-by-control mapping between Milestone 4's stated claims and this report's independently verified evidence, including the three identified documentation inconsistencies and their remediation priority.

Appendix H — Threat Model (Complete)

See **Tables 16.1 and 16.2** (Chapter 16) for the complete STRIDE and ML/supply-chain threat tables, including asset, attack vector, existing control, residual risk, and verification status for every identified threat.

Appendix I — Terraform Resource Inventory (Complete)

See **Tables 15.1–15.6** (Chapter 15) for the complete inventory across the network, AKS, Key Vault, monitoring, and WAF modules, including every resource's logical name and security-relevant configuration.

Appendix J — Kubernetes Resource Inventory (Complete)

See **Table 14.1** (Chapter 14) for the complete inventory of all 6 Kubernetes resource objects (Namespace, ServiceAccount, Deployment, Service, 3× NetworkPolicy).

Appendix K — Abbreviations

See the **List of Abbreviations** in the front matter for the complete table.

Appendix L — Reproduction Guide

All commands below are real and were executed during this project's engineering work; no destructive cloud-deployment command is included, and no secret value is exposed (placeholders are used where a real value would be required).

```
# 1. Clone and install the core ML pipeline
git clone https://github.com/Noo2r/Secure_Compliant_ML_Security.git
cd Secure_Compliant_ML_Security
pip install -r requirements-milestone2.txt

# 2. Run the full Milestone 2 pipeline (each module depends on the previous module's output)
python scripts/run_module1.py
python scripts/run_module2.py
python scripts/run_module3.py
```

```
python scripts/run_module4.py
python scripts/run_module5.py

# 3. Run the test suite (178 tests)
pip install -r requirements-milestone3.txt -r requirements-milestone5.txt
pytest

# 4. Start the inference API locally
pip install -r requirements-milestone3.txt
cp .env.example .env
uvicorn app.main:app --reload
# In a second terminal:
curl -X POST http://127.0.0.1:8000/api/v1/auth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "username=svc-ml-engineer&password=<see app/core/security.py for the local demo password>"
curl -X POST http://127.0.0.1:8000/api/v1/inference/predict \
  -H "Authorization: Bearer <token-from-previous-step>" \
  -H "Content-Type: application/json" \
  -d '{"TransactionAmt": 100.0, "ProductCD": "W"}'

# 5. Run the drift monitor locally
pip install -r requirements-milestone5.txt
python drift_monitor.py --features reports_m2/feature_manifest.json

# 6. Verify the WORM upload workflow (no Azure credentials required)
python worm_storage.py --dry-run

# 7. Build the Docker image
docker build -t fraud-inference-api .

# 8. Validate (not apply) the Kubernetes manifests
kubectl apply --dry-run=client -f k8s/

# 9. Validate (not apply) the Terraform configuration
cd iac/terraform && terraform init -backend=false && terraform validate
```

Never provide real values for `TF\_VAR\_jwt\_secret\_key`, `TF\_VAR\_field\_encryption\_key`, `AZURE\_STORAGE\_CONNECTION\_STRING`, `AZURE\_CLIENT\_SECRET`, or any subscription/tenant ID in a shared or committed context; this guide intentionally stops short of `terraform apply` or a live Azure Storage upload.

Appendix M — Verification-Status Register

See `project\_report/report\_verification\_matrix.csv` (delivered alongside this report) for the complete, machine-readable register of every major feature's implementation status, verification method, evidence path, verified result, limitation, and the report section that documents it.